第01章 QT初步
1.1 QT是什么
1.2 QT的发展历程
1.3 QT开发的优势
1.3.1 Windows 下的 GUI 库
1.3.2 Linux 下的 GUI 库
1.3.3 题外话
1.4 QT的下载安装
1.4.1 QT的下载
1.4.2 Linux下的安装
1.4.3 windows下安装
1.5第一个QT程序
1.6 可视化UI设计
1.7 代码化UI设计
1.7.1 信号与槽
1.7.2 更彻底的代码化UI设计
1.8 混合方式UI设计
第02章 常用界面设计组件
2.1窗体
2.1.1 设置窗体位置、大小及背景颜色
2.1.2 设置窗体标题
2.1.3 多窗体调用
2.2 字符串与输入输出组件
2.2.1 字符串与数值之间的转换
2.2.2 QString的常用功能
2.3 数字输入和显示组件
2.3.1 QSpinBox与QDoubleSpinBox
2.3.2其它数值输入和显示组件
2.4 按钮组件
2.5 布局组件
2.6 容器组件
2.6.1 QGroupBox
2.6.2 QScrollArea
2.6.3 QToolBox
2.6.4 QTabWidget
2.6.5 QStackedWidget
2.7 时间日期与定时器
2.7.1 时间日期相关的类
2.7.2 日期时间数据与字符串之间的转换
2.7.3 QCalendarWidget日历组件
2.7.4 定时器
3. 对话框与多窗体设计
3.1 标准对话框

3.1.1 QFileDialog对话框
3.1.2 QColorDialog对话框
3.1.3 QFontDialog对话框
3.1.4 QInputDialog标准输入对话框
3.1.5 QMessageBox消息对话框
3.2 自定义对话框及其调用
3.3 Splash与登录窗口
4. 文件操作
4.1 文本文件读写
4.1.1QFile读写文本文件
4.1.2 QFile和QTextStream结合读写文本文件
4.2 二进制文件读写
4.3 文件目录操作
5. 数据库编程
5.1 数据库概述
5.2 数据库操作指令
5.3 sql语句
5.4 Qt中使用SQLite
5.5 编程实例
5.5.1 数据库相关类的简单使用
5.5.2 相关类出错信息的捕获
5.5.3 获取单条记录
5.6 员工管理系统
6. Qt事件处理
6.1 Qt事件概述
6.1.1 Qt事件的表示
6.1.2 Qt事件的处理顺序
6.2 覆盖过滤器eventFilter
6.3 覆盖事件处理函数
6.4 综合编程实例:飞舞的雪花
7. 多线程编程
7.1 多线程的创建
7.1.1继承QThread
7.1.2 使用moveToThread
7.2 线程间同步
7.2.1 线程同步基础
7.2.2 QMutex
7.2.3 QReadWriteLock
7.2.4 QWaitCondition
7.2.5 QSemaphore
7.3综合实例
8.网络编程
8.1 主机信息查询

8.2 TCP通信
8.2.1 QTcpServer
8.2.2 QTcpSocket
8.2.3 网络聊天室编程实例
8.3 UDP通信
8.3.1 QUdpSocket
8.3.2 编程实例
8.4 http下载器的实现
9.多媒体编程
9.1 播放音频
9.1.1 命令行的mp3播放器
9.1.2 带UI的mp3播放器
9.2 播放视频
10.项目的发布
10.1 window10系统中项目的发布
10.1.1 使用windeployqt发布Qt 应用程序
10.1.2 制作安装文件
10.2 ubuntu系统中项目的发布

第01章 QT初步

1.1 QT是什么

Qt是一个跨平台的C++开发库， 主要用来开发图形用户界面（Graphical User Interface，GUI）程序 。 Qt 还存在 Python、Ruby、Perl 等脚本语言的绑定，也就是说可以使用脚本语言开发基于 Qt 的程序。

之所以称之为跨平台是因为Qt 支持的操作系统有很多，例如通用操作系统 Windows、Linux、Unix、Android、IOS、WinPhone、QNX、VxWorks等等。



Qt 虽然经常被当做一个 GUI 库，用来开发图形界面应用程序，但这并不是 Qt 的全部；Qt 除了可以绘制漂亮的界面（包括控件、布局、交互），还包含很多其它功能，比如多线程、访问数据库、图像处理、音频视频处理、网络通信、文件操作等，这些 Qt 都已经内置了。

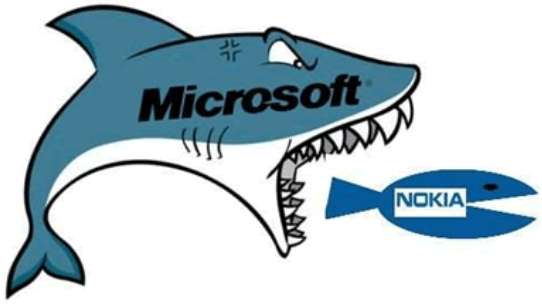
1997年，Qt 被用来开发 Linux 桌面环境 KDE，大获成功，使 Qt 成为 Linux 环境下开发 C++ GUI 程序的事实标准。 WPS、YY语音、Skype、豆瓣电台、虾米音乐、淘宝助理、千牛、暴雪的战网客户端、VirtualBox、Opera、咪咕音乐、Google地图、Adobe Photoshop Album 等知名软件都是使用Qt开发的。

Qt 虽然也支持手机操作系统，但是由于Android本身已经有了Java和Kotlin，IOS本身已经有了Objective-C 和 Swift，所以 Qt 在手机领域的市场份额几乎可以忽略。 总起来说，Qt 主要用于桌面程序开发和嵌入式开发。

1.2 QT的发展历程

Qt 最早是 1991 年由挪威的 Eirik Chambe-Eng 和 Haavard Nord 开发的， 他们随后于 1994 年 3 月 4 号正式成立奇趣科技公司（Trolltech）。Qt 原本是商业授权的跨平台开发库， 在 2000 年奇趣科技公司为开源社区发布了遵循 GPL（GNU General Public License）许可证的开源版本。

在 2008 年，诺基亚公司收购了奇趣科技公司，并增加了 LGPL（GNU Lesser General Public License）的授权模式。诺基亚联合英特尔利用 Qt 开发了全新的智能手机系统 MeeGo，可惜遭遇了微软木马屠城，诺基亚被迫放弃了 MeeGo，而 Qt 商业授权业务也于 2011 年 3 月出售给了芬兰 IT 服务公司 Digia。



Digia 于 2014 年 9 月宣布成立 Qt Company 全资子公司，独立运营 Qt 商业授权业务。目前 Qt 公司大力推广移动平台开发和商业应用， 总的来说 Qt 历经曲折，现在算是步入正轨了。

经过 20 多年的发展，Qt 已经成为最优秀的跨平台开发框架之一，在各行各业的项目开发中得到广泛应用。

1.3 QT开发的优势

世界上的 GUI 库多如牛毛，有的跨平台，有的专属于某个操作系统；有的只有 UI 功能，有的还融合了网络通信、多媒体处理、数据库访问等底层功能。

1.3.1 Windows 下的 GUI 库

Windows 下的 GUI 解决方案比较多：

- 基于 C++ 的有 Qt、MFC、WTL、wxWidgets、DirectUI、Htmlayout；
- 基于 C# 的有 WinForm、WPF；
- 基于 Java的有 AWT、Swing；
- 基于 Pascal 的 有Delphi；
- 基于Go语言的有 walk 和 electron；
- 还有国内初露头角的 aardio；
- Visual Basic 曾经很流行，现在逐渐失去了色彩；
- 如果你有 Web 开发经验，也可以基于 Webkit 或 Chromium 将网页转换为桌面程序。

没有哪一种方案能够独霸 Windows，使用比较多的编程语言是 C++、C#、Java。

用 Qt 来开发 Windows 桌面程序有以下优点：

- 简单易学：Qt 封装的很好，几行代码就可以开发出一个简单的客户端，不需要了解 Windows API。
- 资料丰富：资料丰富能够成倍降低学习成本，否则你只能去看源码，关于 DirectUI、Htmlayout、aardio 的资料就很少。
- 漂亮的界面：Qt 很容易做出漂亮的界面和炫酷的动画，而 MFC、WTL、wxWidgets 比较麻烦。
- 独立安装：Qt 程序最终会编译为本地代码，不需要其他库的支撑，而 Java 要安装虚拟机，C# 要安装 .NET Framework。
- 跨平台：如果你的程序需要运行在多个平台下，同时又希望降低开发成本，Qt 几乎是必备的。

Qt 和 MFC的对比：

- MFC 只能应用在 Windows 平台，而 Qt 是跨平台的，一次编写，到处运行。

- Qt 已经封装了底层细节，学习 Qt 将会非常简单；而 MFC 只是给 Windows API 加了一层包装，不了解 Windows API 也学不好 MFC

1.3.2 Linux 下的 GUI 库

Linux 下常用的 GUI 库有基于 C++ 的 Qt、GTK+、wxWidgets，以及基于 Java 的 AWT 和 Swing。其中最著名的就是 Qt 和 GTK+：KDE 桌面系统已经将 Qt 作为默认的 GUI 库，Gnome 桌面系统也将 GTK+ 作为默认的 GUI 库。

相比 GTK+，Qt 的功能更加强大，更新也很快，比较受人们追捧。

	GTK+	Qt	wxWidgets	AWT/Swing
开发语言	C++	C++	C++	Java
其它编程语言接口	•	•	•	–
平台支持				
Windows	•	•	•	•
Linux	•	•	•	•
Mac OS X	•	•	•	•
许可协议	LGPL	LGPL	WXL*	?
运行速度	☆☆☆☆☆	☆☆☆☆	☆☆☆☆☆	☆☆
学习成本	☆☆	☆☆	☆☆	☆☆☆
UI布局方式				
代码布局	•	•	•	•
图形化工具	•	•	•	•
工具库	一般	非常丰富	较丰富	非常丰富

注意：wxWidgets使用wxWindows License，比GPL更加宽松

1.3.3 题外话

Qt4 时代的主流就是传统组件（或叫控件）编程，所用的语言一般是 C++。 Qt5 诞生之时，正是手机移动设备蓬勃发展的时候, 为了适应手机移动应用开发， Qt5 将 QML 脚本编程提到与传统 C++ 组件编程相同的高度，力推 QML 界面编程，当然 QML 主要用于手机移动应用程序。 QML 包含大量使用手机移动设备的功能模块，比如基本组件（QtQuick 模块）、GPS 定位、渲染特效、蓝牙、NFC、WebKit 等等。

QML 类似于网页设计的 HTML，是一种标记语言，我们可以借助 CSS 对它进行美化，也可以借助 JavaScript 进行交互。有 Web 开发经验的读者学习 QML 将非常轻松。

使用 QML 开发界面主要有以下几个优点：

- QML 非常灵活，可以做出非常炫酷的效果，例如 QQ、360、迅雷等都不在话下。

- QML 是标记语言，见名知意，非常容易编写和阅读，大大提高了开发和维护效率。
- QML 界面简洁大气，有很多动画，适合移动端。
- 不同平台下的 QML 使用相同的渲染机制，界面效果一致，不会随操作系统的不同而变化。

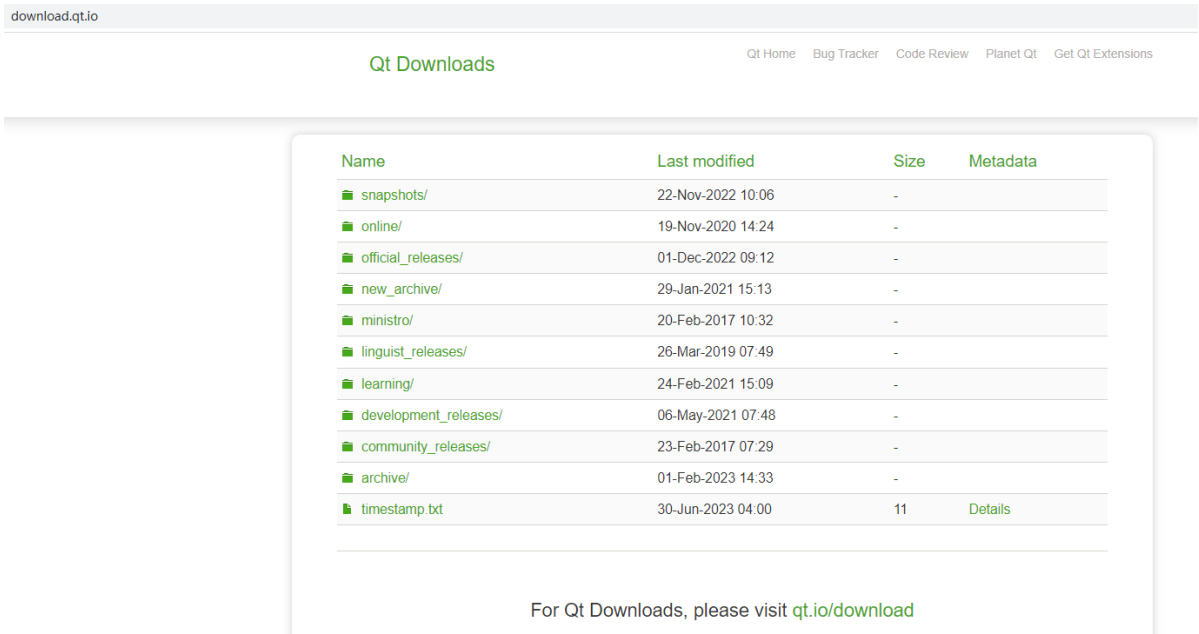
但 QML 只能用来进行界面设计和人机交互，也就是只能胜任 UI 部分，在底层仍然需要调用 C++ 编写的组件来完善功能，比如访问数据库、网络通信、多线程多进程、文件读写、图像处理、音频视频处理等都离不开 C++。 总的来说，C++ 对于 Qt 是不可或缺的，而 QML 只是一个加分项。

1.4 QT的下载安装

1.4.1 QT的下载

- Qt 官方下载（慢）

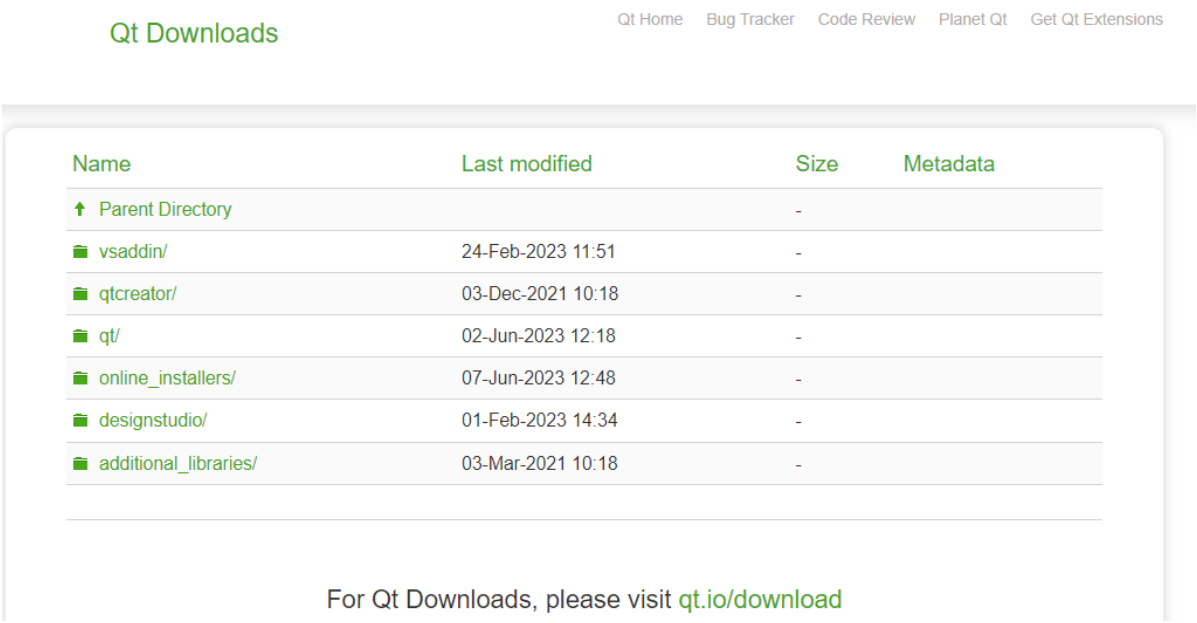
  Qt 官网有一个专门的资源下载网站，所有的开发环境和相关工具都可以从这里下载，具体地址是：http://download.qt.io/ 。



目录	说明
archive	各种 Qt 开发工具安装包，新旧都有（可以下载 Qt 开发环境和源代码）。
community_releases	社区定制的 Qt 库，Tizen 版 Qt 以及 Qt 附加源码包。
development_releases	开发版，有新的和旧的不稳定版本，在 Qt 开发过程中的非正式版本。

目录	说明
learning	有学习 Qt 的文档教程和示范视频。
ministro	迷你版，目前是针对 Android的版本。
official_releases	正式发布版，是与开发版相对的稳定版 Qt 库和开发工具（可以下载Qt开发环境和源代码）。
online	Qt 在线安装源。
snapshots	预览版，最新的开发测试中的 Qt 库和开发工具。

archive 和 official_releases 两个目录都有最新的 Qt 开发环境安装包，我们以 archive 目录里的内容为例来说明。点击进入 archive 目录，会看到六个子目录：



目录	说明
vsaddin	这是 Qt 针对 Visual Studio 集成的插件，本教程基本不使用 Visual Studio ，所以不需要插件。
qtcreator	这是 Qt 官方的集成开发工具，但是 qtcreator 本身是个空壳，它没有编译套件和 Qt 开发库。 除了老版本的 Qt 4 需要手动下载 qtcreator、编译套件、Qt 开发库进行搭配之外，一般用不到。 Qt 5 有专门的大安装包，里面包含开发需要的东西，并且能自动配置好。
qt	这是 Qt 开发环境的下载目录， Qt 5 的大安装包就在这里面。

目录	说明
online_installers	在线安装器，国内用户不建议使用，在线安装是龟速，还经常断线。我们教程采用的全部是离线的大安装包。
designstudio	UI设计开发工具相关。
additional_libraries	附加库文件

我们再进入 qt 子目录，看到如下列表：

Name	Last modified	Size	Metadata
Parent Directory		-	
6.5/	09-Jun-2023 14:59	-	
6.4/	16-Mar-2023 07:27	-	
6.3/	12-Sep-2022 12:49	-	
6.2/	09-Jun-2023 14:59	-	
6.1/	01-Sep-2021 10:42	-	
6.0/	04-May-2021 07:38	-	
5.15/	09-Jun-2023 14:59	-	
5.14/	31-Mar-2020 09:14	-	
5.13/	31-Oct-2019 07:17	-	
5.12/	25-Nov-2021 08:02	-	
5.1/	28-Jan-2021 16:26	-	
5.0/	28-Jan-2021 16:48	-	
4.8/	09-Mar-2021 10:54	-	
4.7/	28-Jan-2021 16:25	-	
4.6/	28-Jan-2021 17:11	-	
4.5/	29-Jan-2021 15:07	-	
4.4/	28-Jan-2021 16:01	-	
4.3/	28-Jan-2021 16:06	-	
4.2/	28-Jan-2021 15:52	-	
4.1/	28-Jan-2021 16:08	-	
4.0/	28-Jan-2021 16:07	-	
3/	28-Jan-2021 17:15	-	
2/	28-Jan-2021 16:07	-	
1/	28-Jan-2021 15:52	-	

可以看到QT的大版本已经到了6时代，国内有些Qt项目使用了Qt5的版本，甚至是Qt4的版本。本课程将演示Qt5离线、Qt6在线安装的方式。

- 国内镜像站下载（快）

  这里给大家推荐几个国内著名的 Qt 镜像网站，主要是各个高校的：

  - 中国科学技术大学：http://mirrors.ustc.edu.cn/qtproject/

- 清华大学：https://mirrors.tuna.tsinghua.edu.cn/qt/
- 北京理工大学：http://mirror.bit.edu.cn/qtproject/
- 中国互联网络信息中心：https://mirrors.cnnic.cn/qt/

1.4.2 Linux下的安装

- 前置条件

```
sudo apt install libfontconfig1-dev libfreetype6-dev libx11-dev
libx11-xcb-dev libxext-dev libxfixes-dev libxi-dev libxrender-dev
libxcb1-dev libxcb-cursor-dev libxcb-glx0-dev libxcb-keysyms1-dev
libxcb-image0-dev libxcb-shm0-dev libxcb-icccm4-dev libxcb-sync-
dev libxcb-xfixes0-dev libxcb-shape0-dev libxcb-randr0-dev
libxcb-render-util0-dev libxcb-util-dev libxcb-xinerama0-dev
libxcb-xkb-dev libxkbcommon-dev libxkbcommon-x11-dev

sudo apt install g++ mesa-utils libgl1-mesa-dev libssl-dev
libncurses5-dev unzip cmake ninja-build git gcc-multilib
libclang-dev
```

在线安装Qt 6

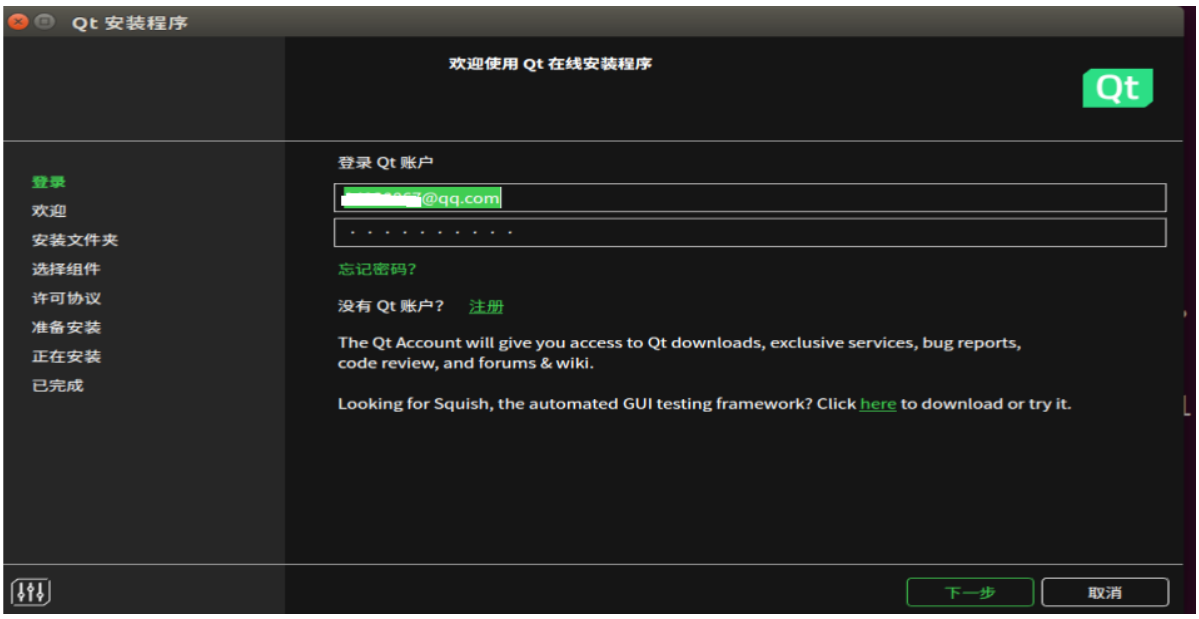
- 使用国内镜像下载在线安装工具



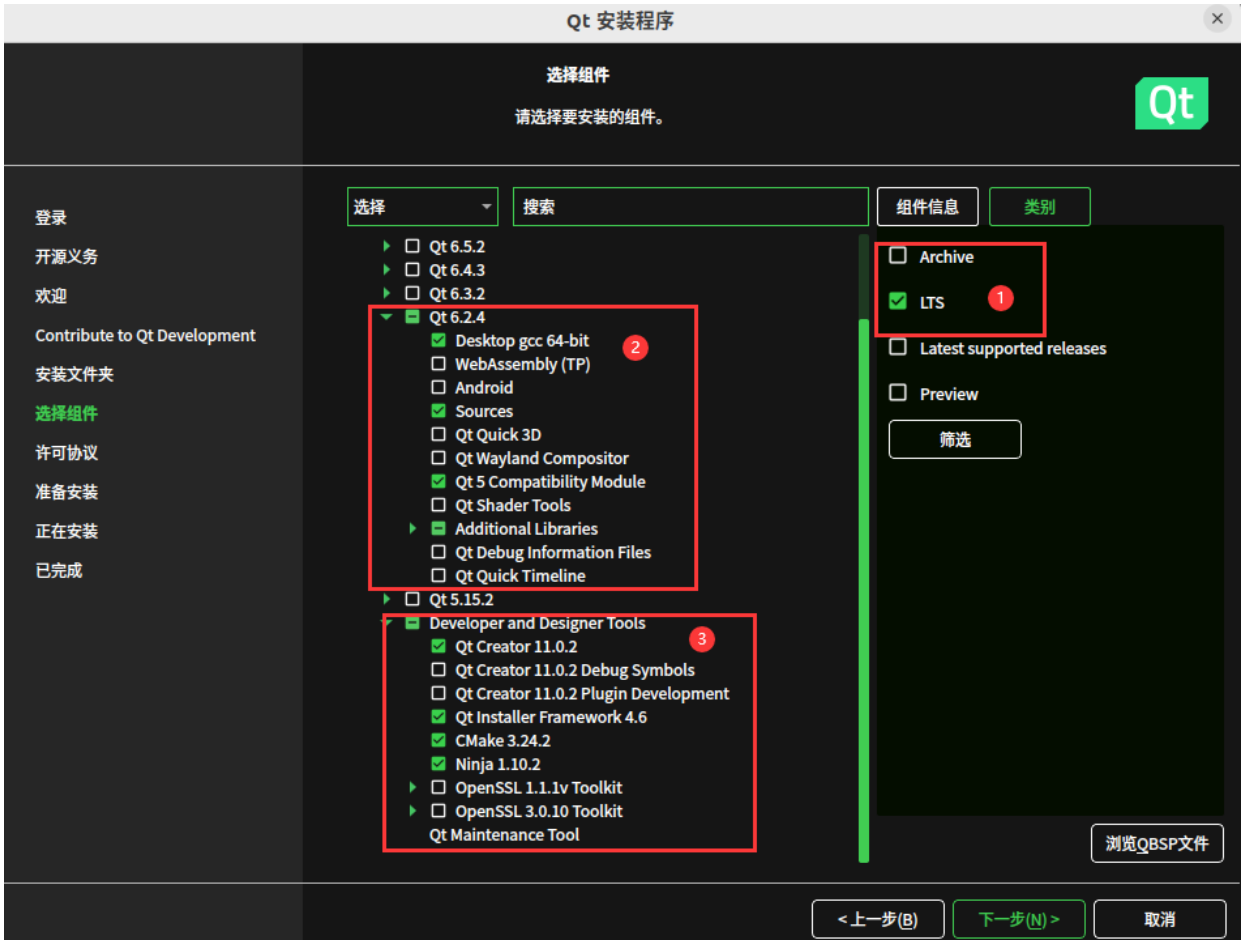
- 在ubuntu环境启动安装工具

```
cp /mnt/hgfs/swap/qt-unified-linux-x64-4.6.0-online.run ./  #按
你自己的方式拿到ubuntu系统中来
sudo apt install --reinstall libxcb-xinerama0 #安装依赖的库
#指定国内镜像源速度会快很多
sudo ./qt-unified-linux-x64-4.6.0-online.run  --mirror
https://mirror.nju.edu.cn/qt
```

下图所示步骤需要输入在QT官网注册的用户名和密码，如果未注册，可以前往https://login.qt.io/register页面完成注册。


Qt 安装程序
欢迎使用 Qt 在线安装程序
登录
欢迎
安装文件夹
选择组件
许可协议
准备安装
正在安装
已完成
登录 Qt 账户
@qq.com
忘记密码?
没有 Qt 账户? 注册
The Qt Account will give you access to Qt downloads, exclusive services, bug reports, code review, and forums & wiki.
Looking for Squish, the automated GUI testing framework? Click here to download or try it.
下一步
取消

下图为组件选择


Qt 安装程序
选择组件
请选择要安装的组件。
登录
开源义务
欢迎
Contribute to Qt Development
安装文件夹
选择组件
许可协议
准备安装
正在安装
已完成
选择
搜索
组件信息
类别
Qt 6.5.2
Qt 6.4.3
Qt 6.3.2
Qt 6.2.4
Desktop gcc 64-bit
WebAssembly (TP)
Android
Sources
Qt Quick 3D
Qt Wayland Compositor
Qt 5 Compatibility Module
Qt Shader Tools
Additional Libraries
Qt Debug Information Files
Qt Quick Timeline
Qt 5.15.2
Developer and Designer Tools
Qt Creator 11.0.2
Qt Creator 11.0.2 Debug Symbols
Qt Creator 11.0.2 Plugin Development
Qt Installer Framework 4.6
CMake 3.24.2
Ninja 1.10.2
OpenSSL 1.1.1v Toolkit
OpenSSL 3.0.10 Toolkit
Qt Maintenance Tool
Archive
LTS
1
Latest supported releases
Preview
筛选
浏览QBSP文件
2
3
< 上一步(B)
下一步(N) >
取消



一路下一步，即可进入安装过程。



为了后续启动qt开发环境方便可以将其对应的路径添加到PATH环境变量

```
vim ~/.bashrc
    #在文件的最后添加
    export
PATH=$PATH:/opt/Qt/6.2.4/gcc_64/bin:/opt/Qt/Tools/QtCreator/bin
source ~/.bashrc
qtcreator
```

启动qtcreator时如果出现以下警告，

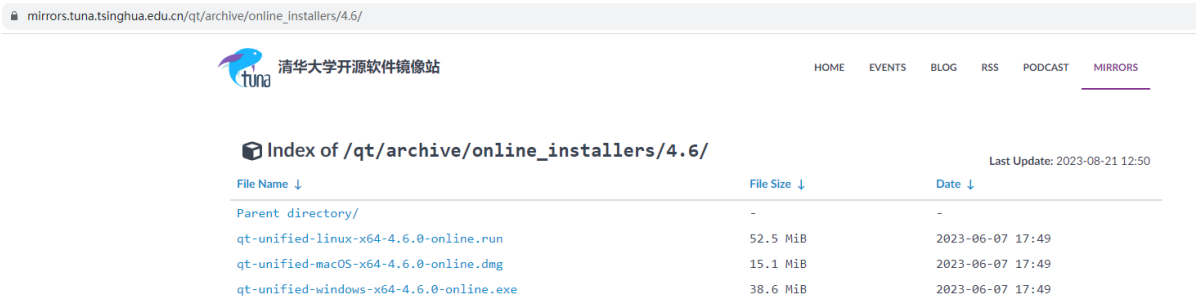
Ignoring XDG_SESSION_TYPE=wayland on Gnome. Use QT_QPA_PLATFORM=wayland to run

可以通过修改/etc/gdm3/custom.conf文件解决

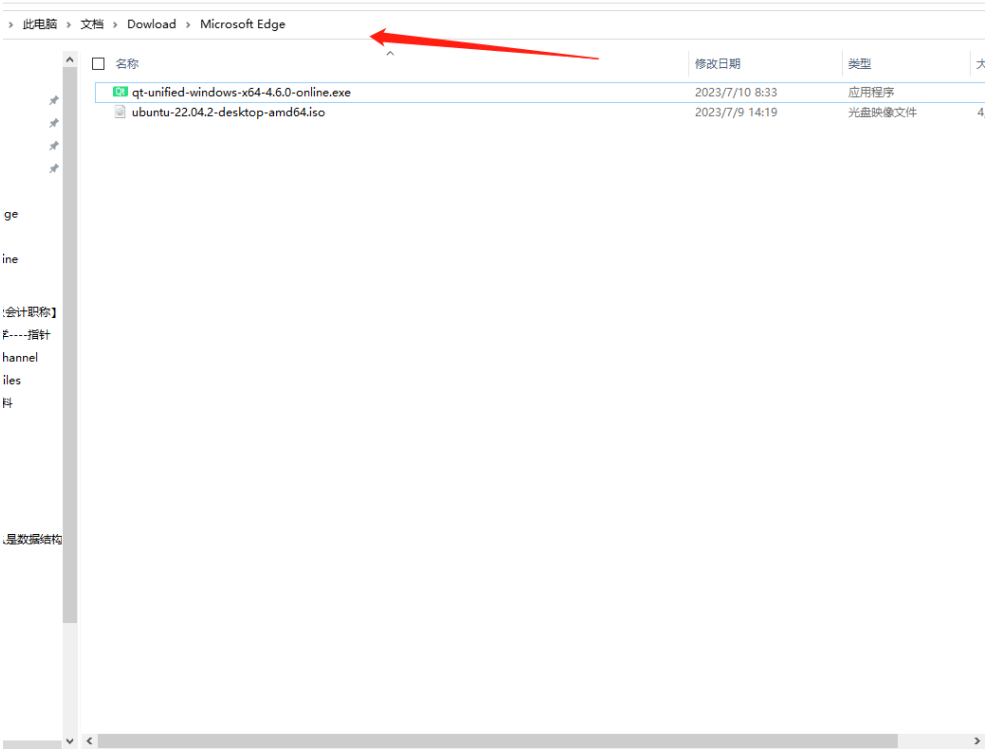
```
sudo vim /etc/gdm3/custom.conf
    WaylandEnable=false    #去掉了开头的#
sudo reboot
```

1.4.3 windows下安装

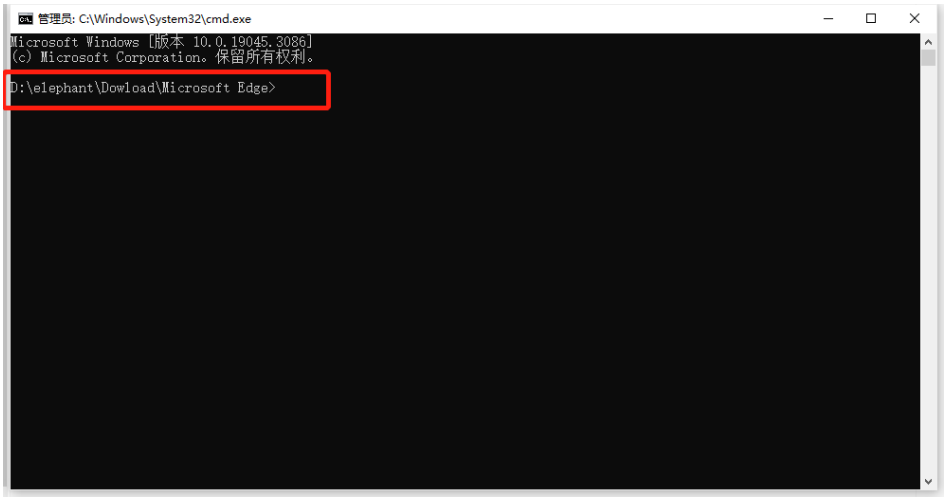
- 使用国内镜像下载在线安装工具



- 进入工具所在目录启动终端



在箭头指向位置输入"cmd",效果如下图所示。

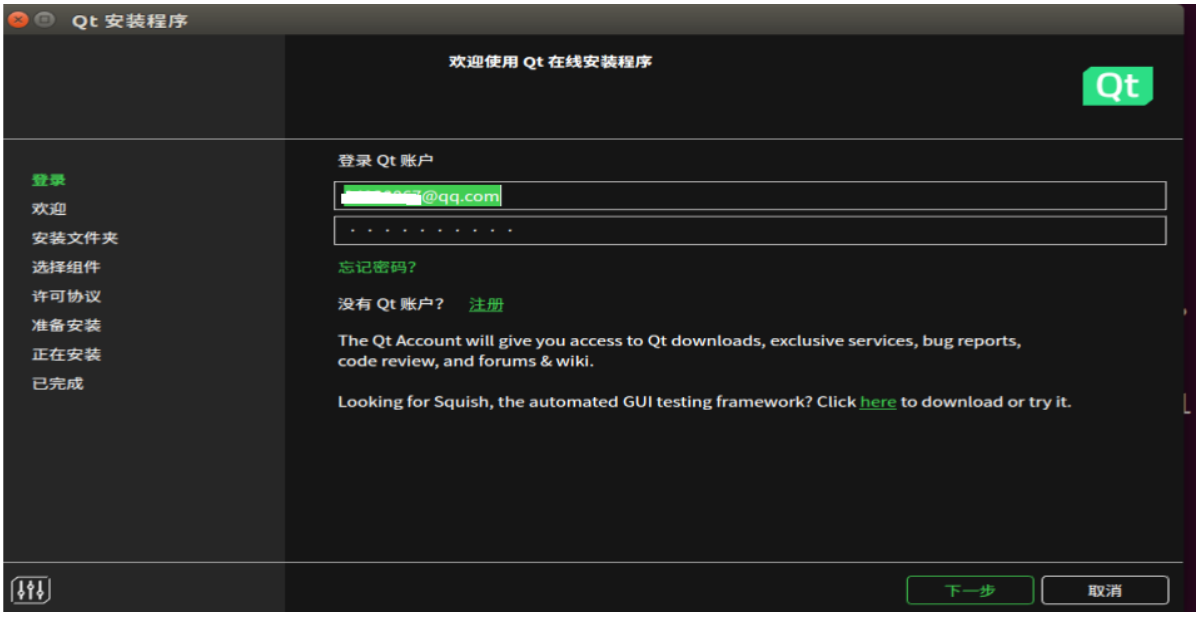


- 启动在线安装器

```
qt-unified-windows-x64-4.6.0-online.exe --mirror
https://mirror.nju.edu.cn/qt
```

- 填写用户名密码 完成安装过程

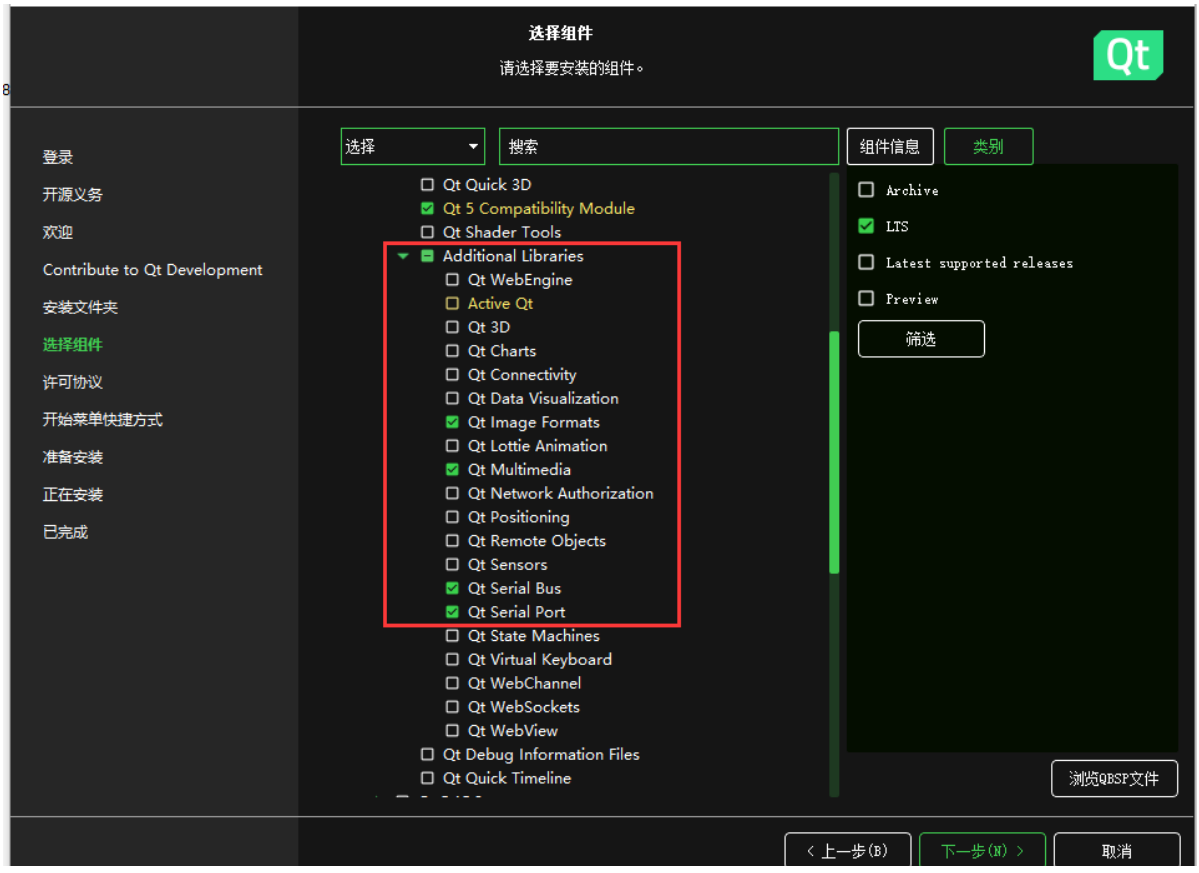
  下图所示步骤需要输入在QT官网注册的用户名和密码，如果未注册，可以前往https://login.qt.io/register页面完成注册。



  组件选择如下图所示（注意红色方框中内容的选择，其它保持默认即可）。



其中Additional Libraries的选择如下图所示，（当然土豪磁盘空间够大的话可以把这些附加库都安装上）。



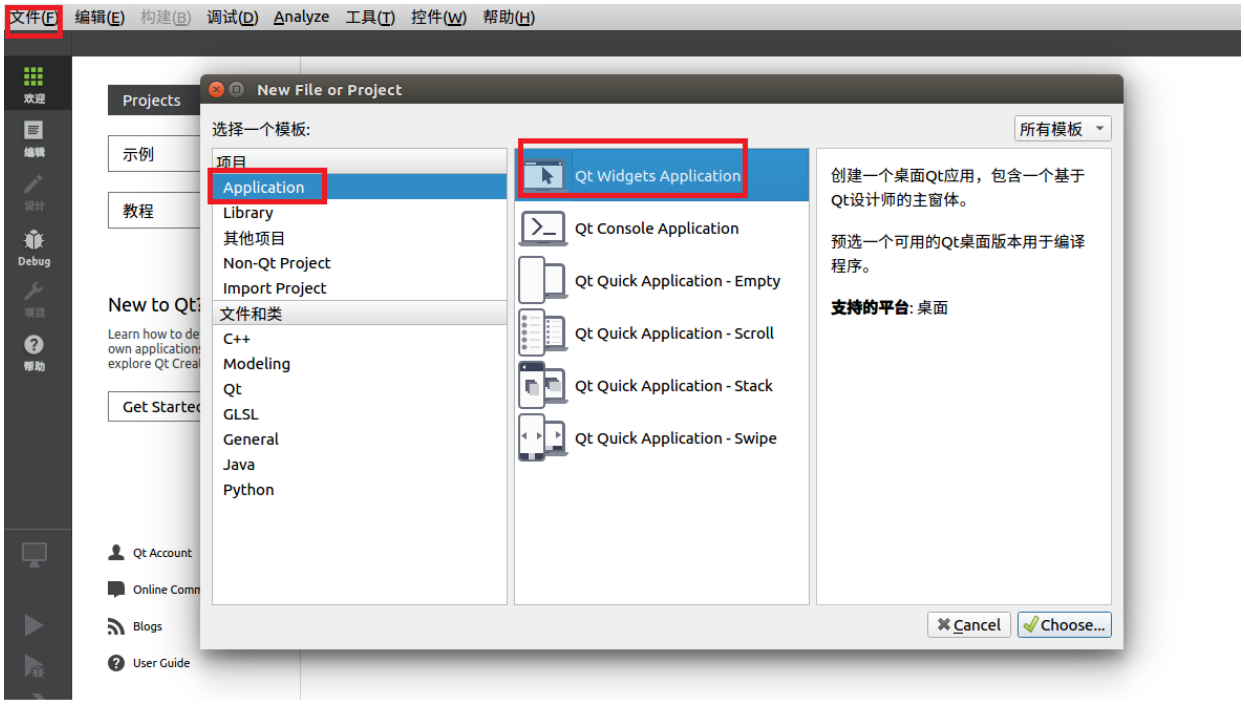


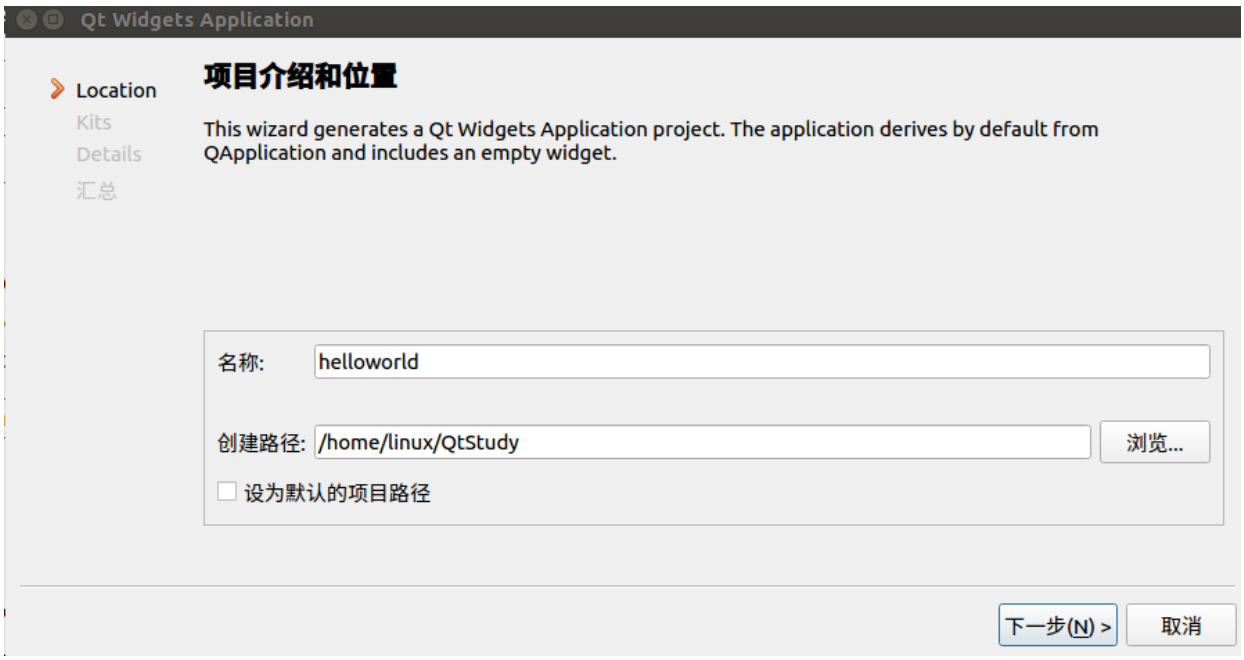
1.5第一个QT程序

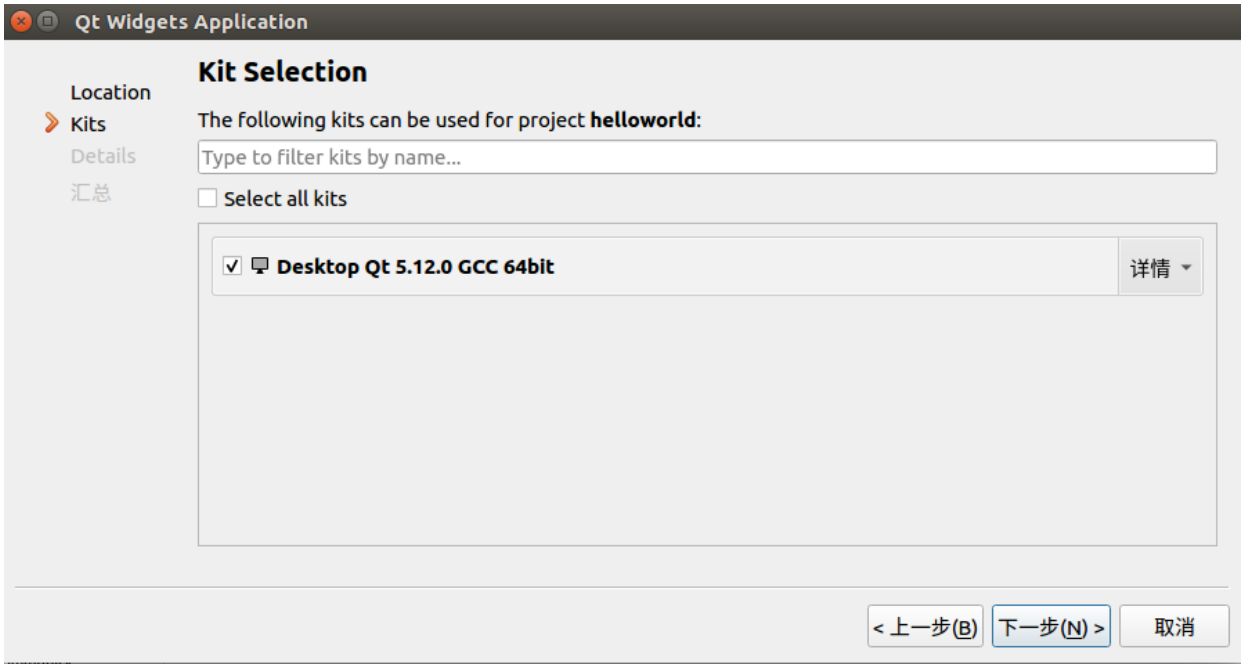
启动qtcreator

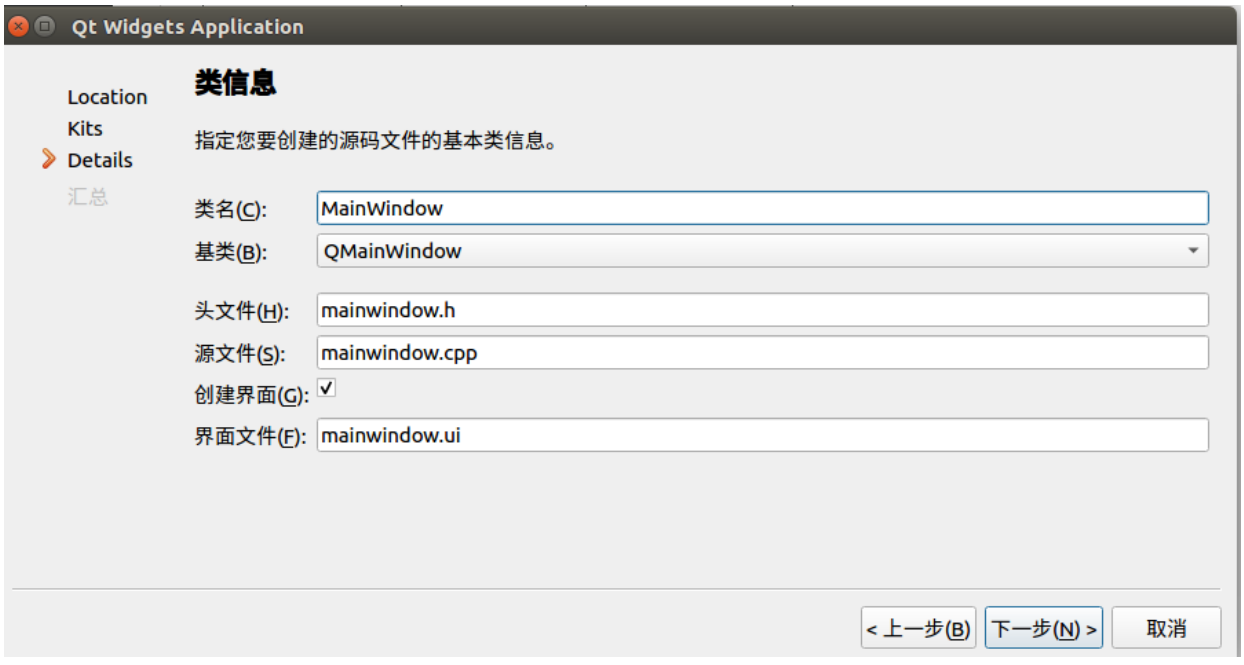
```
qtcreator
```

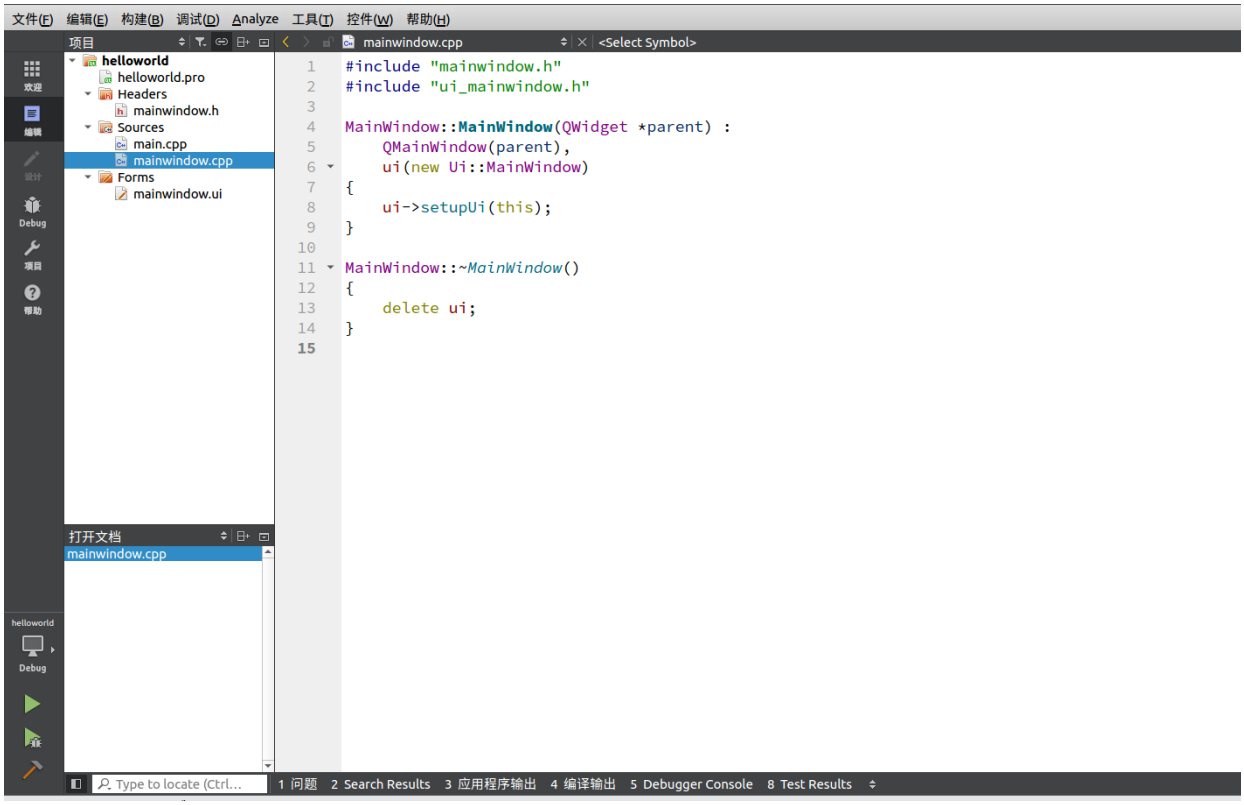
选择 “文件-->新建文件或项目”，如下图所示




Qt Widgets Application
Location
Kits
Details
汇总
项目介绍和位置
This wizard generates a Qt Widgets Application project. The application derives by default from QApplication and includes an empty widget.
名称:
helloworld
创建路径:
/home/linux/QtStudy
浏览...
设为默认的项目路径
下一步(N) >
取消


Qt Widgets Application
Location
Kits
Details
汇总
Kit Selection
The following kits can be used for project helloworld:
Type to filter kits by name...
Select all kits
Desktop Qt 5.12.0 GCC 64bit
详情
< 上一步(B)
下一步(N) >
取消


Qt Widgets Application
Location
Kits
Details
汇总
类信息
指定您要创建的源码文件的基本类信息。
类名(C):
MainWindow
基类(B):
QMainWindow
头文件(H):
mainwindow.h
源文件(S):
mainwindow.cpp
创建界面(C):
界面文件(F):
mainwindow.ui
< 上一步(B)
下一步(N) >
取消

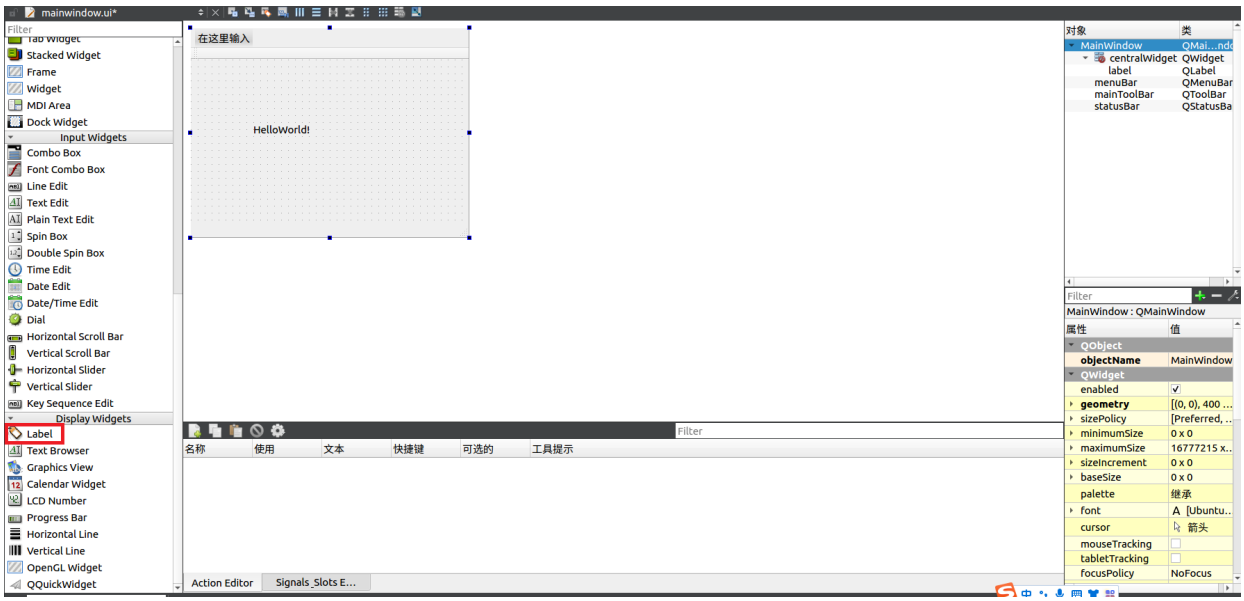


上图窗口左侧有上下两个子窗口，上方的目录树显示了项目内文件的组织结构，显示当前项目为helloworld。项目的名称构成目录树的一个根节点，Qt Creator 可以打开多个项目，但是只有一个活动项目，活动项目的项目名称节点用粗体字体表示。

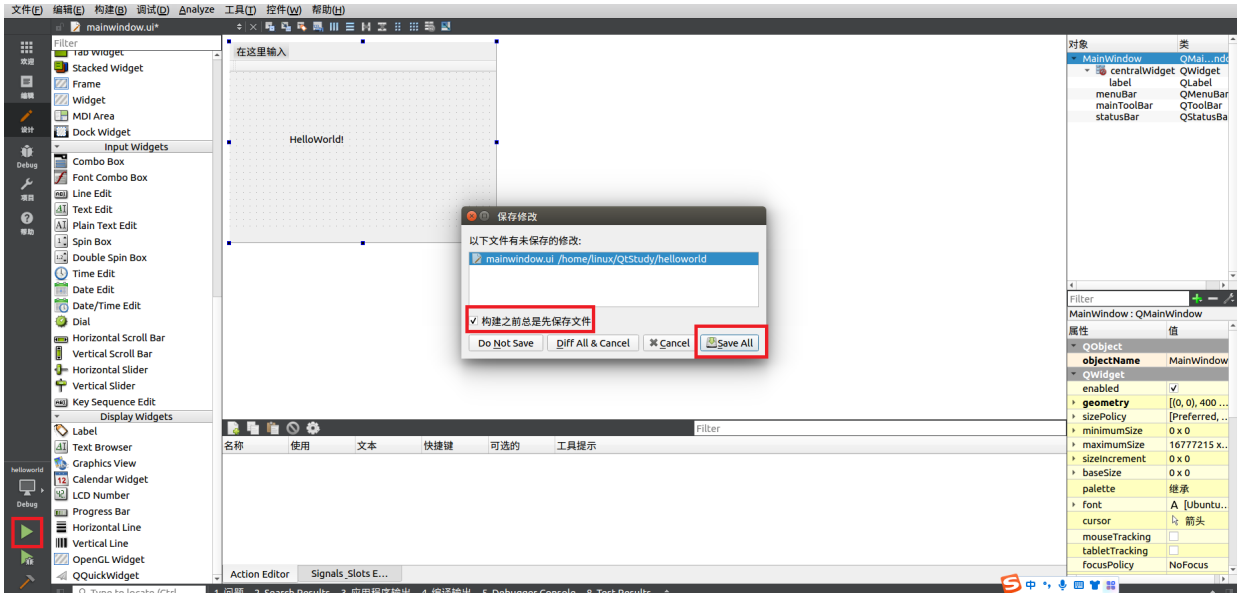
在项目名称节点下面，分组管理着项目内的各种源文件，几个文件及分组分别为以下几项：

- helloworld.pro 是项目管理文件，包括一些对项目的设置项。
- Headers 分组，该节点下是项目内的所有头文件（.h），上图中所示项目有一个头文件 mainwindow.h，是主窗口类的头文件。
- Sources 分组：该节点下是项目内的所有 C++源文件（.cpp），上图中所示项目有两个 C++ 源文件，mainwindow.cpp 是主窗口类的实现文件，与 mainwindow.h 文件对应。main.cpp 是主函数文件，也是应用程序的入口。
- Forms 分组：该节点下是项目内的所有界面文件（.ui）。上图中所示项目有一个界面文件mainwindow.ui，是主窗口的界面文件。界面文件是文本文件，使用 XML 语言描述界面的组成。

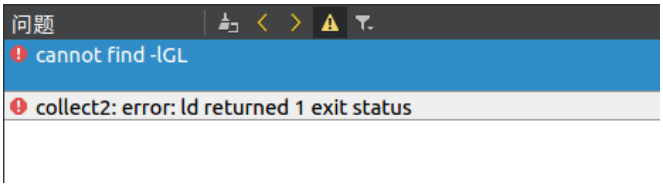
双击文件目录树中的文件mainwindow.ui，出现如下图所示的窗体设计界面：



这个界面实际上是 Qt Creator 中集成的 Qt Designer。窗口左侧是分组的组件面板，中间是设计的窗体。在组件面板的 Display Widgets 分组里，将一个Label组件拖放到设计的窗体上面。双击刚刚放置的 Label 组件，可以编辑其文字内容，将文字内容更改为“HelloWorld!”。还可以在窗口右下方的属性编辑器里编辑标签的 Font 属性，勾选粗体。



出现如下问题



“cannot find -lGL” 问题的解决方式

- 首先需要判断系统中是否安装了libGL

```
linux@ubuntu:~$ locate libGL    #出现以下类似信息说明安装了该库
```

```
/usr/lib/x86_64-linux-gnu/libGLEW.so.1.13
/usr/lib/x86_64-linux-gnu/libGLEW.so.1.13.0
/usr/lib/x86_64-linux-gnu/libGLEWmx.so.1.13
/usr/lib/x86_64-linux-gnu/libGLEWmx.so.1.13.0
/usr/lib/x86_64-linux-gnu/libGLU.so.1
/usr/lib/x86_64-linux-gnu/libGLU.so.1.3.1
/usr/lib/x86_64-linux-gnu/libhybris-egl/libGLESv2.so.2
/usr/lib/x86_64-linux-gnu/libhybris-egl/libGLESv2.so.2.0.0
/usr/lib/x86_64-linux-gnu/mesa/libGL.so.1
/usr/lib/x86_64-linux-gnu/mesa/libGL.so.1.2.0
```

- 若判断该库存在，可以通过以下方式解决错误问题

```
sudo ln -s /usr/lib/x86_64-linux-gnu/mesa/libGL.so.1
/usr/lib/libGL.so
```

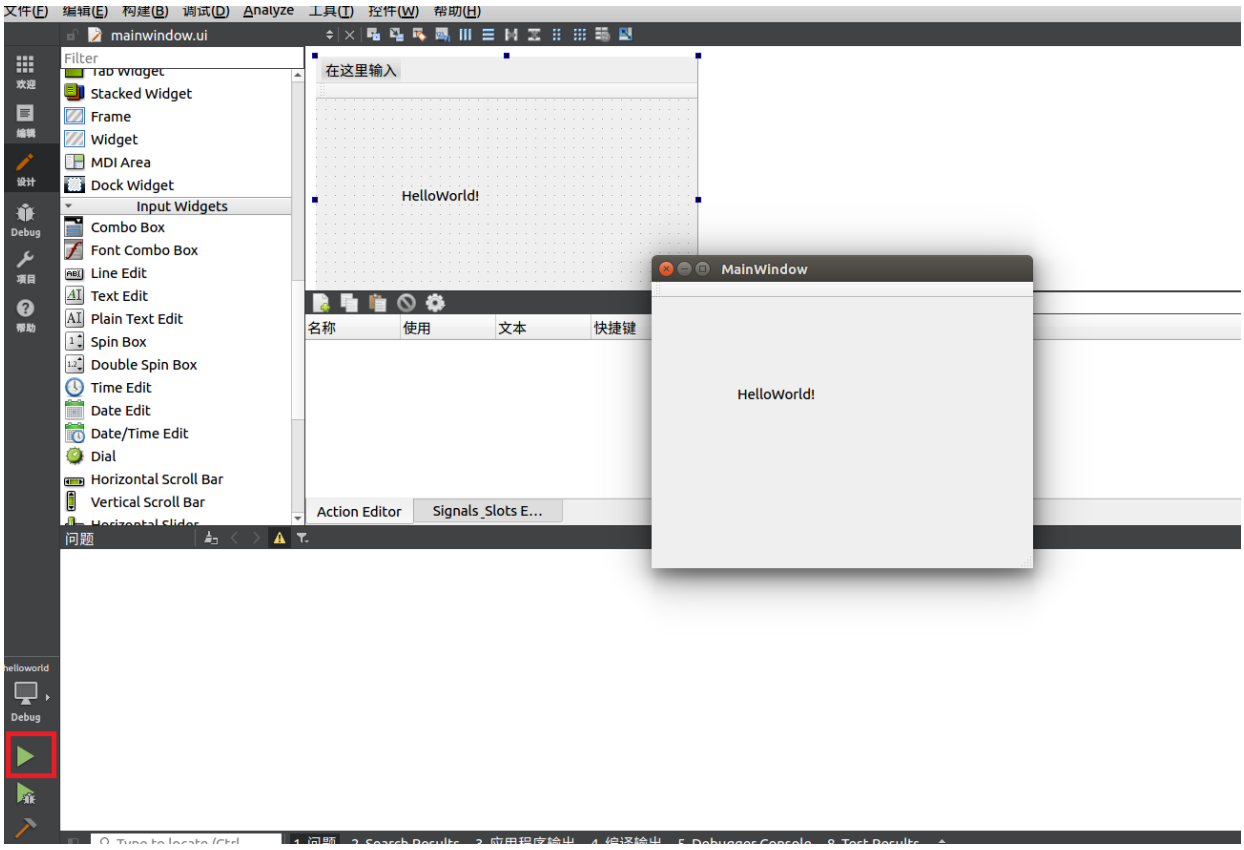
- 若该库不存在，需要下载安装该库

```
sudo apt-get install libgl1-mesa-dev
```

重新编译QT程序，如果报错，需要执行

```
sudo ln -s /usr/lib/x86_64-linux-gnu/mesa/libGL.so.1
/usr/lib/libGL.so
```

再次点击qtcreator环境中的，运行效果如下图所示。



qtcreator环境中不能输入中文的问题

这里建议使用系统自带ibus输入法框架， 不要安装搜狗输入法了。

- 安装中文语言

  在“设置setting”-“区域与语言Region&Language”-“管理已安装的语言Manage Install Languages”-“添加或删除语言Install/Remove Languages”中，安装中文简体

- 选择ibus框架

  在“设置setting”-“区域与语言Region&Language”-“管理已安装的语言Manage Install Languages”-“键盘输入法系统”中，选择ibus

- 应用

  点击“应用到整个系统”，然后关闭

- 安装ibus的pinyin

  ```
  sudo apt install ibus-libpinyin
  ```

- 在ibus首选项中添加中文输入法

  ```
  ibus-setup
  ```

  在ibus首选项中，点击“输入法”，添加“中文”-“智能拼音”，然后关闭。

- reboot

  重启电脑。

Tips:

Qt 官方的开发环境安装包里有自己专门的开发工具， 这里将常用的 Qt 开发工具列表如下：

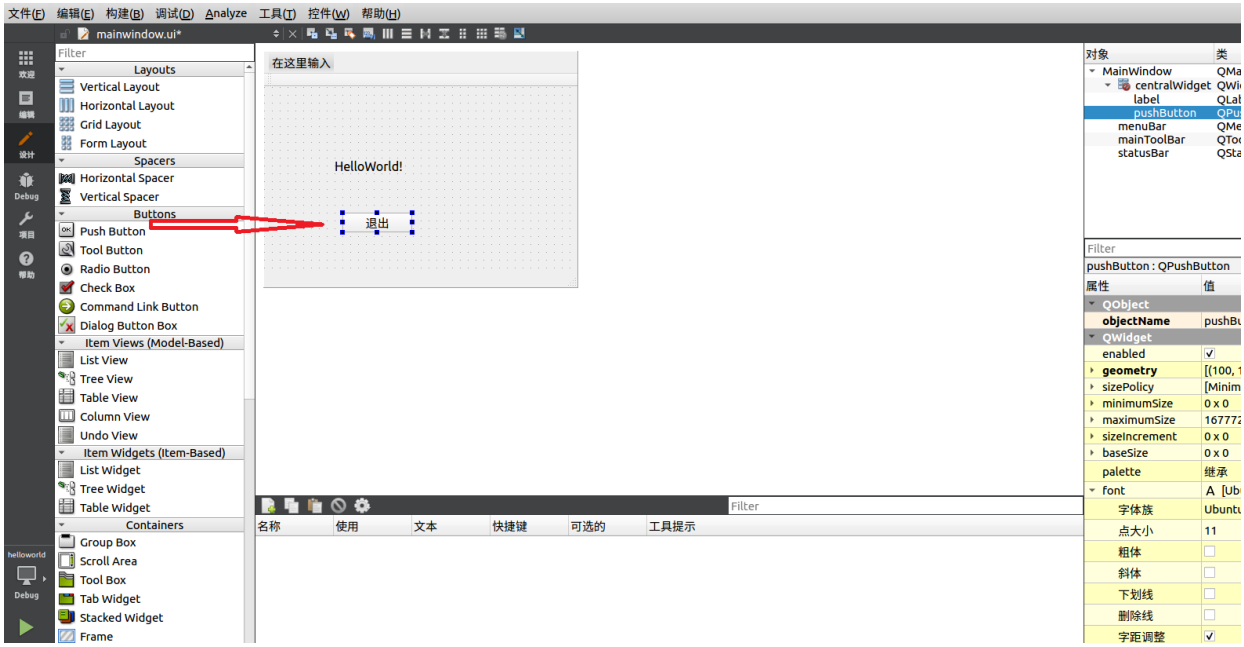
工具	说明
qmake	核心的项目构建工具，可以生成跨平台的 .pro 项目文件，并能依据不同操作系统和编译工具生成相应的 Makefile，用于构建可执行程序或链接库。
uic	User Interface Compiler，用户界面编译器，Qt 使用 XML 语法格式的 .ui 文件定义用户界面，uic 根据 .ui 文件生成用于创建用户界面的 C++ 代码头文件，比如 ui_*.h 。
moc	Meta-Object Compiler，元对象编译器，moc 处理 C++ 头文件的类定义里面的 Q_OBJECT 宏，它会生成源代码文件，比如 moc_*.cpp ，其中包含相应类的元对象代码，元对象代码主要用于实现 Qt 信号/槽机制、运行时类型定义、动态属性系统。

工具	说明
rcc	Resource Compiler，资源文件编译器，负责在项目构建过程中编译 .qrc 资源文件，将资源嵌入到最终的 Qt 程序里。
qtcreator	集成开发环境，包含项目生成管理、代码编辑、图形界面可视化编辑、 编译生成、程序调试、上下文帮助、版本控制系统集成等众多功能， 还支持手机和嵌入式设备的程序生成部署。
assistant	Qt 助手，帮助文档浏览查询工具，Qt 库所有模块和开发工具的帮助文档、示例代码等都可以检索到，是 Qt 开发必备神器，也可用于自学 Qt。
designer	Qt 设计师，专门用于可视化编辑图形用户界面（所见即所得），生成 .ui 文件用于 Qt 项目。
linguist	Qt 语言家，代码里用 tr() 宏包裹的就是可翻译的字符串，开发人员可用 lupdate 命令生成项目的待翻译字符串文件 .ts，用 linguist 翻译多国语言 .ts ，翻译完成后用 lrelease 命令生成 .qm 文件，然后就可用于多国语言界面显示。
qmlscene	在 Qt 4.x 里是用 qmlviewer 进行 QML 程序的原型设计和测试，Qt 5 用 qmlscene 取代了旧的 qmlviewer。新的 qmlscene 另外还支持 Qt 5 中的新特性 scenegraph 。

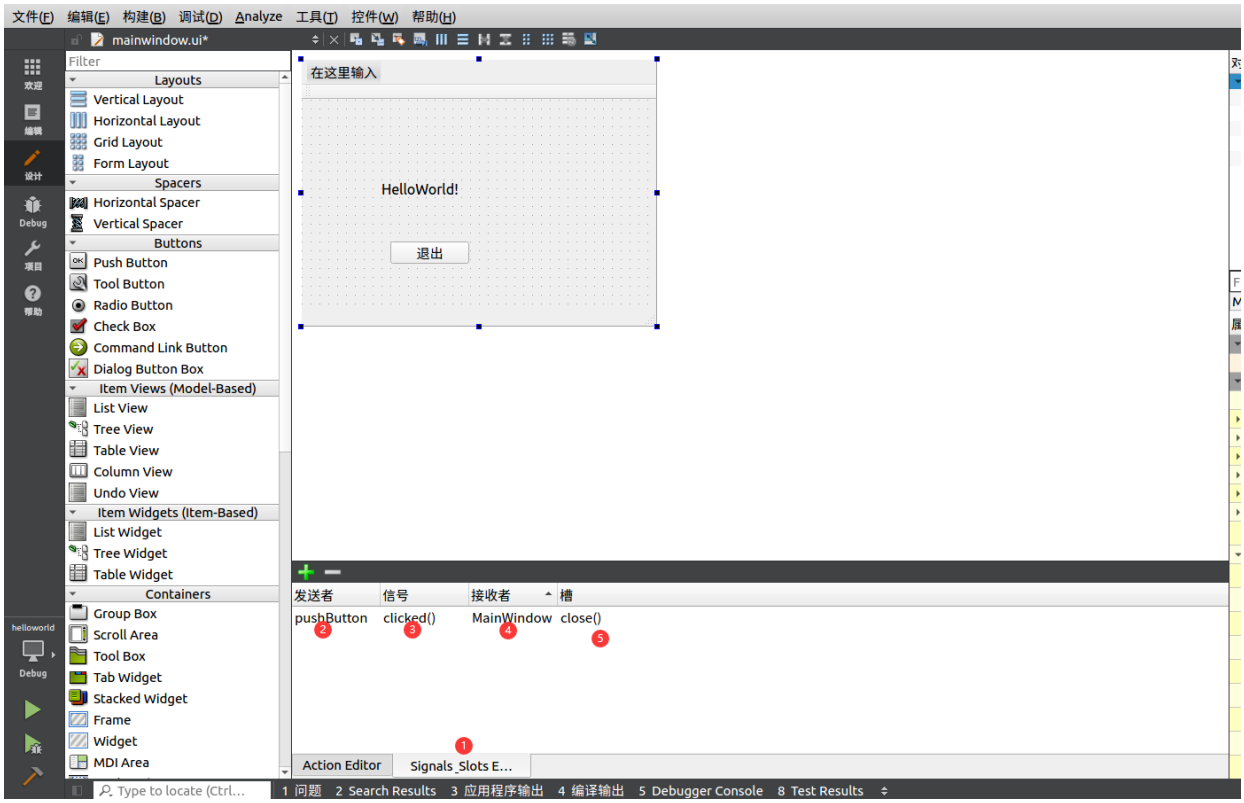
1.6 可视化UI设计

Qt在进行UI界面设计时提供了designer工具，专门用于可视化编辑图形用户界面（所见即所得），该方式效果直观，效率很高。

如图在helloworld项目的基础上，拖动Push Button按钮到布局窗口，并修改文字内容为“退出”。



如下图所示添加对应的信号和槽函数



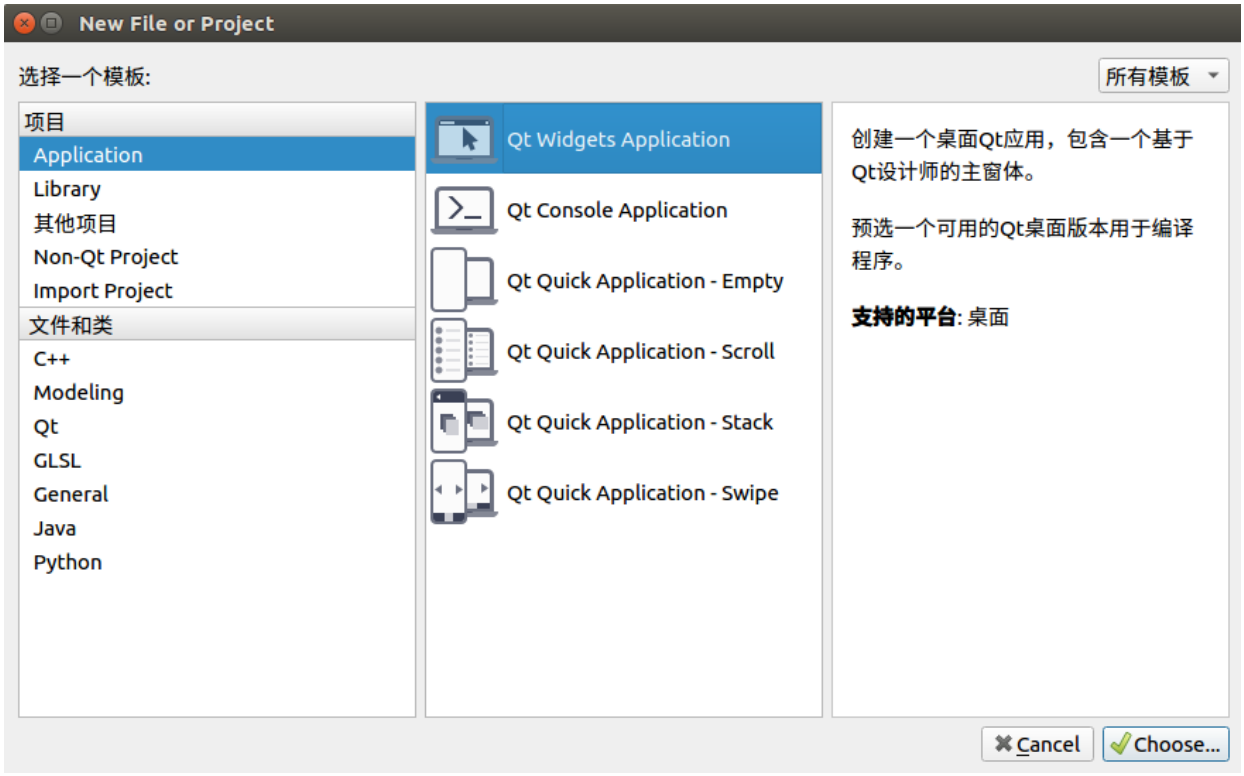
执行程序，点击退出按钮观察实验效果。

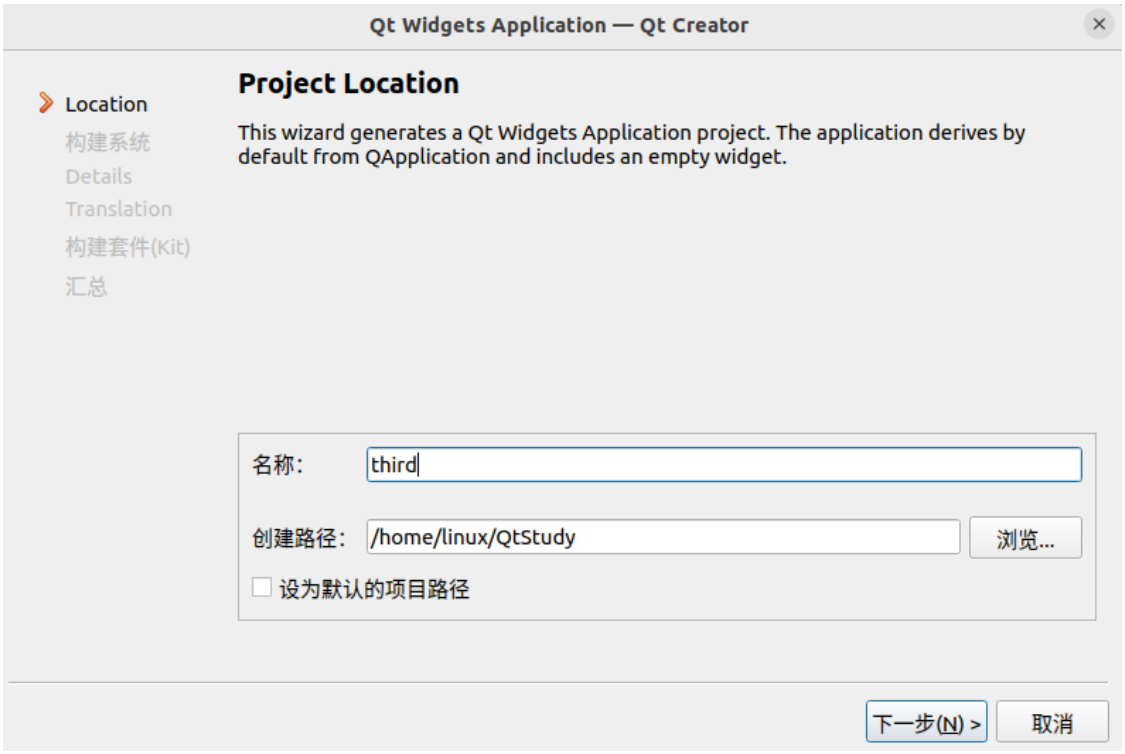
1.7 代码化UI设计

UI的可视化设计时对用户而言的，其实底层都是C++代码实现的，只是QT巧妙地进行了处理，让用户省去了很多繁琐的界面设计工作。由于界面设计的底层都是由C++语言实现的，底层实现的功能比可视化设计更加强大和灵活。某些界面效果是可视化设计无法完成的，或者某些人习惯了用纯代码的方式来实现设计界面，就可

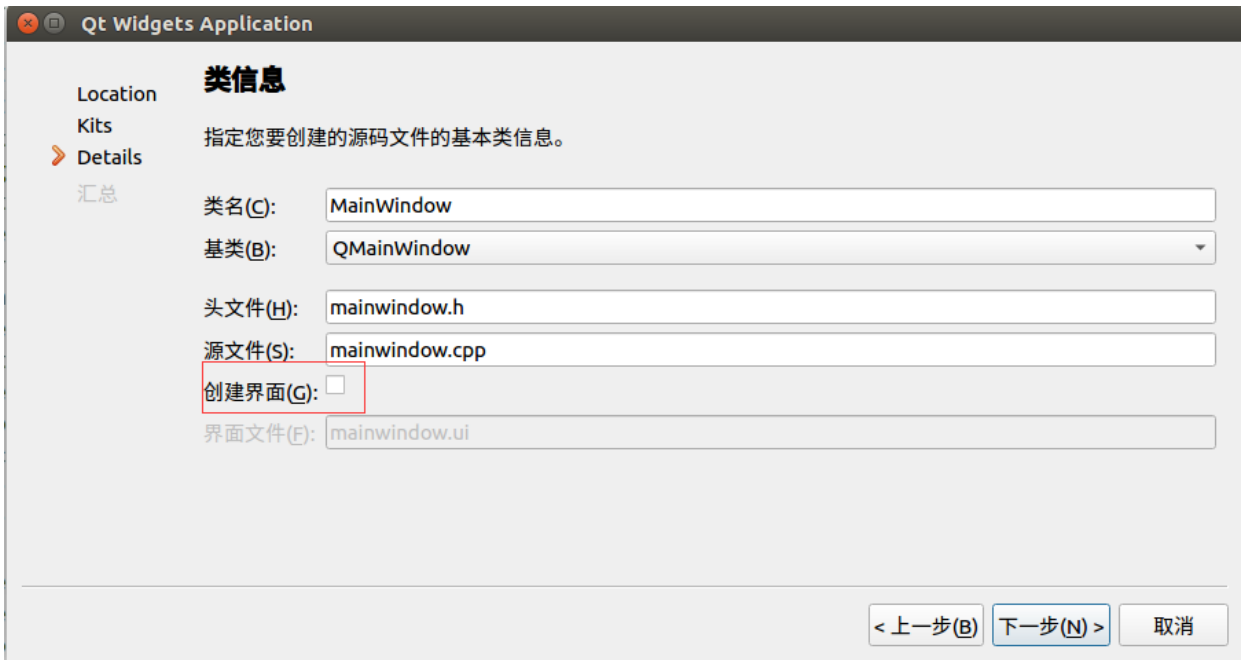
以采用纯代码的方式设计界面，如QT自带的实例基本都是纯代码方式实现用于界面的。

- 新建工程





去掉“创建界面”的复选框。



修改mainwindow.h为如下代码

```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>
#include <QLabel>
#include <QPushButton>
class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    MainWindow(QWidget *parent = 0);
    ~MainWindow();
private:
    QLabel *label1 ;
    QPushButton *btn1;
};

#endif // MAINWINDOW_H
```

修改mainwindow.cpp为如下代码

```
#include "mainwindow.h"

MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
{
    label1 = new QLabel("HelloWorld!", this);
```

```
    btn1 = new QPushButton("退出", this);
    btn1->setGeometry(10,30, 50,50);
    connect(btn1, SIGNAL(clicked()), this, SLOT(close()));
}

MainWindow::~MainWindow()
{

}
```

编译运行程序，和1.6章节代码实现效果相同。

1.7.1 信号与槽

```
// WRONG
QObject::connect(scrollBar, SIGNAL(valueChanged(int value)),
                 label, SLOT(setNum(int value)));
```

```
[static] QMetaObject::Connection QObject::connect(const QObject
*sender, const char *signal, const QObject *receiver, const char
*method, Qt::ConnectionType type = Qt::AutoConnection)
```

*注意，信号和槽参数不能包含任何变量名，只能包含类型。例如，下面的语句将不起作用并返回false:

举例：

```
  QLabel *label = new QLabel;
  QScrollBar *scrollBar = new QScrollBar;必须要先给对象申请空间了
 才能connect
  QObject::connect(scrollBar, SIGNAL(valueChanged(int)),label,
 SLOT(setNum(int)));
```

需要注意：

- 信号与槽函数参数要一致

```
QObject::connect(A,SIGNAL(sigfun(int)),
B,SLOT(slotfun(int)));//ok
QObject::connect(A,SIGNAL(sigfun(int)),
B,SLOT(slotfun(int,int)));//error
```

- 可以带有缺省参数

```
QObject::connect(A,SIGNAL(sigfun(int)),
B,SLOT(slotfun(int,int=0)));//ok
```

- 信号函数的参数可以多于槽函数，多余参数将被忽略

```
QObject::connect(A,SIGNAL(sigfun(int,int)),
B,SLOT(slotfun(int)));//ok
```

- 一个信号可以被连接到多个槽（一对多）

```
QObject::connect(A,SIGNAL(sigfun(int)),
B1,SLOT(slotfun1(int)));//ok
QObject::connect(A,SIGNAL(sigfun(int)),
B2,SLOT(slotfun2(int)));//ok
```

- 多个信号也可以连接到同一个槽（多对一）

```
QObject::connect(A1,SIGNAL(sigfun1(int)),
B,SLOT(slotfun(int)));//ok
QObject::connect(A2,SIGNAL(sigfun2(int)),
B,SLOT(slotfun(int)));//ok
```

- 两个信号可以直接连接(信号级联)

```
QObject::connect(A1,SIGNAL(sigfun1(int)),
A2,SIGNAL(sigfun2(int)));//ok
```

题外话：connect的重载版本

```
QMetaObject::Connection QObject::connect(const QObject *sender,
const char *signal, const QObject *receiver, const char *method,
Qt::ConnectionType type = Qt::AutoConnection)
QMetaObject::Connection QObject::connect(const QObject *sender,
const QMetaMethod &signal, const QObject *receiver, const
QMetaMethod &method, Qt::ConnectionType type =
Qt::AutoConnection)
QMetaObject::Connection QObject::connect(const QObject *sender,
const char *signal, const char *method, Qt::ConnectionType type =
Qt::AutoConnection) const
QMetaObject::Connection QObject::connect(const QObject *sender,
PointerToMemberFunction signal, const QObject *receiver,
PointerToMemberFunction method, Qt::ConnectionType type =
Qt::AutoConnection)
QMetaObject::Connection QObject::connect(const QObject *sender,
PointerToMemberFunction signal, Functor functor)
QMetaObject::Connection QObject::connect(const QObject *sender,
PointerToMemberFunction signal, const QObject *context, Functor
functor, Qt::ConnectionType type = Qt::AutoConnection)
```

- connect一(使用最多)

```
[static] QMetaObject::Connection QObject::connect(const QObject
*sender, const char *signal, const QObject *receiver, const
char *method, Qt::ConnectionType type = Qt::AutoConnection)
```

示例:

```
QLabel *label = new QLabel;
QScrollBar *scrollBar = new QScrollBar;
//指定信号与槽时必须使用SIGNAL 和 SLOT
QObject::connect(scrollBar, SIGNAL(valueChanged(int)),
                 label,  SLOT(setNum(int)));
//语法错误 不能指定形参类型
QObject::connect(scrollBar, SIGNAL(valueChanged(int value)),
                 label, SLOT(setNum(int value)));
```

```
class MyWidget : public QWidget
{
    Q_OBJECT

        public:
    MyWidget();

    signals:
    void buttonClicked();

    private:
    QPushButton *myButton;
};

MyWidget::MyWidget()
{
    myButton = new QPushButton(this);
    //一个信号也可以连接到另一个信号
    connect(myButton, SIGNAL(clicked()),
            this, SIGNAL(buttonClicked()));
}
```

- connect二

```
QMetaObject::Connection QObject::connect(const QObject *sender,
const QMetaMethod &signal, const QObject *receiver, const
QMetaMethod &method, Qt::ConnectionType type =
Qt::AutoConnection)
```

示例:

```
connect(ui-
>pushButton_2,&QPushButton::clicked,this,&::MainWindow::pushBut
on2_clicked);
//若clicked信号有重载版本，则要写成如下形式
connect(ui->pushButton_2, static_cast<void (QPushButton::*)
(bool)>(&QPushButton::clicked), this,
SLOT(pushButon2_clicked()));
```

- connect三

```
QMetaObject::Connection QObject::connect(const QObject *sender,
const char *signal, const char *method, Qt::ConnectionType type
= Qt::AutoConnection) const
```

等价于connect(sender, signal, this, method, type). //方式一

- connect四

```
QMetaObject::Connection QObject::connect(const QObject *sender,
PointerToMemberFunction signal, const QObject *receiver,
PointerToMemberFunction method, Qt::ConnectionType type =
Qt::AutoConnection)
```

示例:

```
QLabel *label = new QLabel;
QLineEdit *lineEdit = new QLineEdit;
QObject::connect(lineEdit, &QLineEdit::textChanged,
                 label,  &QLabel::setText);
```

- connect五

```
QMetaObject::Connection QObject::connect(const QObject *sender,
PointerToMemberFunction signal, Functor functor)
```

示例:

```
void someFunction();
QPushButton *button = new QPushButton;
QObject::connect(button, &QPushButton::clicked, someFunction);

//也可以使用lambda表达式
QByteArray page = ...;
QTcpSocket *socket = new QTcpSocket;
socket->connectToHost("qt-project.org", 80);
QObject::connect(socket, &QTcpSocket::connected, [=] () {
    socket->write("GET " + page + "\r\n");
});
```

- connect六

```
QMetaObject::Connection QObject::connect(const QObject *sender,
PointerToMemberFunction signal, const QObject *context, Functor
functor, Qt::ConnectionType type = Qt::AutoConnection)
```

示例:

```
void someFunction();
QPushButton *button = new QPushButton;
QObject::connect(button, &QPushButton::clicked, this,
someFunction, Qt::QueuedConnection);
```

1.7.2 更彻底的代码化UI设计

```
#include <QApplication>
#include <QWidget>
#include <QLabel>

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    QWidget w;

    QLabel label("helloworld", &w);

    w.resize(140,300);
    w.move(500,400);
    label.move(50,100);


    w.show();
```

```
    return a.exec();
}
```

```
#############################################################
#####
# Automatically generated by qmake (3.1) Thu Jul 20 22:26:55 2023
#############################################################
#####

QT += widgets
TEMPLATE = app
TARGET = 01
INCLUDEPATH += .

# The following define makes your compiler warn you if you use
any
# feature of Qt which has been marked as deprecated (the exact
warnings
# depend on your compiler). Please consult the documentation of
the
# deprecated API in order to know how to port your code away from
it.
DEFINES += QT_DEPRECATED_WARNINGS

# You can also make your code fail to compile if you use
deprecated APIs.
# In order to do so, uncomment the following line.
# You can also select to disable deprecated APIs only up to a
certain version of Qt.
#DEFINES += QT_DISABLE_DEPRECATED_BEFORE=0x060000    # disables
all the APIs deprecated before Qt 6.0.0

# Input
SOURCES += main.cpp
```

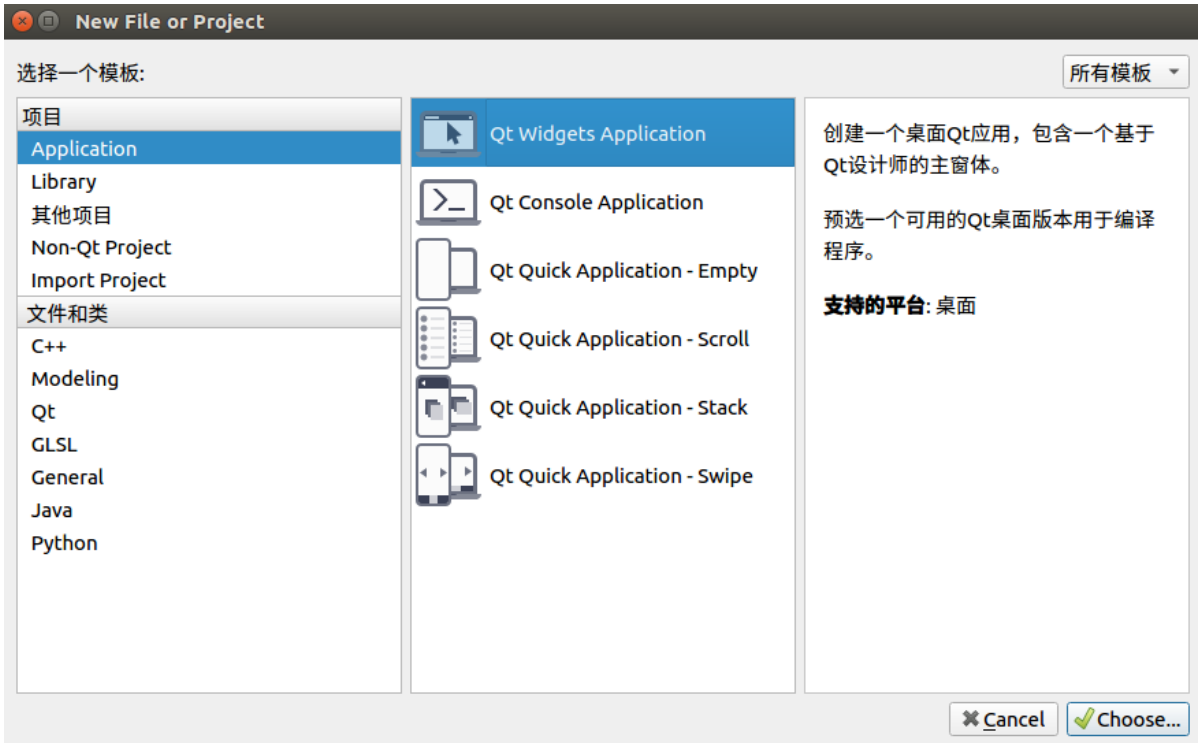
作业： 在以上代码的基础上增加 退出按钮 保证点击按钮时 退出应用

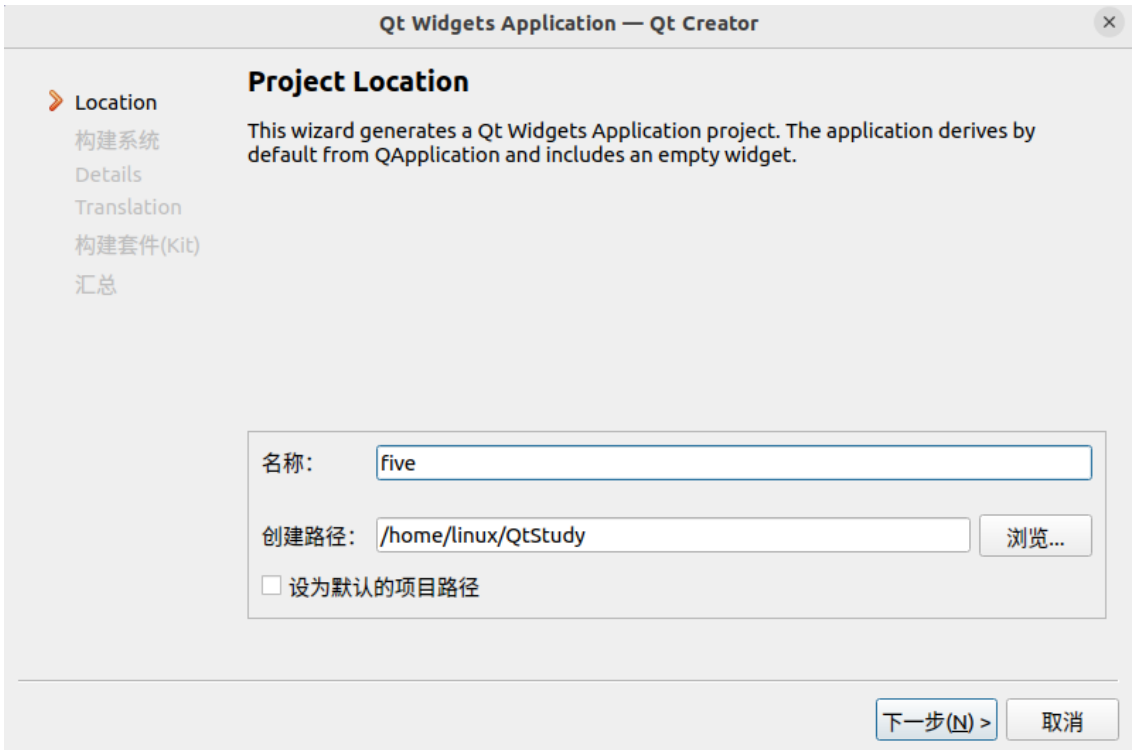
-

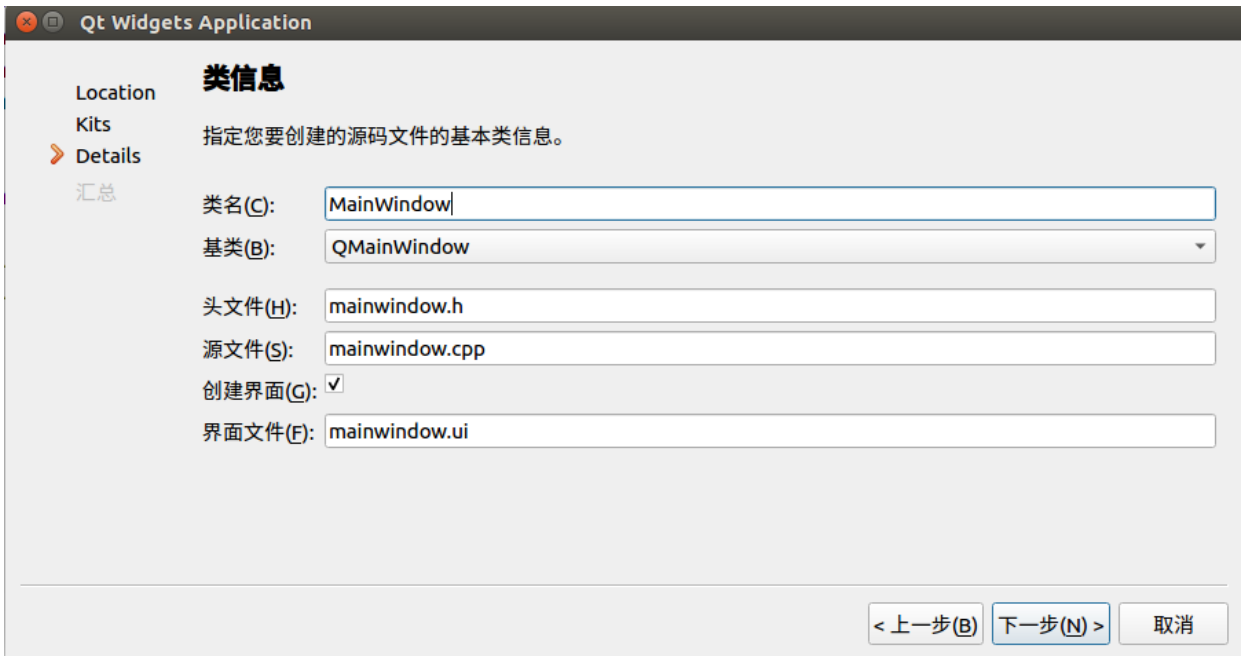
1.8 混合方式UI设计

可视化UI设计无需人工编写代码去处理大量繁琐的界面组件的创建和布局管理工作，可以直接进行界面设计，大大提高工作效率。但是可视化UI设计也存在一些缺陷，如某些组件无法可视化添加到界面上，比如在工具栏无法可视化添加ComboBox组件，而用代码就可以。而采用纯代码方式进行UI设计虽然无所不能，但设计效率太低，过程繁琐，而UI设计简单高效。所以，能用可视化UI设计的尽可能用可视化设计解决，无法解决的在用纯代码方式，将两种方式结合，才是高效设计UI的方法。

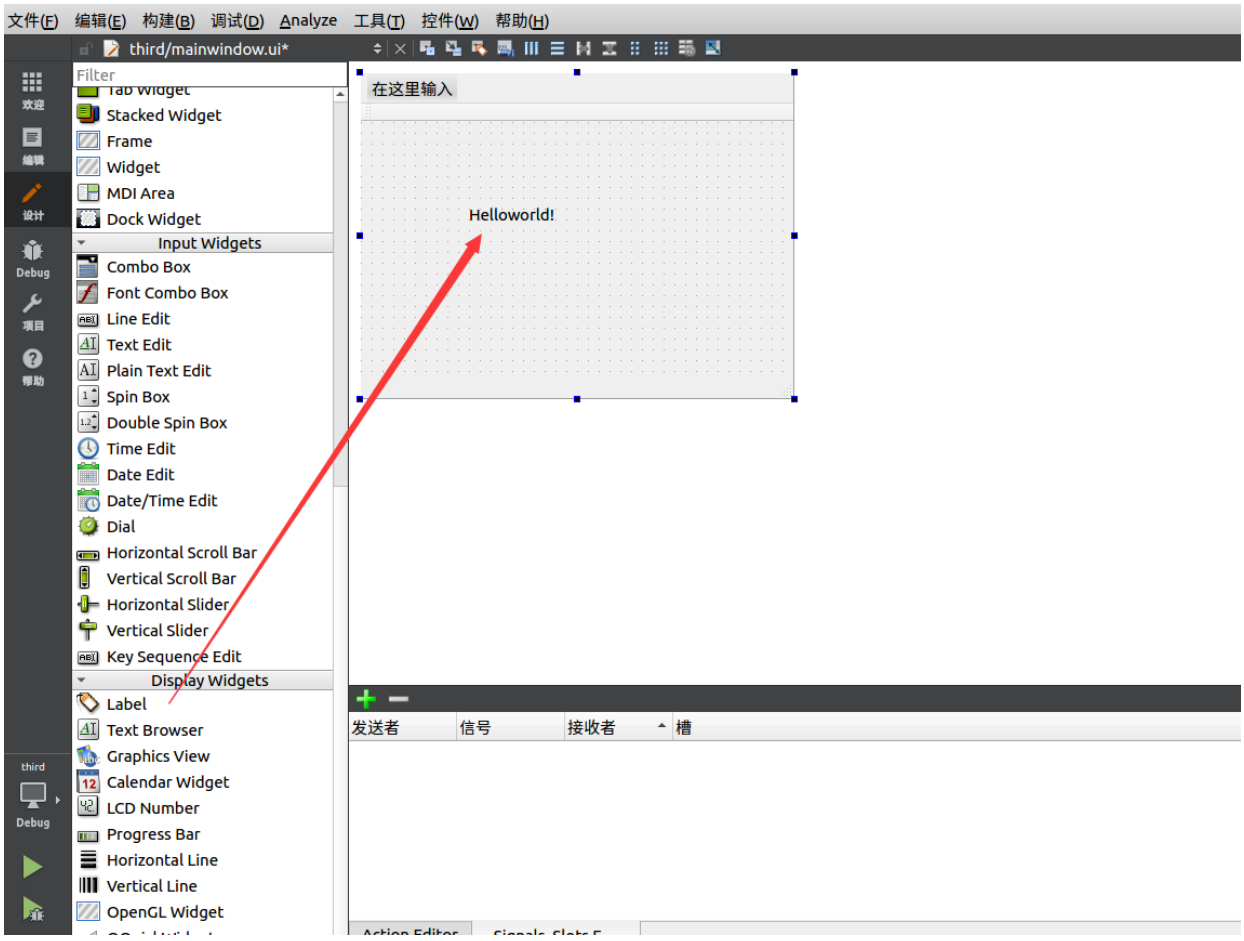
- 创建工程







- UI可视化界面添加QLabel组件



- 代码添加QPushButton组件

  mainwindow.h文件代码如下：

```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>
#include <QPushButton>
```

```
namespace Ui {
class MainWindow;
}

class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    explicit MainWindow(QWidget *parent = nullptr);
    ~MainWindow();

private:
    Ui::MainWindow *ui;
    QPushButton *btn1;
};

#endif // MAINWINDOW_H
```

mainwindow.cpp代码如下所示

```
#include "mainwindow.h"
#include "ui_mainwindow.h"

MainWindow::MainWindow(QWidget *parent) :
    QMainWindow(parent),
    ui(new Ui::MainWindow)
{
    ui->setupUi(this);
    btn1 = new QPushButton("退出", this);
    btn1->setGeometry(100,150, 50,50);
    connect(btn1, SIGNAL(clicked()), this, SLOT(close()));
}

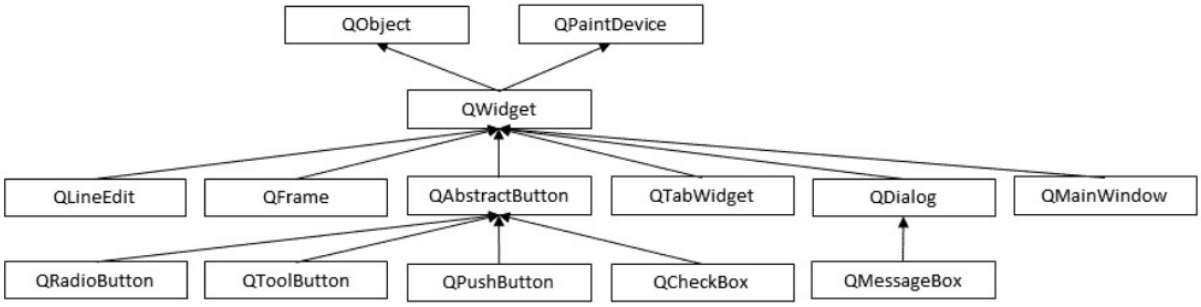
MainWindow::~MainWindow()
{

}
```

编译运行程序，实验效果与1.6、1.7章节代码运行效果相同。

第02章 常用界面设计组件

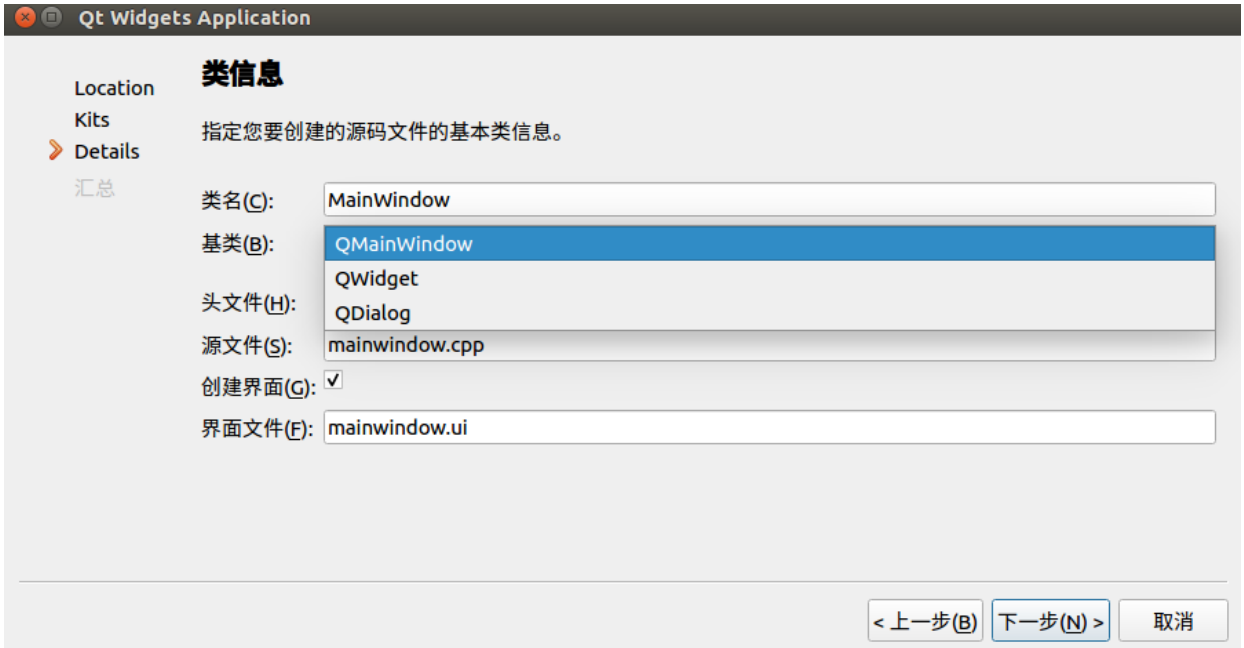
组件是GUI的基本元素，也称为UI控件。它接受来自底层平台的不同用户事件，如鼠标和键盘事件（以及其它事件）。所有组件都继承自QObject。QWidget是一个基本的组件，是所有UI组件的基类。它描述组件所需要的大多数属性，包括几何图形、颜色、键盘行为、工具提示等属性。继承关系如下图所示。



2.1窗体

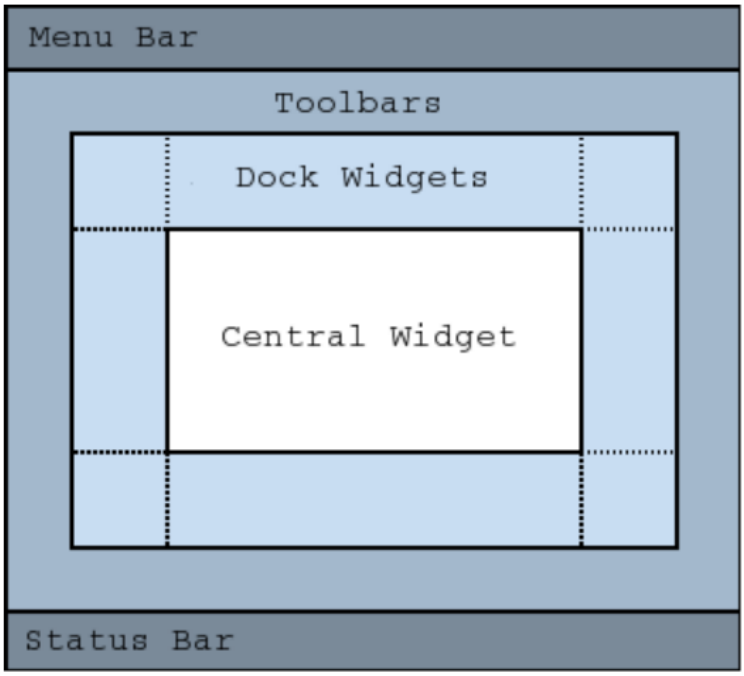
窗体是指程序的整体界面，可以包含标题栏、菜单栏、工具栏、关闭按钮、最小化按钮、最大化按钮等。

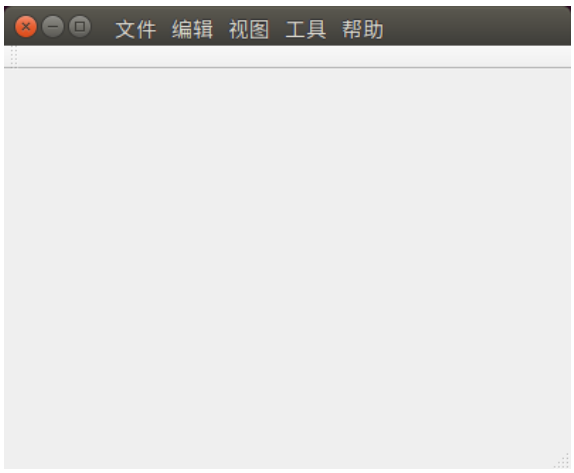
QMainWindow、QWidget、QDialog 三个类就是用来创建窗体的，可以直接使用也可以继承后再使用。



- **QMainWindow**

QMainWindow 窗口可以包含菜单栏、工具栏、状态栏、标题栏等，是最常见的窗口形式，可以作为GUI程序的主窗口。





注意：若不显示菜单栏可按如下方式设置



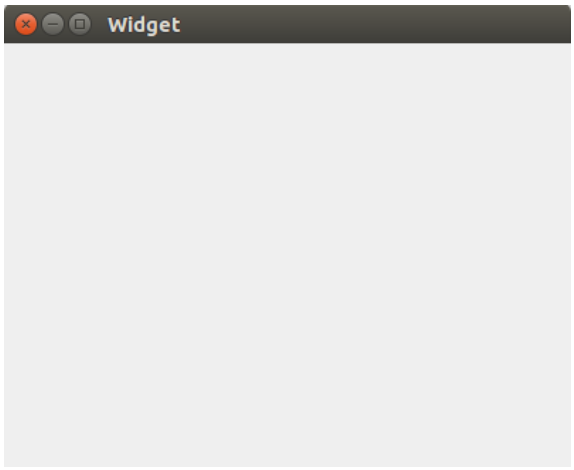
- **QDialog**

QDialog 是对话框窗口的基类。对话框主要用来执行短期任务，或与用户进行互动，它可以是模态的也可以是非模态的。QDialog 没有菜单栏、工具栏、状态栏等。



- **QWidget**

QWidget是Qt中所有可绘制类的基类。任何基于`QWidget`的类都可以在没有父类的情况下显示为窗口 。



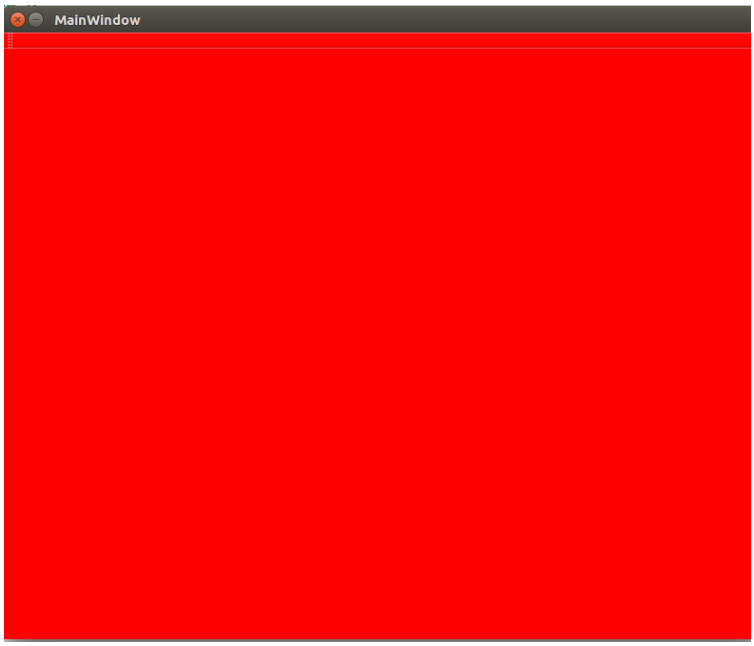
如果是主窗口，就使用 QMainWindow；如果是对话框，就是用 QDialog；如果不确定，或有可能作为顶级窗口，也有可能嵌入到其他窗口中，那么使用 QWidget。

2.1.1 设置窗体位置、大小及背景颜色

窗体初始位置为（100,100），控制窗体不可更改大，最大、最小均为 800x640，背景色为红色。

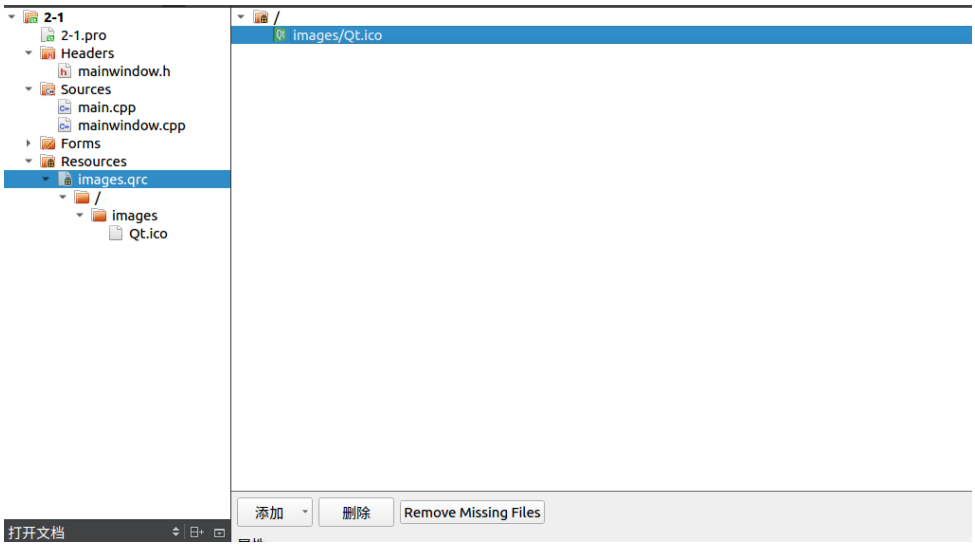
```
MainWindow::MainWindow(QWidget *parent) :
    QMainWindow(parent),
    ui(new Ui::MainWindow)
{
    ui->setupUi(this);
    //或者使用 setGeometry
    move(100,100);
    setMaximumSize(800,640);
    setMinimumSize(800,640);
    setStyleSheet("background:red");
}
```

运行效果如下图所示：



2.1.2 设置窗体标题

- 添加图片资源



注：图标可以去阿里矢量图库下载： https://www.iconfont.cn/

或者https://www.51tool.com/ico/?action=make 转换ico图标。

- 修改代码

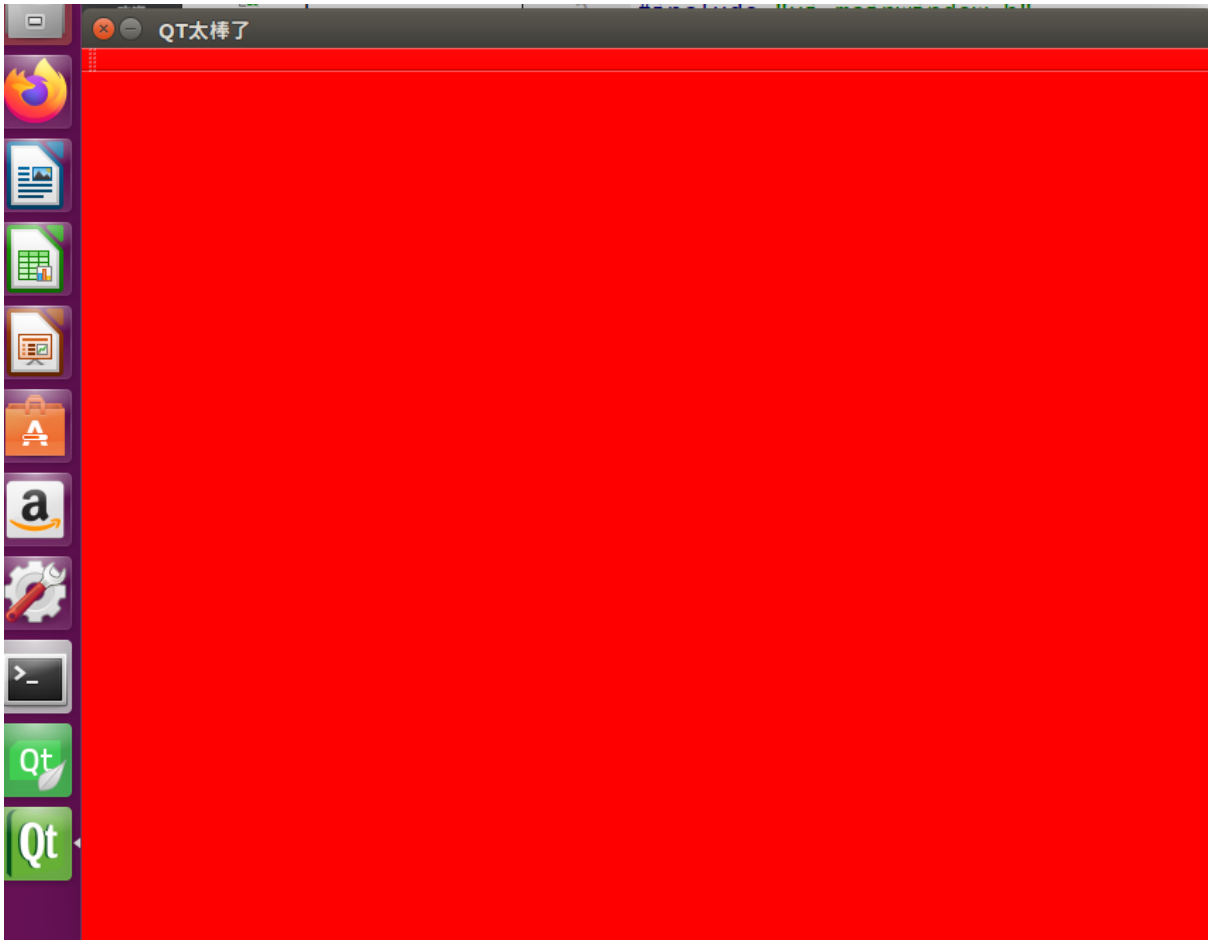
```
#include "mainwindow.h"
#include "ui_mainwindow.h"
#include <QIcon>

MainWindow::MainWindow(QWidget *parent) :
    QMainWindow(parent),
    ui(new Ui::MainWindow)
{
    ui->setupUi(this);
    //或者使用 setGeometry
    move(100,100);
    setMaximumSize(800,640);
    setMinimumSize(800,640);
    setStyleSheet("background:red");

    setWindowTitle("QT太棒了");
    setWindowIcon(QIcon(":/images/Qt.ico"));
}

MainWindow::~MainWindow()
{
    delete ui;
}
```

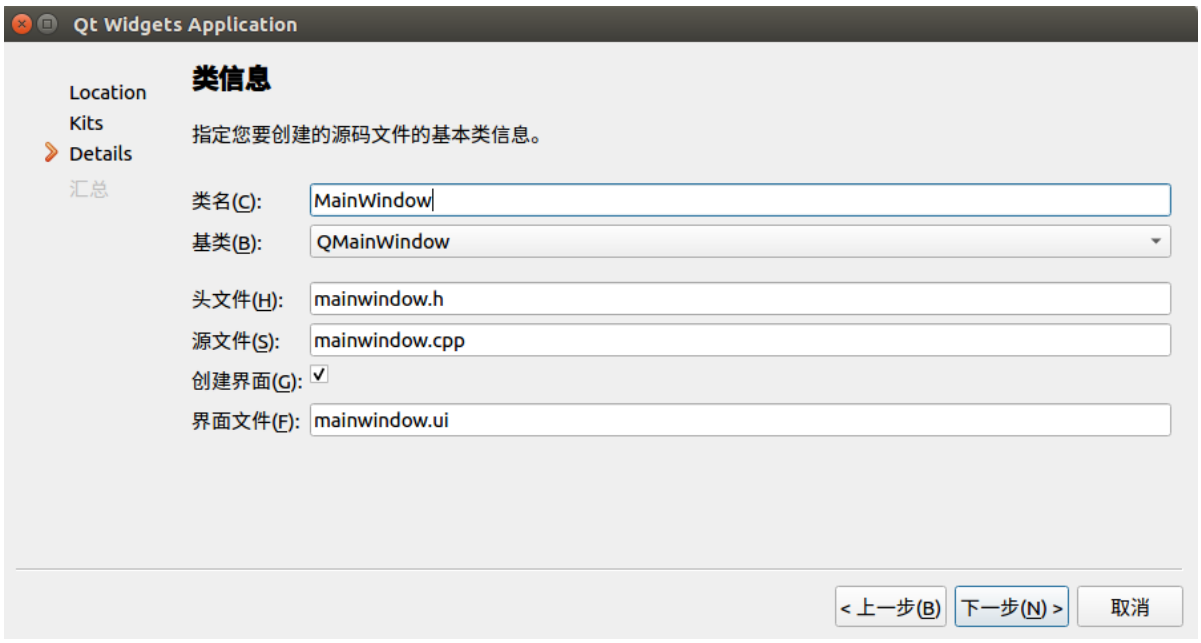
- 运行效果



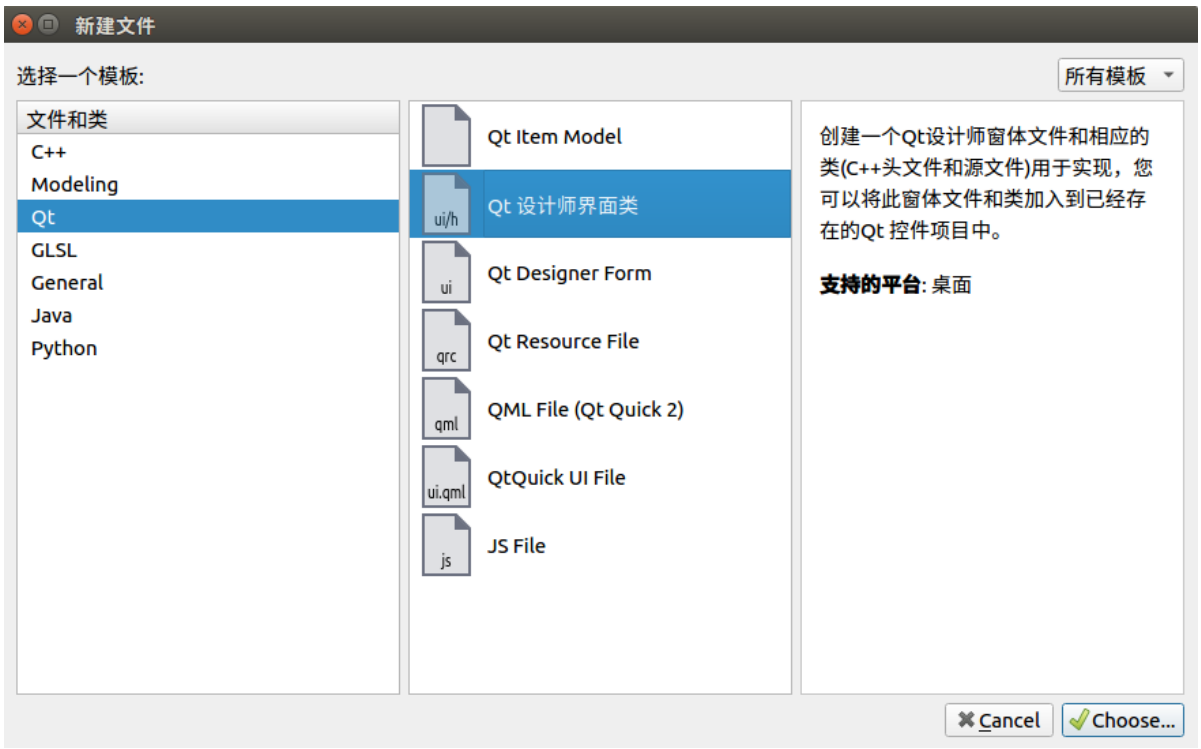
注意：窗口左上角的图标显示不出来，是跟ubuntu系统的显示相关（windows环境下正常显示）。

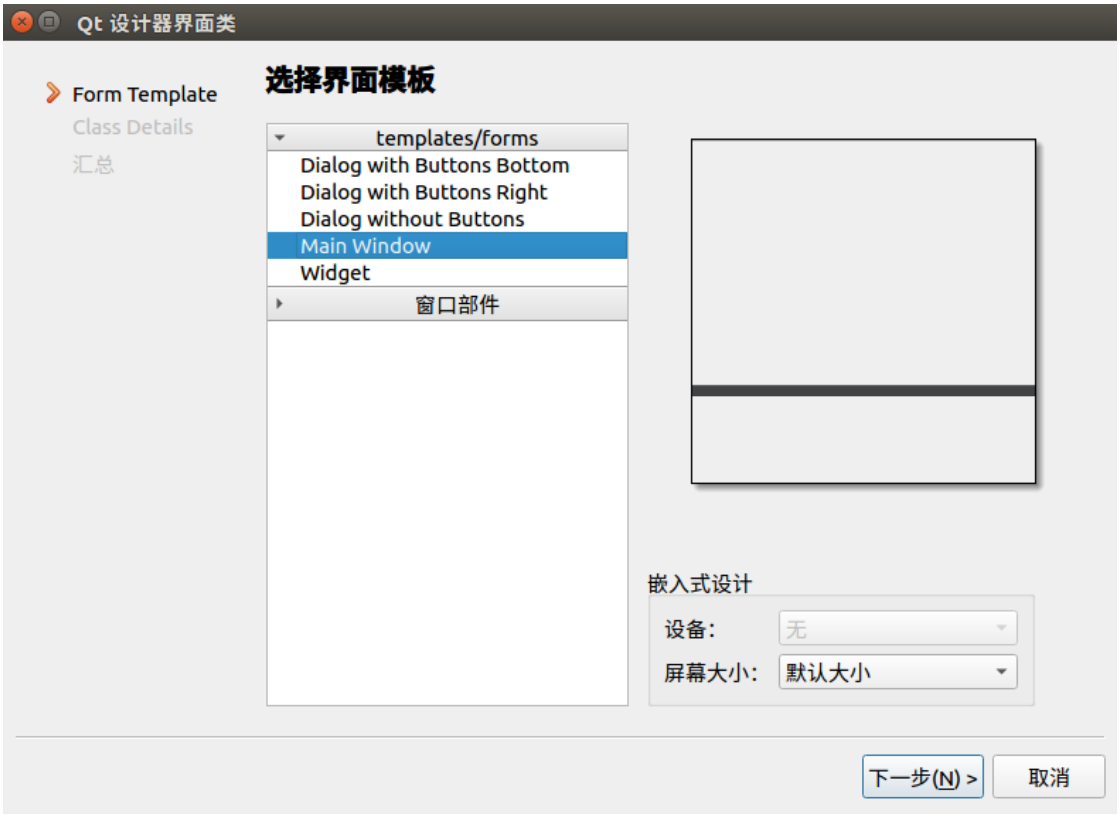
2.1.3 多窗体调用

- 创建基于QMainWindow的项目



- 增加新的窗口类





- 代码编辑

```
/*mainwindow.cpp*/
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>
#include <mainwindow2.h>

namespace Ui {
class MainWindow;
```

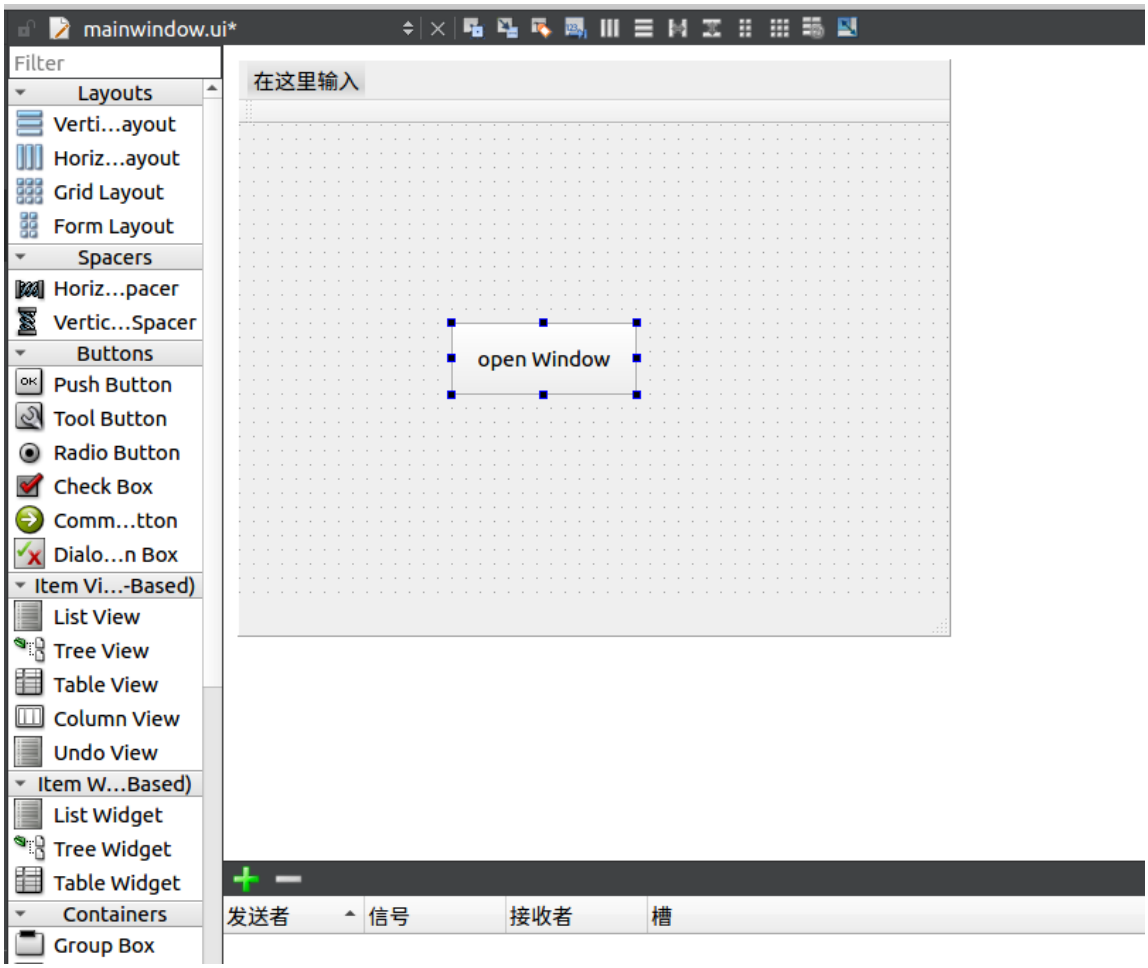
```
}

class MainWindow : public QMainWindow
{
    Q_OBJECT

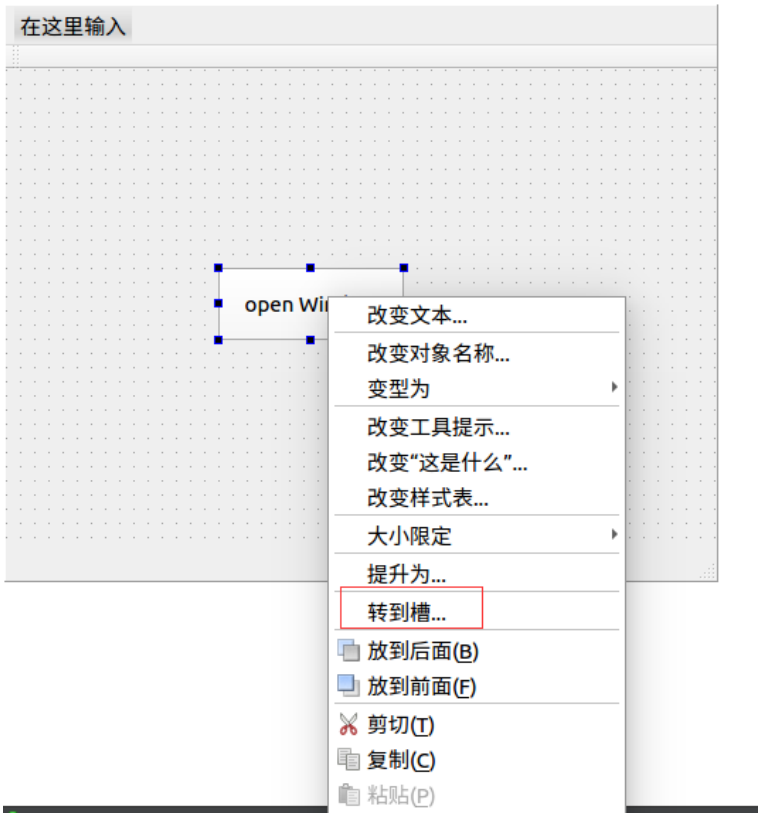
public:
    explicit MainWindow(QWidget *parent = nullptr);
    ~MainWindow();

private:
    Ui::MainWindow *ui;
    MainWindow2 w2;
};

#endif // MAINWINDOW_H
```



- 添加按钮对应的槽函数



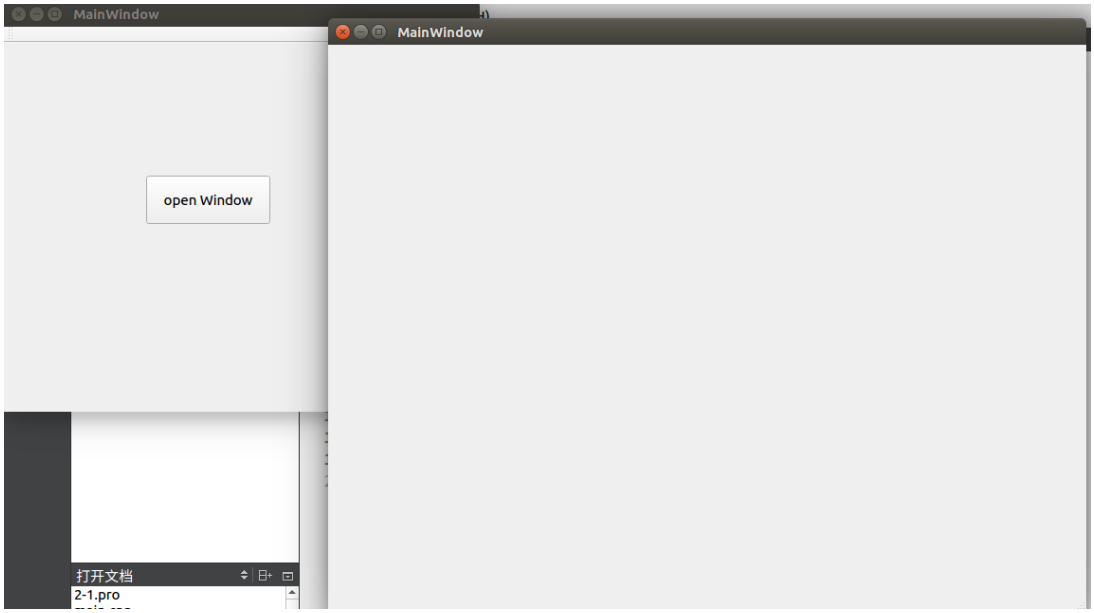

```
/*mainwindow.cpp*/
#include "mainwindow.h"
#include "ui_mainwindow.h"

MainWindow::MainWindow(QWidget *parent) :
    QMainWindow(parent),
    ui(new Ui::MainWindow)
{
    ui->setupUi(this);
}

MainWindow::~MainWindow()
{
    delete ui;
}

void MainWindow::on_pushButton_clicked()
{
    w2.show();
}
```

- 运行效果



2.2 字符串与输入输出组件

2.2.1 字符串与数值之间的转换

界面设计时使用最多的组件恐怕就是QLabel和QLineEdit了，QLabel用于显示字符串，QLineEdit用于显示和输入字符串。这两个类都有如下两个成员函数用于读取和设置显示文字。

```
QString text() const;
void setText(const QString &text);
```

这两个函数都涉及到了QString类。QString类是Qt程序中经常使用的类，用于处理字符串。QString类可以进行字符串与数字之间的转换，使用QLineEdit就可以实现数字量的输入输出。

下图为基于QWidget创建的可视化窗体。



选中"=",右键快捷菜单中单击“Go to slot ...”，并在对话框里选择clicked()信号创建槽函数，在自动生成的函数体编写如下代码，实现从界面中读取用户输入的数量和单价，相乘计算后将结果显示到总价对应的文本框里。

```
#include "widget.h"
#include "ui_widget.h"

Widget::Widget(QWidget *parent) :
```

```
    QWidget(parent),
    ui(new Ui::Widget)
{
    ui->setupUi(this);
}

Widget::~Widget()
{
    delete ui;
}
void Widget::on_btnCalc_clicked()
{
    QString text = ui->le_num->text();
    int num = text.toInt();
    text = ui->le_price->text();
    float price = text.toFloat();
    float sum = price * num;

    text = text.asprintf("%.2f", double(sum));
    ui->le_sum->setText(text);
}
```

小结:

- 字符串转数字

```
int toInt(bool *ok = nullptr, int base = 10) const

long toLong(bool *ok = nullptr, int base = 10) const

short toShort(bool *ok = nullptr, int base = 10) const

uint toUInt(bool *ok = nullptr, int base = 10) const

ulong toULong(bool *ok = nullptr, int base = 10) const

ushort toUShort(bool *ok = nullptr, int base = 10) const

double toDouble(bool *ok = nullptr) const
float toFloat(bool *ok = nullptr) const
```

- 数字转字符串

```
text = QString::number(double(sum), 'f', 2);
text = QString::asprintf("%.2f", double(sum));//重点记忆一种就可以了 这个和C语法一致建议重点记忆
text = text.sprintf("%.2f", double(sum));
text = text.setNum(sum, 'f',2);
```

QString提供了大量的功能，可以通过查看QT帮助手册得到。例如字符串拼接

```
QString &append(const char *str)
QString QString::arg(const QString &a, int fieldWidth = 0,
QChar fillChar = QLatin1Char(' ')) const
//example
QString status = QString("Processing file %1 of %2:
%3".arg(i).arg(total).arg(fileName);
```

2.2.2 QString的常用功能

QString提供了大量的功能，有助于灵活实现字符串处理功能，可以可以通过查看QT帮助手册得到。下面举几个例子。

- append（）和prepend()
- toUpper() 和toLower()
- count() 、size() 和 length()

```
/*字符串中如果有汉字 每个汉字算一个字符*/
text = "NI 好啊";
num = text.size();  // 5
```

- trimmed() 和simplified()

```
//Returns a string that has whitespace removed from the start
and the end.
//This includes the ASCII characters '\t', '\n', '\v', '\f',
'\r', and ' '.
QString str = "  lots\t of\nwhitespace\r\n ";
str = str.trimmed(); //str ="lots\t of\nwhitespace"

/*Returns a string that has whitespace removed from the start
and the end, and that has each sequence of internal whitespace
replaced with a single space*/
QString str = "  lots\t of\nwhitespace\r\n ";
str = str.simplified();// str = "lots of whitespace";
```

- indexOf() 和lastIndexOf()
- append () 和arg()

```
  QString i;           // current file's number
  QString total;       // number of files to process
  QString fileName;    // current file's name

  QString status = QString("Processing file %1 of %2: %3")
                   .arg(i).arg(total).arg(fileName);
```

- section()

```
  QString str;
  QString csv = "forename,middlename,surname,phone";
  QString path = "/usr/local/bin/myapp"; // First field is
empty
  QString::SectionFlag flag = QString::SectionSkipEmpty;

  str = csv.section(',', 2, 2);   // str == "surname"
  str = path.section('/', 3, 4);  // str == "bin/myapp"
  str = path.section('/', 3, 3, flag); // str == "myapp"
```

- endsWith()和startsWith()
- isNull()和isEmpty()

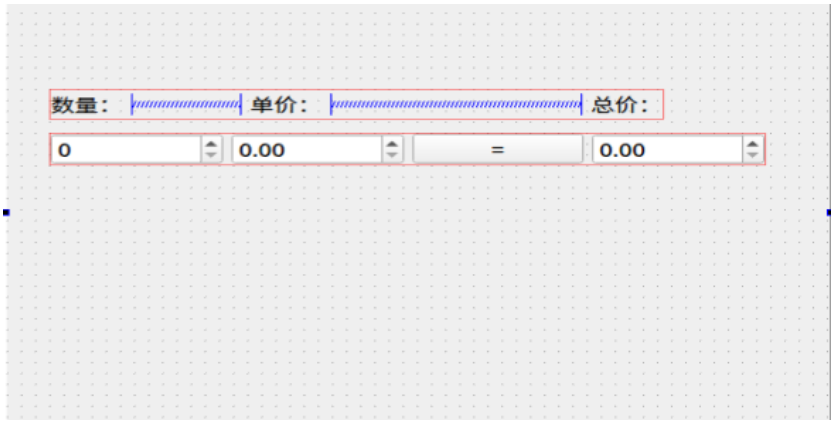
2.3 数字输入和显示组件

2.3.1 QSpinBox与QDoubleSpinBox

QSpinBox用于整数的显示和输入，一般显示十进制数，也可以显示二进制、十六进制数，而且可以在显示框中增加前缀或后缀。QDoubleSpinBox用于浮点数的显

示和输入，可以设置显示小数的位数，也可以设置显示前缀和后缀。

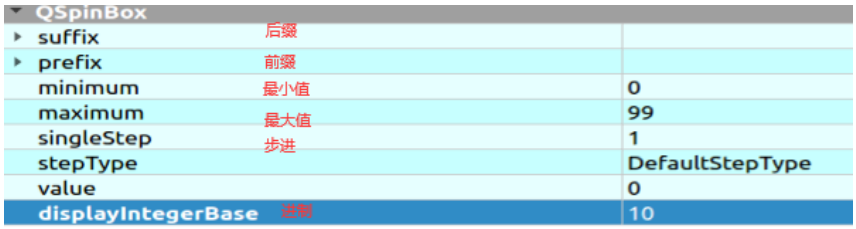
设计如下窗口：

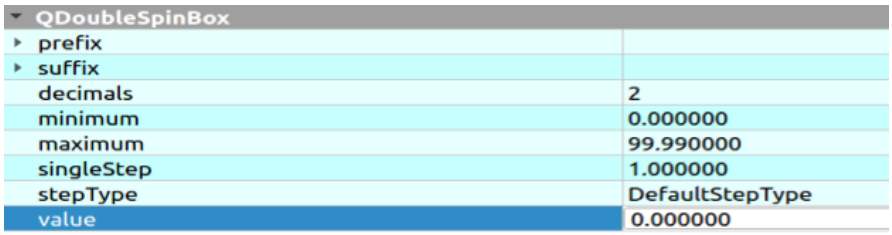


设计按钮对应的槽函数

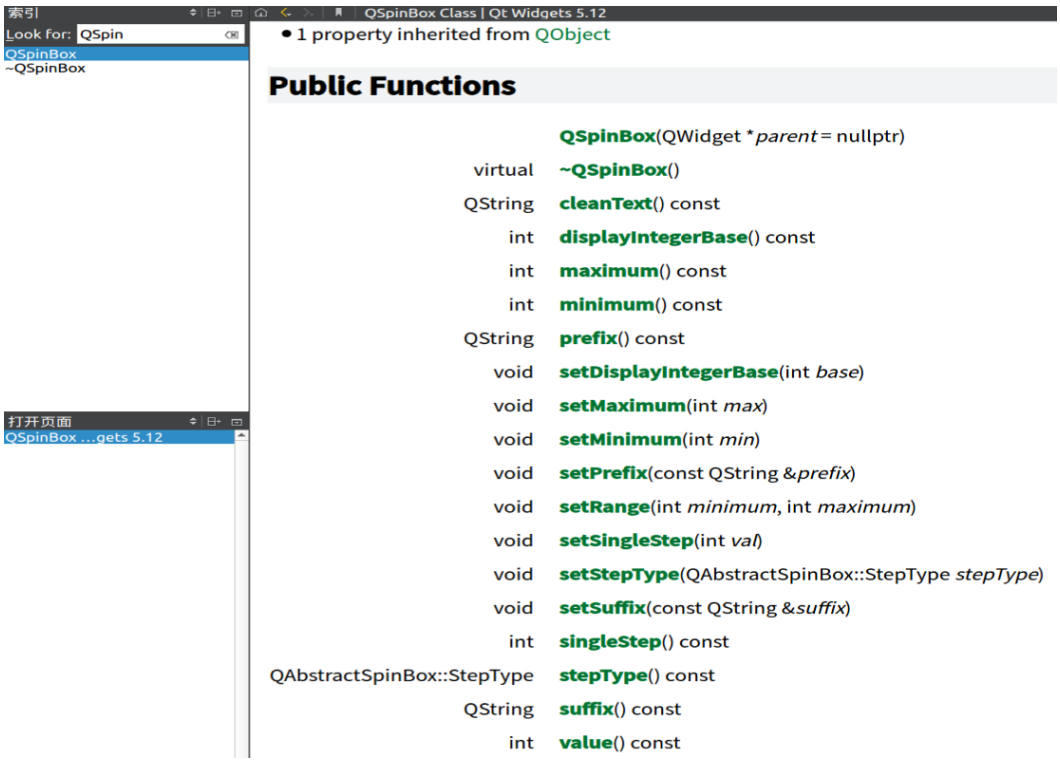
```
void Widget::on_btnCalc_clicked()
{
    int num = ui->sb_num->value();
    double price = ui->dsb_price->value();
    double sum = num * price;
    ui->dsb_sum->setValue(sum);
}
```

可以在Designer中直接设置以下属性





对应的成员函数可以查阅QT官方手册。

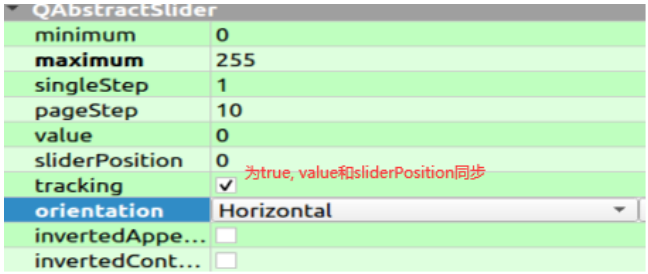


2.3.2其它数值输入和显示组件

- QSlider

  QSlider和QDial 2个组件都从QAbstractSlider继承而来，有一些共有的属性。QSlider是滑动的标尺型组件，滑动标尺上的一个滑块可以改变值。

  QAbstractSlider的主要属性包括以下几种：

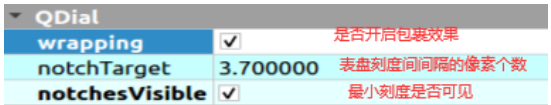


属于QSlider的专有属性有两个：



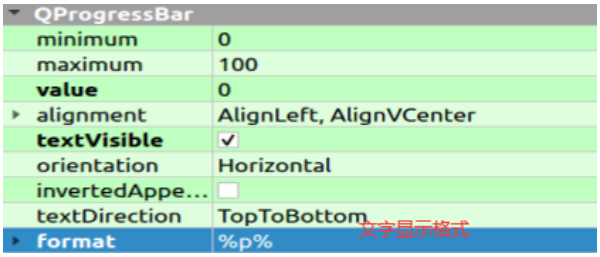
- QDial

  QDial是仪表盘式的组件，通过旋转表盘获得输入值。特有的属性包括：



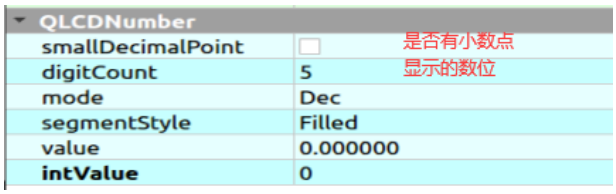
- QProgressBar

  它的父类是QWidget，一般用于显示进度

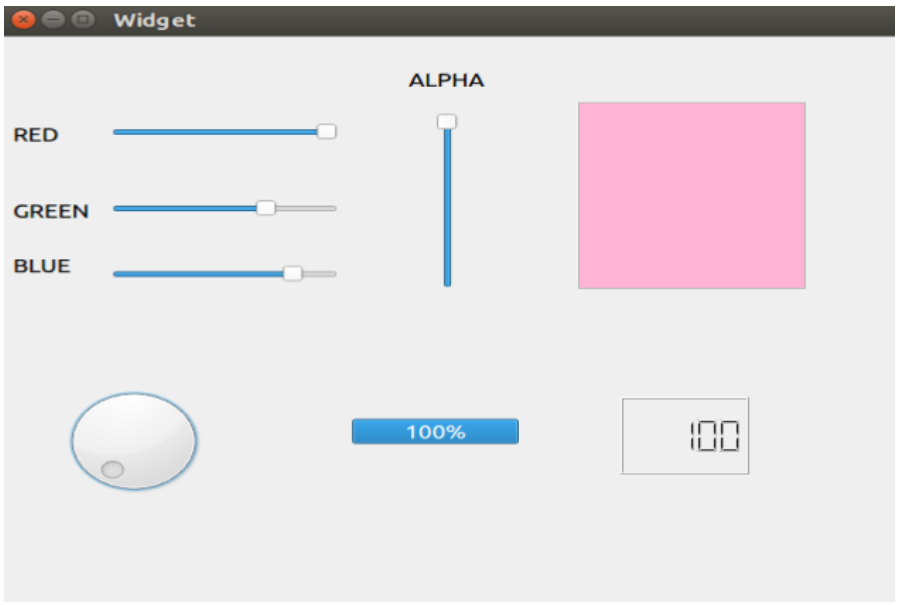


- QLCDNumber

  QLCDNumber是模拟LCD显示数字的组件，可以显示整数或小数。小数点也算一个数位。



创建基于QWidget的工程，如下图所示：



增加的代码如下：

```
#include "widget.h"
#include "ui_widget.h"
#include <QPalette>
#include <QDebug>

Widget::Widget(QWidget *parent) :
    QWidget(parent),
    ui(new Ui::Widget)
{
    ui->setupUi(this);
    connect(ui->rSlider, SIGNAL(valueChanged(int)),
            this, SLOT(on_slider_value_changed(int)));
    connect(ui->gSlider, SIGNAL(valueChanged(int)),
            this, SLOT(on_slider_value_changed(int)));
```

```
    connect(ui->bSlider, SIGNAL(valueChanged(int)),
            this, SLOT(on_slider_value_changed(int)));
    connect(ui->aSlider, SIGNAL(valueChanged(int)),
            this, SLOT(on_slider_value_changed(int)));
}

Widget::~Widget()
{
    delete ui;
}

void Widget::on_slider_value_changed(int val){
    Q_UNUSED(val);
    QColor color;


    int red = ui->rSlider->value();
    int green = ui->gSlider->value();
    int blue = ui->bSlider->value();
    int alpha = ui->aSlider->value();

    color.setRgb(red,green,blue, alpha);
    QPalette pal = ui->textEdit->palette();
    pal.setColor(QPalette::Base, color);
    ui->textEdit->setPalette(pal);
}

void Widget::on_dial_sliderMoved(int position)
{
    Q_UNUSED(position);
    int value = ui->dial->value();
    ui->progressBar->setValue(value);
    ui->lcdNumber->display(value);
}
```

2.4 按钮组件

QPushButton、QRadioButton 、QCheckBox都从QAbstractButton，拥有一些共同的属性，如下图所示:

QAbstractButton	
text	男 显示的文本
icon	显示的图标
iconSize	18 x 18
shortcut	
checkable	☑
checked	☐
autoRepeat	☐
autoExclusive	☑
autoRepeatDelay	300
autoRepeatInterval	100

图标使用setIcon()来设置，文本可以在构造函数或通过setText()来设置。 可以使用 isChecked() 检查是否被选中。

- QPushButton

  QPushButton 是Qt常用的控件之一，提供普通的按钮功能。 通过信号槽机制接收触发信号并执行对应动作。

- QRadioButton

  单选框默认开启自动互斥（autoExclusive）。如果启用了自动互斥，属于同一个父部件的单选框的行为就和属于一个互斥按钮组的一样。如果你需要为属于同一父部件的单选框设置多个互斥按钮组，把它们加入QButtonGroup中。

- QCheckBox

  它也是一个可选择的按钮， 常见用途是在要求用户选择一个或多个可用选项的情况下。 与单选按钮不同，默认情况下复选框不是互斥的。 checkBox按钮可以通过在QButtonGroup对象中添加它们而互斥。 每当选中或清除复选框时，它都会发出信号状态Changed()。如果要在每次复选框更改状态时触发操作，请连接到此信号。您可以使用 isChecked() 查询复选框是否被选中。

  注意：QCheckBox可以有两种状态，也可以有三种状态（未选中，选中，半选），默认是两种状态。

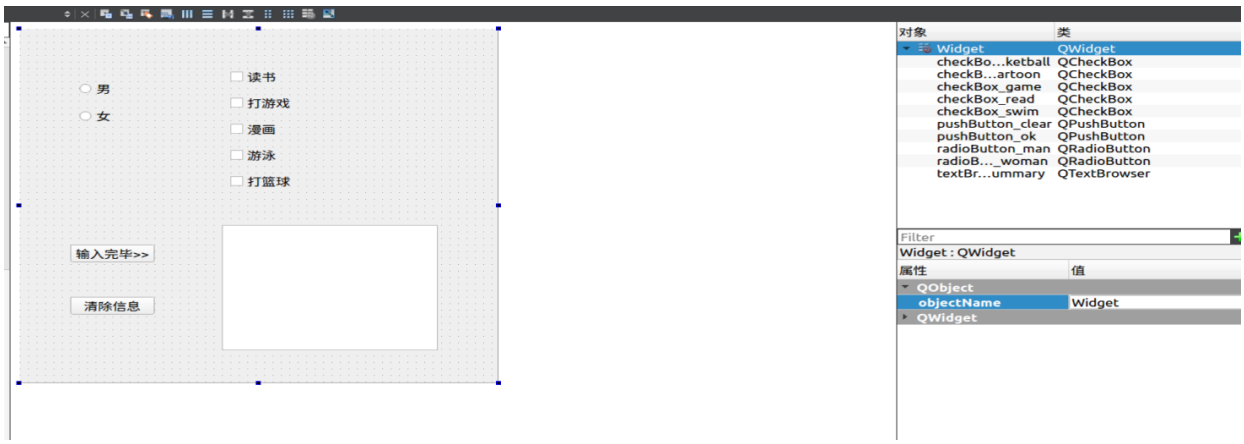
enum Qt::**CheckState**

This enum describes the state of checkable items, controls, and widgets.

Constant	Value	Description
Qt::Unchecked	0	The item is unchecked.
Qt::PartiallyChecked	1	The item is partially checked. Items in hierarchical models may be partially checked if some, but not all, of their children are checked.
Qt::Checked	2	The item is checked.

See also QCheckBox, Qt::ItemFlags, and Qt::ItemDataRole.

创建基于QWidget的工程，如下图所示：



对应的代码：

```
#include "widget.h"
#include "ui_widget.h"
#include <QDebug>

Widget::Widget(QWidget *parent) :
    QWidget(parent),
    ui(new Ui::Widget)
{
    ui->setupUi(this);
#if 0
    /*改变按钮形状*/
    ui->pushButton_ok->resize(80, 80);

    ui->pushButton_ok->setStyleSheet("QPushButton {"
                                     "background-color: green;" //
按钮背景色
                                     "font: bold 20px;"    // 按钮字
体
                                     "border-width: 1px;"  // 按钮边
框线宽
                                     "border-radius: 40px;" // 按钮边
框圆角半径
                                     "color: white;"      // 按钮文字
颜色
                                     "}");
#endif

}

Widget::~Widget()
{
    delete ui;
}

void Widget::on_pushButton_ok_clicked()
{
    QString info = "我是";
    if(ui->radioButton_man->isChecked()){
        info += "男生";
    }
    else if(ui->radioButton_woman->isChecked()){
```

```
        info += "女生";
    }
    else{
        info += "性别不便透露";
    }
    info += "平时喜欢:";
    if(ui->checkBox_game->checkState()){

            info += ui->checkBox_game->text();
            info += ",";
    }
    if(ui->checkBox_read->isChecked()){
            info += ui->checkBox_read->text();
            info += ",";
    }
    if(ui->checkBox_swim->isChecked()){
            info += ui->checkBox_swim->text();
            info += ",";
    }
    if(ui->checkBox_cartoon->isChecked()){
            info += ui->checkBox_cartoon->text();
            info += ",";
    }
    if(ui->checkBox_basketball->isChecked()){
            info += ui->checkBox_basketball->text();
            info += ",";
    }
    int pos = info.lastIndexOf(",");
    info.replace(pos,1,".");

    ui->textBrowser_summary->setText(info);

}

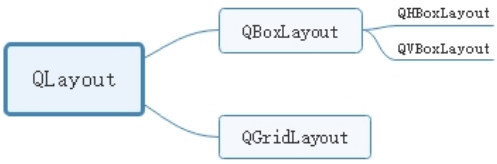
void Widget::on_pushButton_clear_clicked()
{
    //注意此处radiobutton设置为未选中的方式：  鼠标点击该按钮能从选中状态变为未选中状态吗？
    ui->radioButton_man->setAutoExclusive(false);
    ui->radioButton_man->setChecked(false);
    ui->radioButton_man->setAutoExclusive(true);
    ui->radioButton_woman->setAutoExclusive(false);
    ui->radioButton_woman->setChecked(false);
```

```
    ui->radioButton_woman->setAutoExclusive(true);
    ui->checkBox_game->setChecked(false);
    ui->checkBox_read->setChecked(false);
    ui->checkBox_swim->setChecked(false);
    ui->checkBox_cartoon->setChecked(false);
    ui->checkBox_basketball->setChecked(false);
    ui->textBrowser_summary->clear();
}
```

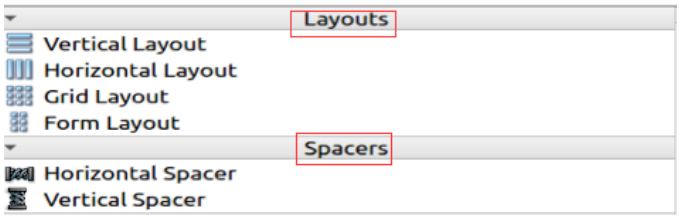
2.5 布局组件

Qt 的界面设计使用了布局（Layout）功能。所谓布局，就是界面上组件的排列方式，使用布局可以使组件有规则地分布，并且随着窗体大小变化自动地调整大小和相对位置。

Qt提供了QHBoxLayout类(水平排列布局)，QVBoxLayout类(垂直排列布局)，QGridLayout类(网格排列布局)等基本布局管理。它们之间的继承关系如下图所示。



在designer软件界面中我们可以看到：



每个组件的作用如下表所示。

布局组件	功能
Vertical Layout	垂直方向布局，组件自动在垂直方向上分布
Horizontal Layout	水平方向布局，组件自动在水平方向上分布
Grid Layout	网格状布局，网状布局大小改变时，每个网格的大小都改变
Form Layout	窗体布局，与网格状布局类似，但是只有最右侧的一列网格会改变大小
Horizontal Spacer	一个用于水平分隔的空格
Vertical Spacer	一个用于垂直分隔的空格

addWidget()方法用于向布局中加入组件。

addLayout()方法用于向布局中加入子布局 。

```
LeftLayout=new QGridLayout();  //由于LeftLayout布局管理器不是主布局管理器，所以不用指定父窗口
QGridLayout *mainLayout=new QGridLayout(this);  //mainLayout实现主布局，所有要指定父窗口
//QHBoxLayout默认采取的是以自左向右的方式顺序排列插入控件或子布局，也可以通过setDirection()方法设定排列的顺序
TopRightLayout->setDirection(QBoxLayout::RightToLeft);
//QHBoxLayout是水平布局，可以设定水平各个控件之间的间距
TopRightLayout=new QHBoxLayout();
TopRightLayout->setSpacing(20);  //设定各个控件之间的间距为20
```

布局如下窗口：



- 代码方式

```
#ifndef WIDGET_H
#define WIDGET_H
#include <QWidget>
#include <QLabel>
#include <QLineEdit>
#include <QRadioButton>
#include <QPushButton>

class Widget : public QWidget
{
    Q_OBJECT

public:
    Widget(QWidget *parent = nullptr);
    ~Widget();

    // 初始化标签
    void initLabel();
```

```
    // 初始化输入框
    void initLineEdit();

    // 初始化布局
    void initLayout();

    // 初始化按钮
    void initBtn();
protected:
    // 标签
    QLabel *nameLabel;
    QLabel *ageLabel;
    QLabel *emailLabel;
    QLabel *numLabel;
    QLabel *sexLabel;

    // 输入框
    QLineEdit *ageLineEdit;
    QLineEdit *nameLineEdit;
    QLineEdit *emailLineEdit;
    QLineEdit *numLineEdit;

    // 单选按钮
    QRadioButton *mBtn;
    QRadioButton *wBtn;
    QPushButton *okBtn;
};
#endif
```

```
#include "widget.h"
#include <QFormLayout>
#include <QHBoxLayout>
#include <QVBoxLayout>

Widget::Widget(QWidget *parent)
    : QWidget(parent)
{
    // 构造函数里逐个调用
    this->initBtn();
    this->initLabel();
    this->initLineEdit();
```

```
    this->initLayout();
}

Widget::~Widget()
{
}

void Widget::initLabel()
{
    this->nameLabel = new QLabel("姓名：(&N)");
    this->ageLabel = new QLabel("年龄：(&A)");
    this->emailLabel = new QLabel("邮箱：(&E)");
    this->numLabel = new QLabel("电话号码：(&N)");

    this->sexLabel = new QLabel("性别");
}

void Widget::initLineEdit()
{
    this->nameLineEdit = new QLineEdit;
    this->ageLineEdit = new QLineEdit;
    this->emailLineEdit = new QLineEdit;
    this->numLineEdit = new QLineEdit;
}

// 初始化布局
void Widget::initLayout()
{
    nameLabel->setBuddy(nameLineEdit);
    ageLabel->setBuddy(ageLineEdit);
    emailLabel->setBuddy(emailLineEdit);

    // 创建一个表格类布局
    QFormLayout *headerLayout = new QFormLayout;

   // addRow函数字面意思，添加一行，比如addRow(nameLabel,nameLineEdie)，就是把这两个控件放到同一行，图片效果可以划到下面的函数效果演示
    headerLayout->addRow(nameLabel, nameLineEdit);
    headerLayout->addRow(ageLabel,  ageLineEdit);
    headerLayout->addRow(emailLabel, emailLineEdit);
    headerLayout->addRow(numLabel, numLineEdit);
```

```
    // 创建水平布局类，意思是横线方向的布局
    QHBoxLayout *sexLayout = new QHBoxLayout;
    sexLayout->addWidget(sexLabel);
    sexLayout->addWidget(mBtn);
    sexLayout->addWidget(wBtn);

    // 空隙
    QSpacerItem *vSpacer = new QSpacerItem(30, 30);
    QSpacerItem *hSpacer = new QSpacerItem(150, 20);

    // 创建水平布局
    QHBoxLayout *okLayout = new QHBoxLayout;
    // 把上面创建的空隙先添加进来
    okLayout->addItem(hSpacer);
    // 把按钮添加到布局
    okLayout->addWidget(okBtn);

    // 创建一个垂直布局，作为主布局，把之前创建的布局都添加到这个布局中
    QVBoxLayout *mainLayout = new QVBoxLayout(this);
    mainLayout->addLayout(headerLayout);
    mainLayout->addLayout(sexLayout);
    mainLayout->addItem(vSpacer);
    mainLayout->addLayout(okLayout);
    mainLayout->setMargin(50);
    mainLayout->setSpacing(10);

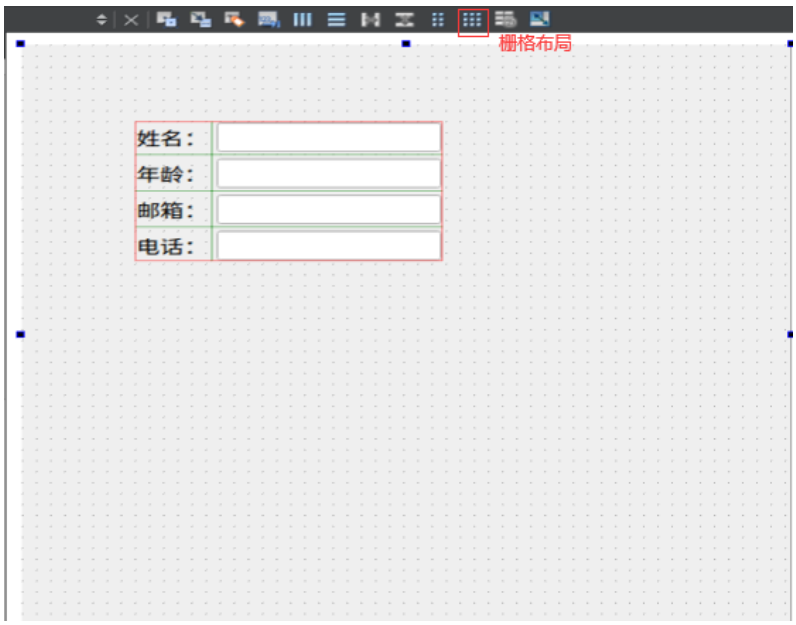
    // 把垂直布局设置为当前界面的
    this->setLayout(mainLayout);
}

void Widget::initBtn()
{

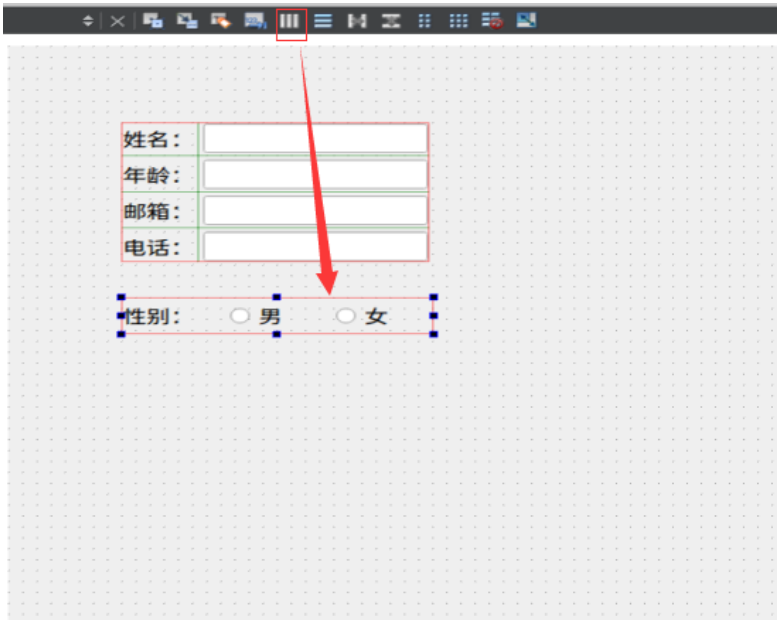
    this->mBtn = new QRadioButton("男");
    this->wBtn = new QRadioButton("女");
    this->okBtn = new QPushButton("确定");
}
```

- 可视化方式

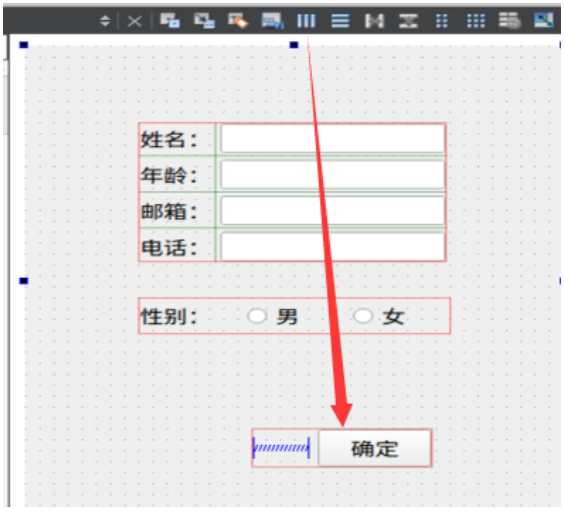
第一步：



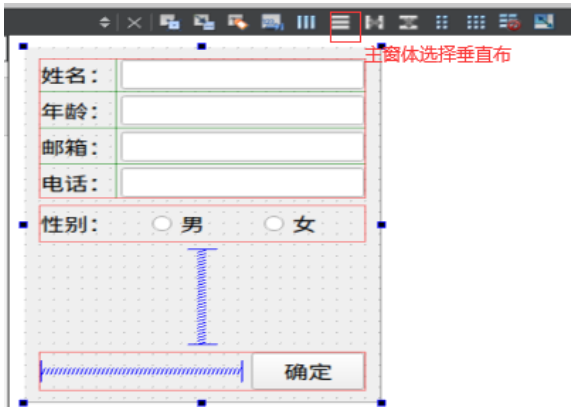
第二步：



第三步：



第四步：



Tips:

```
def initUI(self):
    layout = QHBoxLayout()
    layout.addStretch(1)  # 注意1
    layout.addWidget(QPushButton(str(1))
    layout.addWidget(QPushButton(str(2))
    layout.addStretch(3)  # 注意2
```



2.6 容器组件

为了将界面上的各个组件的分布设计得更加美观，经常使用一些容器类，如QgoupBox、QtabWidget、QToolBox等。

2.6.1 QGroupBox

实例效果：



代码化UI设计：

```
#ifndef WIDGET_H
#define WIDGET_H
```

```
#include <QWidget>
#include <QGroupBox>
#include <QRadioButton>

class Widget : public QWidget
{
    Q_OBJECT

public:
    Widget(QWidget *parent = 0);
    ~Widget();
private:
    QGroupBox *gb1;
    QGroupBox *gb2;
    QRadioButton *rb1;
    QRadioButton *rb2;
    QRadioButton *rb3;
    QRadioButton *rb4;
    QRadioButton *rb5;

};

#endif // WIDGET_H
```

```
#include "widget.h"
#include <QVBoxLayout>
#include <QHBoxLayout>
Widget::Widget(QWidget *parent)
    : QWidget(parent)
{
    gb1 = new QGroupBox("性别",this);
    rb1 = new QRadioButton("男");
    rb2 = new QRadioButton("女");
    QVBoxLayout *vb1 = new QVBoxLayout();
    vb1->addWidget(rb1);
    vb1->addWidget(rb2);
    gb1->setLayout(vb1);


    gb2 = new QGroupBox("所在年级");
    rb3 = new QRadioButton("三年级");
    rb4 = new QRadioButton("四年级");
```

```
    rb5 = new QRadioButton("五年级");
    QVBoxLayout *vb2 = new QVBoxLayout;
    vb2->addWidget(rb3);
    vb2->addWidget(rb4);
    vb2->addWidget(rb5);
    gb2->setLayout(vb2);


    QHBoxLayout *mainLayout = new QHBoxLayout(this);
    mainLayout->addWidget(gb1);
    mainLayout->addWidget(gb2);
    setLayout(mainLayout);
}

Widget::~Widget()
{

}
```

可视化UI设计



2.6.2 QScrollArea

实例效果:



代码化UI设计：

```
#ifndef WIDGET_H
#define WIDGET_H

#include <QWidget>
#include <QLabel>
#include <QScrollArea>

class Widget : public QWidget
{
    Q_OBJECT

public:
    Widget(QWidget *parent = 0);
    ~Widget();
private:
    QLabel *lb;
    QScrollArea *sa;
};

#endif // WIDGET_H
```

```
#include "widget.h"
#include <QVBoxLayout>

Widget::Widget(QWidget *parent)
    : QWidget(parent)
{
    lb = new QLabel(this);
```

```
    QPixmap pix("1.png");
    lb->setPixmap(pix);

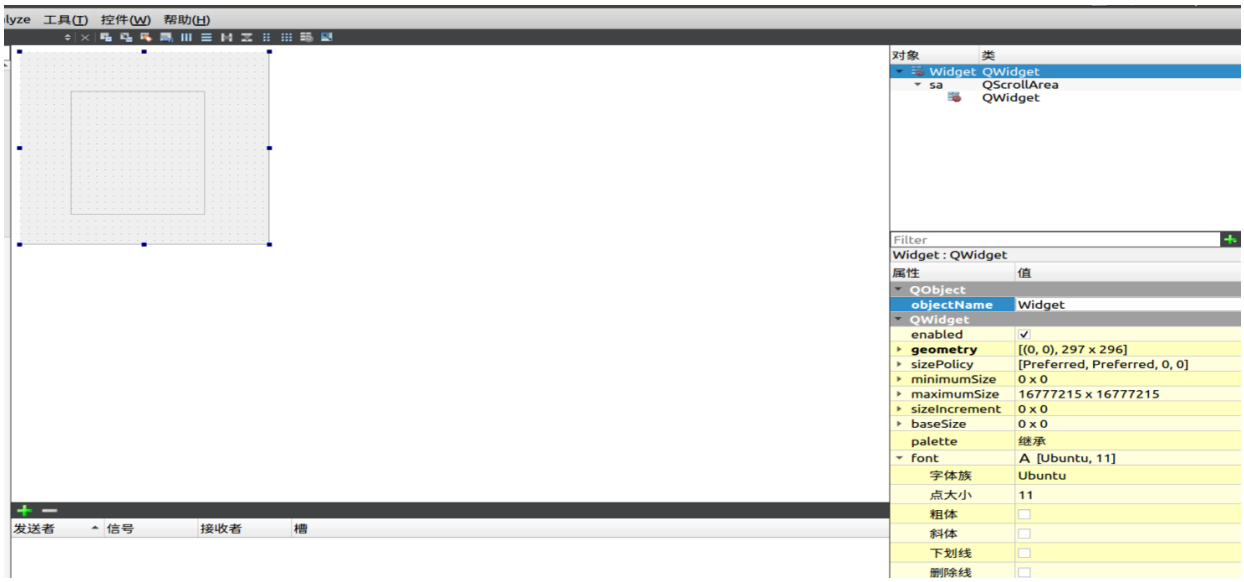
    sa = new QScrollArea(this);
    sa->setWidget(lb);

    QVBoxLayout *mainbox = new QVBoxLayout;
    mainbox->addWidget(sa);
    this->setLayout(mainbox);
}

Widget::~Widget()
{

}
```

可视化UI设计




```
#include "widget.h"
#include "ui_widget.h"
#include <QLabel>
#include <QHBoxLayout>

Widget::Widget(QWidget *parent) :
    QWidget(parent),
    ui(new Ui::Widget)
{
    ui->setupUi(this);
    QLabel *lb = new QLabel(this);
    QPixmap pix("1.png");
    lb->setPixmap(pix);
```

```
    ui->sa->setWidget(lb);

    QVBoxLayout *mainbox = new QVBoxLayout;
    mainbox->addWidget(ui->sa);
    setLayout(mainbox);
}

Widget::~Widget()
{
    delete ui;
}
```

2.6.3 QToolBox

QToolBox类提供了一列选项卡窗口，当前项显示在当前选项卡下面，适用于**分类浏览**、**内容展示**、**操作指引**这一类的使用场景。



代码化UI设计：

```
#include "widget.h"
#include <QPushButton>
#include <QToolBox>
#include <QHBoxLayout>

Widget::Widget(QWidget *parent)
    : QWidget(parent)
{
    ui->setupUi(this);

    QToolBox *tb = new QToolBox(this);

    QWidget *page1 = new QWidget(this);
    QVBoxLayout *layout1 = new QVBoxLayout(this);
    QPushButton *pb1 = new QPushButton("大话西游", page1);
    QPushButton *pb2 = new QPushButton("功夫", page1);
```

```
    QPushButton *pb3 = new QPushButton("整蛊专家", page1);
    layout1->addWidget(pb1);
    layout1->addWidget(pb2);
    layout1->addWidget(pb3);
    page1->setLayout(layout1);
    tb->addItem(page1, "周星驰");

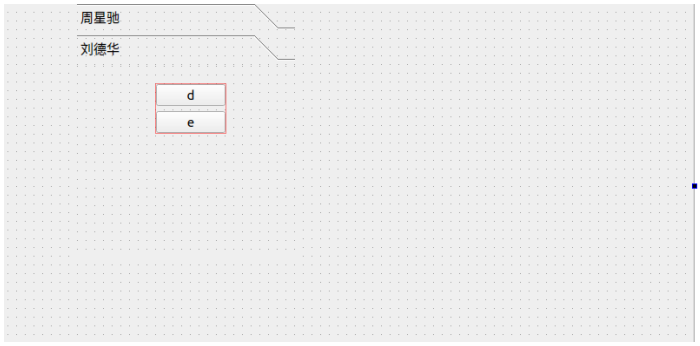
    QWidget *page2 = new QWidget(this);
    QVBoxLayout *layout2 = new QVBoxLayout(this);
    QPushButton *pb5 = new QPushButton("无间道", page2);
    QPushButton *pb6 = new QPushButton("拆弹专家", page2);
    layout2->addWidget(pb5);
    layout2->addWidget(pb6);
    page2->setLayout(layout2);
    tb->addItem(page2, "刘德华");

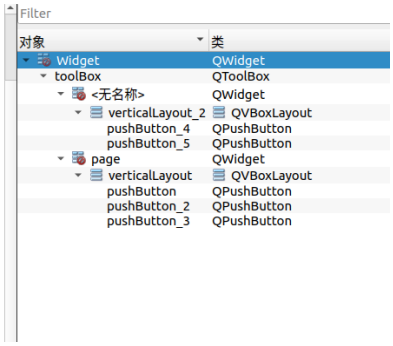

    QHBoxLayout *mainLayout = new QHBoxLayout;
    mainLayout->addWidget(tb);
    setLayout(mainLayout);

}

Widget::~Widget()
{
    delete ui;
}
```

可视化UI设计

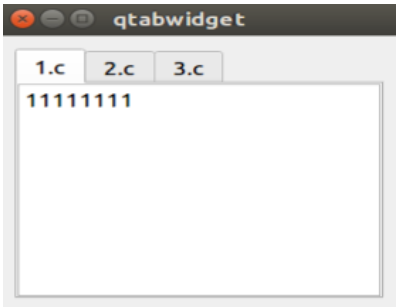




2.6.4 QTabWidget

QTabWidget 的一种带标签页的窗口，在这种类型的窗口中可以存储多个子窗口，每个子窗口的显示可以通过对应的标签进行切换。

实例效果



代码化UI设计

```
#include "widget.h"
#include <QVBoxLayout>

Widget::Widget(QWidget *parent)
    : QWidget(parent)
{
    tw = new QTabWidget;

    te1 = new QTextEdit("11111111");
    te2 = new QTextEdit("112221111");
    te3 = new QTextEdit("11113333");

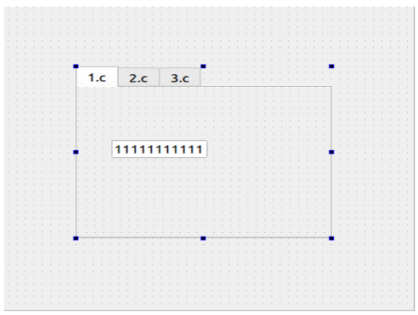
    tw->addTab(te1, "1.c");
    tw->addTab(te2, "2.c");
    tw->addTab(te3, "3.c");

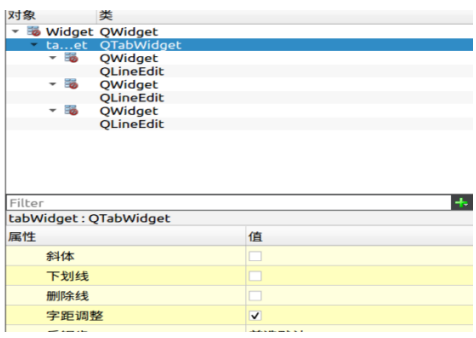
    QVBoxLayout *mainbox = new QVBoxLayout;
    mainbox->addWidget(tw);
    this->setLayout(mainbox);
}

Widget::~Widget()
{

}
```

可视化UI设计





2.6.5 QStackedWidget

QStackedWidget列提供了堆栈窗体的效果。在实际应用中，堆栈窗体多与列表框QlistWidget和下拉列表框QCombobox配合使用。

实例效果



代码化UI设计

```
#include "widget.h"
#include <QVBoxLayout>

Widget::Widget(QWidget *parent)
    : QWidget(parent)
{
    sw = new QStackedWidget;

    te1 = new QTextEdit("aaaa");
    te2 = new QTextEdit("aaadsfasdfasdfa");
    te3 = new QTextEdit("aaasdfaaa");

    sw->addWidget(te1);
    sw->addWidget(te2);
    sw->addWidget(te3);

    cb = new QComboBox;
    cb->addItem("1111");
    cb->addItem("2222");
    cb->addItem("3333");

    QVBoxLayout *mainbox = new QVBoxLayout;
    mainbox->addWidget(sw);
    mainbox->addWidget(cb);
    this->setLayout(mainbox);

    connect(cb, SIGNAL(activated(int)), sw,
SLOT(setCurrentIndex(int)));
```

```
}

Widget::~Widget()
{

}
```

可视化UI设计



2.7 时间日期与定时器

2.7.1 时间日期相关的类

时间日期是经常遇到的数据类型，Qt中时间日期类型的类如下：

- QTime ，时间数据类型， 仅表示时间， 如18:08:08。
- QDate， 日期数据类型， 仅表示日期， 如2023-7-21。
- QDateTime ，日期时间数据类型，表示日期和时间， 如2023-07-21 18:08:08。

QT中有专门用于日期、时间编辑和显示的界面组件，如下：

- QTimeEdit， 编辑显示时间的组件。
- QDateEdit， 编辑和显示日期的组件。
- QDateTimeEdit, 编辑和显示日期时间的组件。
- QCalendarWidget, 一个用日历形式选择日期日期的组件。

定时器是用来处理周期性事件的一种对象，类似于硬件定时器。例如设置一个定时器的定时周期为1000毫秒，那么每1000毫秒就会发射定时器的timeout()信号，在信号关联的槽函数里就可以做相应的处理。Qt中的定时器类为QTimer,它直接从QObject类继承而来，不是界面组件类。

2.7.2 日期时间数据与字符串之间的转换

设计如下图所示对话框：



QDateEdit和QTimeEdit都从QDateTimeEdit继承而来，实现针对日期或时间的特定显示功能。实际上QDateEdit和QTimeEdit的显示功能都可以通过QDateTimeEdit实现，只需要设置好属性即可。



时间日期转字符串的函数：

```
QString QDateTime::toString(const QString &format) const
```

设置“读取当前日期时间”按钮对应的槽函数

```
#include "widget.h"
#include "ui_widget.h"
#include <QDateTime>

Widget::Widget(QWidget *parent) :
    QWidget(parent),
    ui(new Ui::Widget)
{
```

```
    ui->setupUi(this);
}

Widget::~Widget()
{
    delete ui;
}

void Widget::on_btnGetTime_clicked()
{
    //获取当期日期时间
    QDateTime curDateTime = QDateTime::currentDateTime();
    ui->timeEdit->setTime(curDateTime.time());
    ui->editTime->setText(curDateTime.toString("hh:mm:ss"));
    ui->dateEdit->setDate(curDateTime.date());
    ui->editDate->setText(curDateTime.toString("yyyy-MM-dd"));
    ui->dateTimeEdit->setDateTime(curDateTime);
    ui->editDateTIme->setText(curDateTime.toString("yyyy-MM-dd
hh:mm:ss"));
}
```

字符串转时间日期的函数：

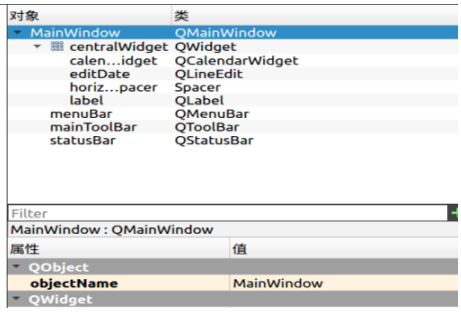
```
QDateTime QDateTime::fromString(const QString &string, const
QString &format)
```

```
void Widget::on_btnSetDateTime_clicked()
{
    QString str = ui->editDateTime->text();
    str = str.trimmed();
    if(!str.isEmpty()){
        QDateTime datetime = QDateTime::fromString(str, "yyyy年MM
月dd日 hh:mm:ss");
        ui->dateTimeEdit->setDateTime(datetime);
    }
}
```

课后作业：仿照“设置日期时间”按钮， 完成“设置时间” “设置日期” 按钮对应的槽函数。

2.7.3 QCalendarWidget日历组件






```
#include "mainwindow.h"
#include "ui_mainwindow.h"

MainWindow::MainWindow(QWidget *parent) :
    QMainWindow(parent),
    ui(new Ui::MainWindow)
{
    ui->setupUi(this);
    connect(ui->calendarWidget, SIGNAL(selectionChanged()),
            this, SLOT(on_calendar_changed()));

}

MainWindow::~MainWindow()
{
    delete ui;
}
void MainWindow::on_calendar_changed(){
    QDate date = ui->calendarWidget->selectedDate();
    ui->editDate->setText(date.toString("yyyy-MM-dd"));
}
```

2.7.4 定时器

Qt中的定时器类是QTimer。QTimer不是一个可见的界面组件，在UI设计器的组件面板里找不到它。QTimer主要的属性是interval,是定时中断的周期，单位是毫秒。QTimer主要的信号是timeout(),在定时中断时发射此信号，要想在定时中断里做出响应，这就需要编写timeout()信号的槽函数。

QElapsedTimer 用于快速计算两个事件的间隔时间，它没有父类，不支持 Qt 的元对象系统,QElapsedTimer 的主要接口函数定义如下：

```
void start() //复位并启动计时器
qint64 elapsed() //返回流逝的时间，单位：毫秒
qint64 nsecsElapsed() //返回流逝的时间，单位：纳秒
qint64 restart() //重新启动计时器
```




```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>
#include <QTimer>
#include <QElapsedTimer>


namespace Ui {
class MainWindow;
}

class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    explicit MainWindow(QWidget *parent = nullptr);
    ~MainWindow();

private:
    Ui::MainWindow *ui;
```

```
    QTimer timer;
    QElapsedTimer counter;
private slots:
    void on_timeout();
    void on_btnStart_clicked();
    void on_btnStop_clicked();
    void on_sbInter_valueChanged(const QString &arg1);
};

#endif // MAINWINDOW_H
```

```
#include "mainwindow.h"
#include "ui_mainwindow.h"
#include <QDebug>


MainWindow::MainWindow(QWidget *parent) :
    QMainWindow(parent),
    ui(new Ui::MainWindow)
{
    ui->setupUi(this);
    timer.setInterval(ui->sbInter->value());
    bool flag = connect(&timer, SIGNAL(timeout()), this,
SLOT(on_timeout()));
    if(flag){
        qDebug()<<"successed";
    }
    else {
        qDebug()<<"failed";
    }
}

MainWindow::~MainWindow()
{
    delete ui;
}

void MainWindow::on_timeout(){
    QTime curTime = QTime::currentTime();
    ui->LCDH->display(curTime.hour());
    ui->LCDM->display(curTime.minute());
```

```
    ui->LCDS->display(curTime.second());

}

void MainWindow::on_btnStart_clicked()
{
    timer.start();
    counter.start();
    ui->btnStart->setEnabled(false);
    ui->btnStop->setEnabled(true);
    ui->sbInter->setEnabled(false);

}

void MainWindow::on_btnStop_clicked()
{
    timer.stop();

    int msec = counter.elapsed();
    int ms = msec%1000;
    int sec = msec/1000;
    QString text = QString::asprintf("%d秒,%d毫秒", sec, ms);

    ui->letimes->setText(text);
    ui->btnStop->setEnabled(false);
    ui->btnStart->setEnabled(true);
    ui->sbInter->setEnabled(true);

}

void MainWindow::on_sbInter_valueChanged(const QString &arg1)
{
        timer.setInterval(arg1.section(' ',0,0).toInt());
}
```

3. 对话框与多窗体设计

一个完整的应用程序设计中，不可避免地会涉及多个窗体、对框框的设计和调用，如何设计和调用这些对话框和窗体是搞清楚一个庞大的应用程序设计的基础。本章将介绍对话框和多窗体设计、调用方式、数据传递等问题。

3.1 标准对话框

Qt为应用程序设计提供了一些常用的标准对话框，如打开文件对话框、选择颜色对话框、信息提示和确认选择对话框、标准输入对话框等，用户无需再自己设计这些常用的对话框，这样可以减少程序设计工作量。

3.1.1 QFileDialog对话框

QFileDialog Class

The QFileDialog class provides a dialog that allow users to select files or directories. More...

Header: #include <QFileDialog>

qmake: QT += widgets

Inherits: QDialog

- List of all members, including inherited members
- Obsolete members

选择打开一个文件

若要打开一个文件，可调用静态函数

```
[static] QString QFileDialog::getOpenFileName()
//example
//该函数返回用户选择的文件名称， 若用户取消返回null
//父窗口， 窗口标题，初始目录，文件过滤器
QString fileName = QFileDialog::getOpenFileName(this, tr("Open
File"),"/home",
                                                tr("Images (*.png
*.xpm *.jpg)"));
```

实例：




```
#include "mainwindow.h"
```

```
#include "ui_mainwindow.h"
#include <QDir>
#include <QFileDialog>

MainWindow::MainWindow(QWidget *parent) :
    QMainWindow(parent),
    ui(new Ui::MainWindow)
{
    ui->setupUi(this);

}

MainWindow::~MainWindow()
{
    delete ui;
}

void MainWindow::on_pushButton_clicked()
{
    QString curPath = QDir::currentPath();
    QString dlgTitle = "选择一个文件";
    QString filter = "文本文件(*.txt);;图片文件(*.jpg *.gif);;所有文件(*.*)";
    QString aFileName = QFileDialog::getOpenFileName(this, dlgTitle, curPath, filter);

    if(!aFileName.isEmpty()){
        ui->plainTextEdit->appendPlainText(aFileName);
    }

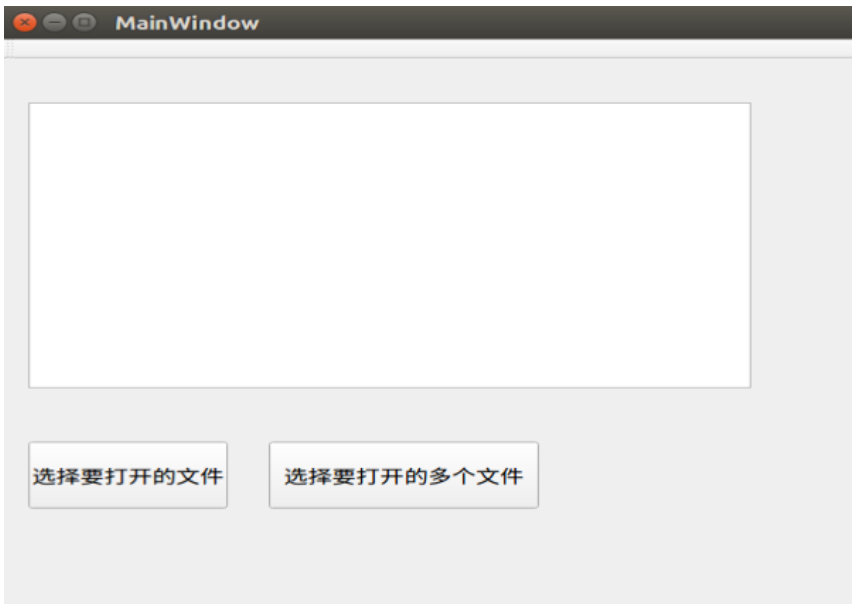
}
```

选择打开多个文件

```
[static] QStringList QFileDialog::getOpenFileNames()

//example
QStringList files = QFileDialog::getOpenFileNames(
                            this,
                            "Select one or more files to open",
                            "/home",
                            "Images (*.png *.xpm *.jpg)");
```

实例:




```
void MainWindow::on_btn_select_files_clicked()
{
    QString curPath = QDir::currentPath();
    QString dlgTitle = "选择多个文件";
    QString filter = "文本文件(*.txt);;图片文件(*.jpg *.gif);;所有文件(*.*)";
    QStringList files = QFileDialog::getOpenFileNames(this, dlgTitle,curPath, filter);

    for(int i=0; i<files.count(); i++){
        ui->plainTextEdit->appendPlainText(files.at(i));
    }
}
```

选择已有目录

```
QString QFileDialog::getExistingDirectory()
//example
QString dir = QFileDialog::getExistingDirectory(this, tr("Open Directory"),
                                                 "/home",

QFileDialog::ShowDirsOnly|QFileDialog::DontResolveSymlinks);
```

实例:




```
void MainWindow::on_btn_select_dir_clicked()
{
    QString curPath = QDir::currentPath();
    QString dlgTitle = "选择目录";
    QString dirName = QFileDialog::getExistingDirectory(this,
dlgTitle,
                                                        curPath,

 QFileDialog::ShowDirsOnly);
    if(!dirName.isEmpty()){
        ui->plainTextEdit->appendPlainText(dirName);
    }
}
```

选择保存文件名

```
 QString QFileDialog::getSaveFileName()
 //example
 QString fileName = QFileDialog::getSaveFileName(this, tr("Save
File"),

"/home/jana/untitled.png",
                                                 tr("Images (*.png
*.xpm *.jpg)"));
```

实例:

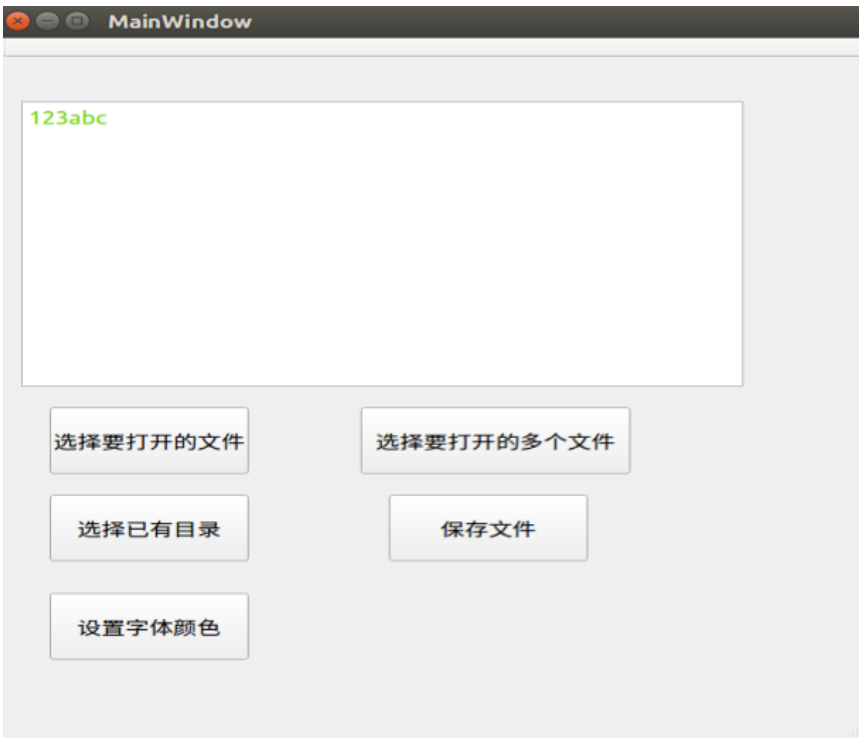



```
void MainWindow::on_btn_save_file_clicked()
{
    QString curPath = QDir::currentPath();
    QString dlgTitle = "保存文件";
    QString filter = "h文件(*.h);;C++文件(*.cpp);;所有文件(*.*)";
    QString filename = QFileDialog::getSaveFileName(this,
dlgTitle,
                                                    curPath,
filter);
    if(!filename.isEmpty()){
        ui->plainTextEdit->appendPlainText(filename);
    }
}
```

3.1.2 QColorDialog对话框

```
[static] QColor QColorDialog::getColor(const QColor &initial =
Qt::white, QWidget *parent = nullptr, const QString &title =
QString(), QColorDialog::ColorDialogOptions options = ...)
```

实例:




```
void MainWindow::on_btn_set_color_clicked()
{
    QPalette pal = ui->plainTextEdit->palette();
    QColor initColor = pal.color(QPalette::Text); //现有文字的颜色
    QColor color = QColorDialog::getColor(initColor, this, "选择颜
色");

    if(color.isValid()){
        pal.setColor(QPalette::Text, color);
        ui->plainTextEdit->setPalette(pal);
    }
}
```

3.1.3 QFontDialog对话框

```
[static] QFont QFontDialog::getFont(bool *ok, const QFont
&initial, QWidget *parent = nullptr, const QString &title =
QString(), QFontDialog::FontDialogOptions options = ...)
//example
bool ok;
QFont font = QFontDialog::getFont(&ok, QFont("Times", 12), this);
if (ok) {
    // font is set to the font the user selected
} else {
    // the user canceled the dialog; font is set to the initial
    // value, in this case Times, 12.
}
```

实例:

```
#include "mainwindow.h"
#include "ui_mainwindow.h"
#include <QDir>
#include <QFileDialog>
#include <QStringList>
#include <QPalette>
#include <QColorDialog>
#include <QFontDialog>

MainWindow::MainWindow(QWidget *parent) :
    QMainWindow(parent),
    ui(new Ui::MainWindow)
{
    ui->setupUi(this);

}

MainWindow::~MainWindow()
{
    delete ui;
}

void MainWindow::on_pushButton_clicked()
{
    QString curPath = QDir::currentPath();
    QString dlgTitle = "选择一个文件";
    QString filter = "文本文件(*.txt);;图片文件(*.jpg *.gif);;所有文
件(*.*)";
    QString aFileName = QFileDialog::getOpenFileName(this,
dlgTitle, curPath, filter);

    if(!aFileName.isEmpty()){
        ui->plainTextEdit->appendPlainText(aFileName);
    }

}

void MainWindow::on_btn_select_files_clicked()
{
    QString curPath = QDir::currentPath();
```

```
    QString dlgTitle = "选择多个文件";
    QString filter = "文本文件(*.txt);;图片文件(*.jpg *.gif);;所有文
件(*.*)";
    QStringList files = QFileDialog::getOpenFileNames(this,
dlgTitle,curPath, filter);

    for(int i=0; i<files.count(); i++){
        ui->plainTextEdit->appendPlainText(files.at(i));
    }
}

void MainWindow::on_btn_select_dir_clicked()
{
    QString curPath = QDir::currentPath();
    QString dlgTitle = "选择目录";
    QString dirName = QFileDialog::getExistingDirectory(this,
dlgTitle,
                                                        curPath,

 QFileDialog::ShowDirsOnly);
    if(!dirName.isEmpty()){
        ui->plainTextEdit->appendPlainText(dirName);
    }
}

void MainWindow::on_btn_save_file_clicked()
{
    QString curPath = QDir::currentPath();
    QString dlgTitle = "保存文件";
    QString filter = "h文件(*.h);;C++文件(*.cpp);;所有文件(*.*)";
    QString filename = QFileDialog::getSaveFileName(this,
dlgTitle,
                                                    curPath,
filter);
    if(!filename.isEmpty()){
        ui->plainTextEdit->appendPlainText(filename);
    }
}

void MainWindow::on_btn_set_color_clicked()
{
    QPalette pal = ui->plainTextEdit->palette();
    QColor initColor = pal.color(QPalette::Text); //现有文字的颜色
```

```
    QColor color = QColorDialog::getColor(initColor, this, "选择颜色");

    if(color.isValid()){
        pal.setColor(QPalette::Text, color);
        ui->plainTextEdit->setPalette(pal);
    }
}

void MainWindow::on_btn_set_font_clicked()
{
    QFont initFont = ui->plainTextEdit->font();
    bool ok = false;
    QFont  font = QFontDialog::getFont(&ok, initFont);
    if(ok){
        ui->plainTextEdit->setFont(font);
    }
}
```

3.1.4 QInputDialog标准输入对话框

QInputDialog有单行字符串输入、整数输入、浮点数输入、列表框选择和多行文本等多种输入方式。

文字输入

```
[static] QString QInputDialog::getText(QWidget *parent, const QString &title, const QString &label, QLineEdit::EchoMode mode = QLineEdit::Normal, const QString &text = QString(), bool *ok = nullptr, Qt::WindowFlags flags = ..., Qt::InputMethodHints inputMethodHints = Qt::ImhNone)
    parent, 父窗口
    titile, 显示的标题
    label, 用户输入提示信息
    mode, 输入的回显模式
    text, 默认输入文本
    ok, 判断用户按下了 确定还是 取消
    flags, 对话框模式
    inputMethodHints, 输入提示方式
//example
 bool ok;
QString text = QInputDialog::getText(this, tr("QInputDialog::getText()"),
```

```
                                        tr("User name:"),
QLineEdit::Normal,
                                        QDir::home().dirName(),
&ok);
if (ok && !text.isEmpty())
    textLabel->setText(text);
```

实例:




```
#include "mainwindow.h"
#include "ui_mainwindow.h"
#include <QLineEdit>
#include <QInputDialog>

MainWindow::MainWindow(QWidget *parent) :
    QMainWindow(parent),
    ui(new Ui::MainWindow)
{
    ui->setupUi(this);
}

MainWindow::~MainWindow()
{
    delete ui;
}

void MainWindow::on_btn_input_string_clicked()
{
    QString dlgTitle = "输入文字对话框";
    QString txtLabel = "请输入文件名";
    QString defaultInput = "新建文件.txt";
```

```
    QLineEdit::EchoMode echoMode = QLineEdit::Normal;
    bool ok = false;

    QString text = QInputDialog::getText(this, dlgTitle,
txtLabel,
                                         echoMode, defaultInput,
&ok);
    if(ok && !text.isEmpty()){
        ui->plainTextEdit->appendPlainText(text);
    }
}
```

输入整数

实例




```
void MainWindow::on_btn_input_int_clicked()
{
    QString dlgTitle = "输入整数对话框";
    QString txtLabel = "设置字体大小";
    int defaultVal = ui->plainTextEdit->font().pointSize();
    int minVal = 6, maxVal = 50, stepVal = 1;
    bool ok = false;
    int inputVal = QInputDialog::getInt(this, dlgTitle, txtLabel,
                                        defaultVal,
minVal,maxVal,stepVal, &ok);
    if(ok){
        QFont font = ui->plainTextEdit->font();
        font.setPixelSize(inputVal);
        ui->plainTextEdit->setFont(font);
```

```
    }
}
```

输入浮点数

```
[static] double QInputDialog::getDouble(QWidget *parent, const
QString &title, const QString &label, double value = 0, double
min = -2147483647, double max = 2147483647, int decimals = 1,
bool *ok = nullptr, Qt::WindowFlags flags = ...)
```

实例:




```
void MainWindow::on_btn_input_double_clicked()
{
    QString dlgTitle = "输入浮点数对话框";
    QString txtLabel = "输入一个浮点数";
    double defaultVal = 3.14;
    double minVal =0, maxVal = 10000;
    int decimals = 2;
    bool ok = false;

    double inputVal = QInputDialog::getDouble(this, dlgTitle,
txtLabel,
                                              defaultVal, minVal,
maxVal,
                                              decimals, &ok);
    if(ok){
        QString str = QString::asprintf("输入了一个浮点数:%.2f",
inputVal);
        ui->plainTextEdit->appendPlainText(str);
    }
}
```

下拉列表选择输入

```
[static] QString QInputDialog::getItem(QWidget *parent, const
QString &title, const QString &label, const QStringList &items,
int current = 0, bool editable = true, bool *ok = nullptr,
Qt::WindowFlags flags = ..., Qt::InputMethodHints
inputMethodHints = Qt::ImhNone)
```

实例:




```
void MainWindow::on_btn_input_listitem_clicked()
{
    QStringList items;
    items << "优秀" <<"良好" << "合格" << "不及格";
    QString dlgTitle = "条目选择对话框";
    QString txtLabel = "请选择级别";
    int curIndex = 0; //初始选择项
    bool editable = true; //列表框是否可编辑
    bool ok = false;

    QString text = QInputDialog::getItem(this, dlgTitle,
txtLabel,
                                         items, curIndex,
editable, &ok);
    if(ok && !text.isEmpty()){
        ui->plainTextEdit->appendPlainText(text);
    }

}
```

3.1.5 QMessageBox消息对话框

简单信息提示

消息对话框QMessageBox用于显示提示、警告、错误等信息，或者进行确认选择，由几个静态函数实现这些功能：

```
[static] QMessageBox::StandardButton
QMessageBox::information(QWidget *parent, const QString &title,
const QString &text, QMessageBox::StandardButtons buttons,
QMessageBox::StandardButton defaultButton = NoButton);
[static] QMessageBox::StandardButton QMessageBox::warning(QWidget
*parent, const QString &title, const QString &text,
QMessageBox::StandardButtons buttons, QMessageBox::StandardButton
defaultButton = NoButton);
[static] QMessageBox::StandardButton critical(QWidget *parent,
const QString &title, const QString &text,
QMessageBox::StandardButtons buttons, QMessageBox::StandardButton
defaultButton = NoButton);
[static]void about(QWidget *parent, const QString &title, const
QString &text);
```

实例：




```
#include "mainwindow.h"
#include "ui_mainwindow.h"
#include <QMessageBox>

MainWindow::MainWindow(QWidget *parent) :
    QMainWindow(parent),
    ui(new Ui::MainWindow)
{
    ui->setupUi(this);
```

```
}

MainWindow::~MainWindow()
{
    delete ui;
}

void MainWindow::on_btn_info_clicked()
{
    QString dlgTitle = "information消息框";
    QString strInfo = "文件已经打开";
    QMessageBox::information(this, dlgTitle, strInfo,
QMessageBox::Ok,
                             QMessageBox::NoButton);
}

void MainWindow::on_btn_about_clicked()
{
    QString dlgTitle = "about消息框";
    QString strInfo = "V1.0\n 保留所有版权";
    QMessageBox::about(this, dlgTitle, strInfo);
}

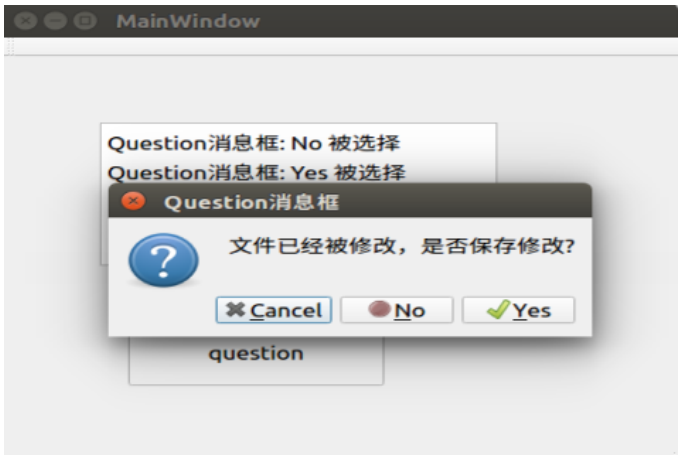
void MainWindow::on_btn_warning_clicked()
{
    QString dlgTitle = "warning消息对话框";
    QString strInfo = "文件内容被修改";
    QMessageBox::warning(this, dlgTitle, strInfo);
}

void MainWindow::on_btn_critical_clicked()
{
    QString dlgTitle = "critical消息框";
    QString strInfo = "不明程序";
    QMessageBox::critical(this, dlgTitle, strInfo);
}
```

确认选择对话

```
[static]QMessageBox::StandardButton question(QWidget *parent,
const QString &title, const QString &text,
QMessageBox::StandardButtons buttons = ...,
QMessageBox::StandardButton defaultButton = NoButton);
```

实例:




```
void MainWindow::on_btn_question_clicked()
{
    QString dlgTitle = "Question消息框";
    QString strInfo = "文件已经被修改，是否保存修改?";
    QMessageBox::StandardButton result ;
    QString str;
    result = QMessageBox::question(this, dlgTitle, strInfo,

QMessageBox::Yes|QMessageBox::No|QMessageBox::Cancel,
                                   QMessageBox::NoButton);
    if(result == QMessageBox::Yes){
        str = "Yes";
    }
    else if(result == QMessageBox::No){
        str = "No";
    }
    else if(result == QMessageBox::Cancel){
        str = "Cancel";
    }
    else{
        str = "无按钮";
    }
    ui->plainTextEdit-
>appendPlainText(QString::asprintf("Question消息框: %s 被选择",

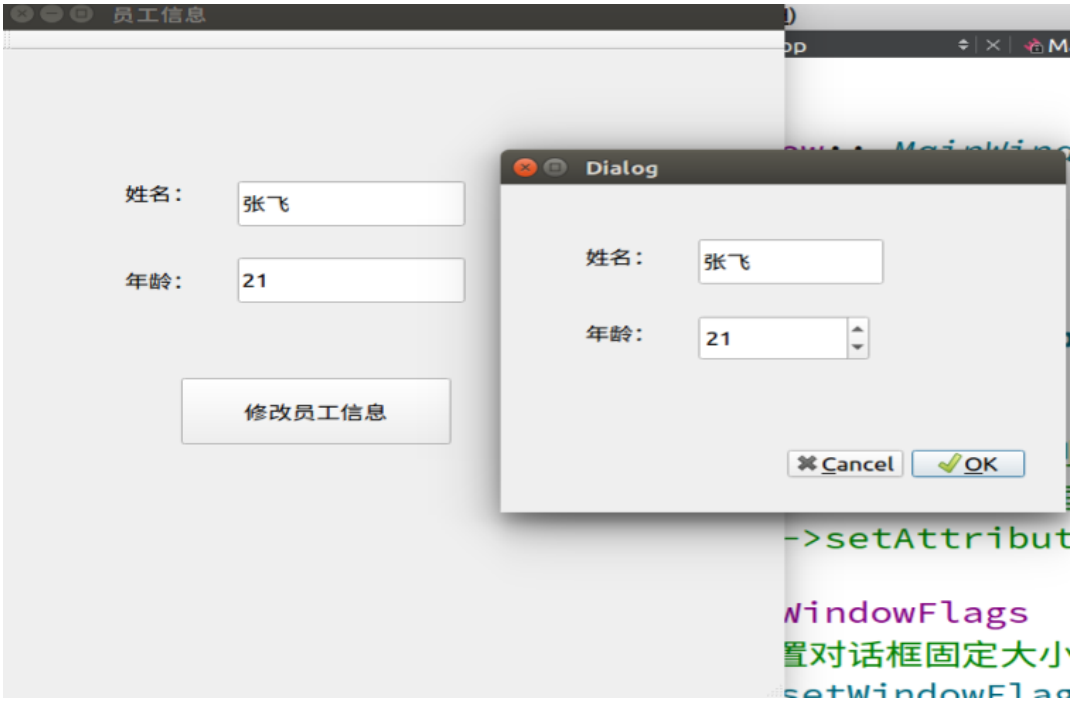
str.toStdString().c_str()));
```

```
}
```

3.2 自定义对话框及其调用

在一个应用程序设计中，为了实现一些特定的功能，必须设计自定义对话框。自定义对话框的设计一般从QDialog继承，并且可以采用UI设计器可视化地设计对话框。对话框的调用一般包括创建对话框、传递数据给对话框、显示对话框获取输入、判断对话框单击按钮的返回类型、获取对话框输入数据等过程。

实例：




```
#ifndef MODIFYDIALOG_H
#define MODIFYDIALOG_H

#include <QDialog>

namespace Ui {
class ModifyDialog;
}

class ModifyDialog : public QDialog
{
    Q_OBJECT

public:
    explicit ModifyDialog(QWidget *parent = nullptr);
    ~ModifyDialog();

private:
```

```
    Ui::ModifyDialog *ui;

public:
    void set_name_age(const QString&, int);
    QString get_name();
    int get_age();


};

#endif // MODIFYDIALOG_H
```

```
#include "modifydialog.h"
#include "ui_modifydialog.h"

ModifyDialog::ModifyDialog(QWidget *parent) :
    QDialog(parent),
    ui(new Ui::ModifyDialog)
{
    ui->setupUi(this);
}

ModifyDialog::~ModifyDialog()
{
    delete ui;
}

void ModifyDialog::set_name_age(const QString &name, int age){

    ui->sb_age->setValue(age);
    ui->le_name->setText(name);
}

QString ModifyDialog::get_name(){
    return ui->le_name->text();

}
int ModifyDialog::get_age(){
    return ui->sb_age->value();
}
```

```
#include "mainwindow.h"
```

```cpp
#include "ui_mainwindow.h"
#include "modifydialog.h"

MainWindow::MainWindow(QWidget *parent) :
    QMainWindow(parent),
    ui(new Ui::MainWindow)
{
    ui->setupUi(this);
}

MainWindow::~MainWindow()
{
    delete ui;
}

void MainWindow::on_pushButton_clicked()
{
    ModifyDialog *md = new ModifyDialog(this);
    //对话框关闭时自动删除对话框对象,用于不需要读取返回值的对话框
    //如果需要获取对话框的返回值，不能设置该属性，可以在调用完对话框后删除对话框
    //md->setAttribute(Qt::WA_DeleteOnClose);

    Qt::WindowFlags    flags=md->windowFlags();
    //设置对话框固定大小
    md->setWindowFlags(flags | Qt::MSWindowsFixedSizeDialogHint);

    //设置自定义对话框中启动时显示的默认值
    md->set_name_age(ui->le_name->text(),ui->le_age->text().toInt());

    int ret = md->exec();
    if(ret == QDialog::Accepted){
        ui->le_age->setText(QString::asprintf("%d",md->get_age()));
        ui->le_name->setText(md->get_name());
    }

    delete md;

}
```

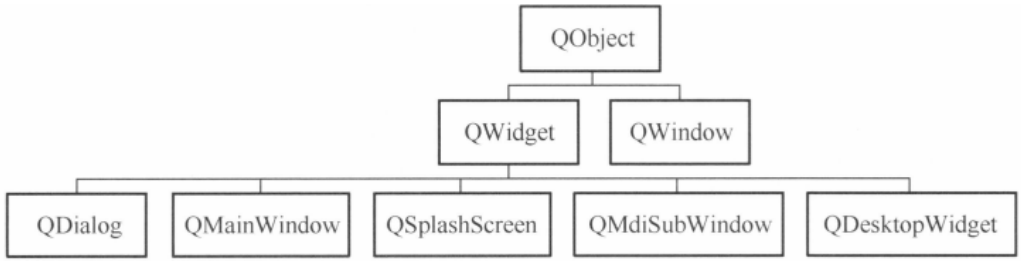
小结：

如果主程序中需要自定义窗口的信息，那就不要调用setAttribute(Qt::WA_DeleteOnClose)设定自定义窗口结束后自动销毁。当使用完毕后可以通过delete的方式

自定义窗口在运行期间想获取更多主窗口中的信息，可以通过parentWidget()函数获取到主窗口句柄。

3.3 Splash与登录窗口

前面课程介绍过，常用的窗体基类是QWidget、QDialog和QMainWindow，在创建GUI应用程序时选择窗体基类就从这3个类中选择。



一般大型应用程序在启动时会显示一个启动画面，即Splash窗口。Splash窗口是一个无边框的对话框，一般显示一个图片，展示软件的信息。Splash窗口显示时，程序在后台做一些比较耗时的启动准备工作，Splash窗口显示一段时间后自动关闭，软后软件的主窗口显示出来。Qt有一个QSplashScreen类可以实现Splash窗口的工作，它提供而来载入图片，自动设置窗口无边框效果等功能。有的应用程序还有软件登录界面，要求用户输入用户名和密码才可以进入软件。




```
#include "logindialog.h"
#include "ui_logindialog.h"

LoginDialog::LoginDialog(QWidget *parent) :
```

```
    QDialog(parent),
    ui(new Ui::LoginDialog)
{
    ui->setupUi(this);

    setAttribute(Qt::WA_DeleteOnClose);
    setWindowFlags(Qt::SplashScreen);

}

LoginDialog::~LoginDialog()
{
    delete ui;
}

void LoginDialog::on_buttonBox_accepted()
{
    if(ui->le_name->text().trimmed() == "张飞" &&
            ui->le_passwd->text().trimmed() == "123456")
        return accept();
    else {
        return reject();
    }
}
```

```
#include "mainwindow.h"
#include <QApplication>
#include "logindialog.h"

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);

    LoginDialog *dlgLogin = new LoginDialog;
    if(dlgLogin->exec()==QDialog::Accepted){
        MainWindow w;
        w.show();
        return a.exec();
    }
    else{
        return 0;
    }
}
```

4. 文件操作

文件的读写是很多应用程序具有的功能，甚至某些应用程序就是围绕着某一种格式文件的处理而开发的，所以文件读写是应用程序开发的一个基本功能。

注意：

从物理上讲，文本文件与二进制文件本质上是没有区别的，都是以二进制形式保存在磁盘中。

Windows系统中，文本文件读写时存在\r \n 和 \n的替换问题（写入时\n 会被替换为\r \n, 读出时\r \n会被替换为\n）。

二进制文件读写时没有任何替换。

Linux系统中，文本文件与二进制文件没有区别。

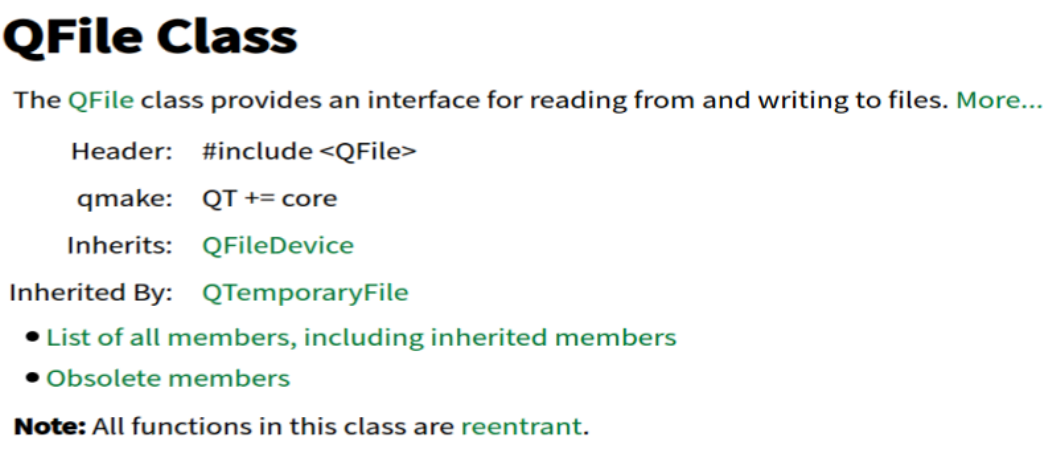
4.1 文本文件读写

文本文件是以纯文本格式存储的文件，例如大家编程实现的.c .h .cpp文件。HTML和XML文件也是纯文本文件，只是读取之后需要对内容进行解析之后再显示。

Qt提供了两种读写纯文本文件的基本方法，一种是用QFile类的IODevice读写功能直接进行读写，另一种是利用QFile和QTextStream结合起来，用流(stream)的方法进行读写。

4.1.1QFile读写文本文件

QFile类是直接与IO设备打交道，进行文件读写操作的类，使用QFile可以直接打开或保存文本文件。



open文件

```
bool QFile::open(QIODevice::OpenMode mode)
```

```
bool QFile::open(FILE *fh, QIODevice::OpenMode mode,
QFileDevice::FileHandleFlags handleFlags = DontCloseHandle)
bool QFile::open(int fd, QIODevice::OpenMode mode,
QFileDevice::FileHandleFlags handleFlags = DontCloseHandle)
    QIODevice::OpenMode，常用取值
            QIODevice::ReadOnly
            QIODevice::WriteOnly
            QIODevice::ReadWrite
            QIODevice::Append
            QIODevice::Truncate
            QIODevice::Text
    QFileDevice::FileHandleFlags
            QFileDevice::AutoCloseHandle  // QFile.close（）后自动
关闭文件句柄
            QFileDevice::DontCloseHandle  // QFile.close（）后不关
闭文件句柄
//example
QFile file1("a.txt");
file1.open( QIODevice::ReadOnly|QIODevice::Text);

QFile file2;
file2.open(stderr, QIODevice::WriteOnly|QIODevice::Text);
//stderr 定义在stdio.h中

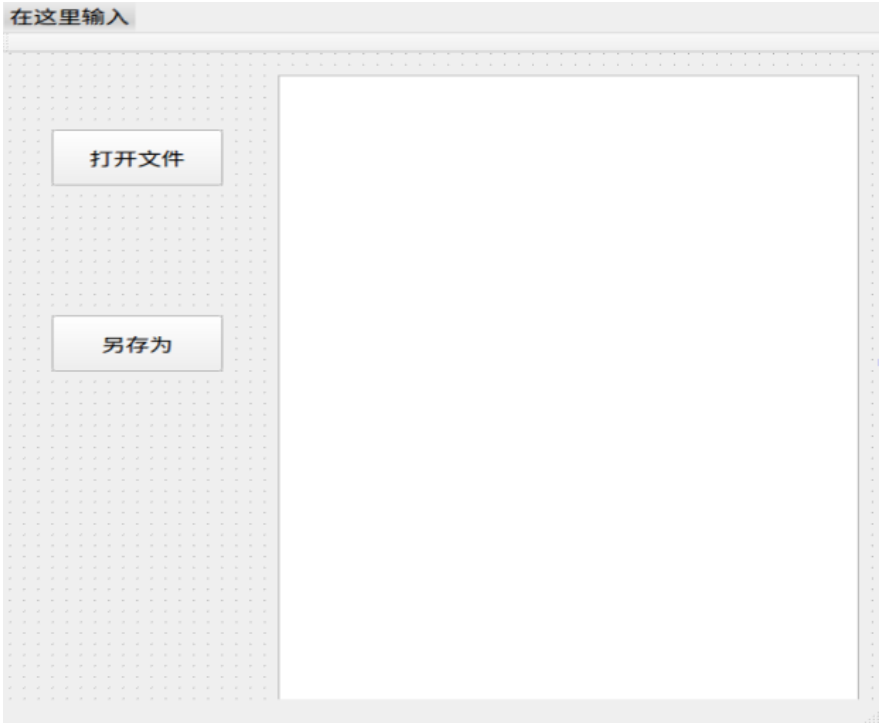
QFile file3;
file3.open(STDERR_FILENO,
QIODevice::WriteOnly|QIODevice::Text);//STDERR_FILENO 定义在
unistd.h中
```

读操作

```
qint64 QIODevice::read(char *data, qint64 maxSize);
QByteArray QIODevice::read(qint64 maxSize);
QByteArray QIODevice::readAll()
qint64 QIODevice::readLine(char *data, qint64 maxSize)
QByteArray QIODevice::readLine(qint64 maxSize = 0)
```

写操作:

```
qint64 QIODevice::write(const char *data, qint64 maxSize);
qint64 QIODevice::write(const char *data);
qint64 QIODevice::write(const QByteArray &byteArray)
```




```
#include "mainwindow.h"
#include "ui_mainwindow.h"
#include <QDir>
#include <QFileDialog>
#include <QFile>

MainWindow::MainWindow(QWidget *parent) :
    QMainWindow(parent),
    ui(new Ui::MainWindow)
{
    ui->setupUi(this);
}

MainWindow::~MainWindow()
{
    delete ui;
}

void MainWindow::on_pushButton_clicked()
{
    QString aFileName = QFileDialog::getOpenFileName(this,
                                                     "打开一个文
件",

QDir::currentPath(),
```

```
                                                    "程序文件(*.h
*.cpp *.c);;文本文件(*.txt);;所有文件(*.*)");
    if(aFileName.isEmpty()){
        return;
    }

    QFile aFile(aFileName);
    if(!aFile.exists()){
        return ;
    }

    if(!aFile.open(QIODevice::ReadOnly | QIODevice::Text)){
        return ;
    }
    //ui->plainTextEdit->setPlainText(aFile.readAll());

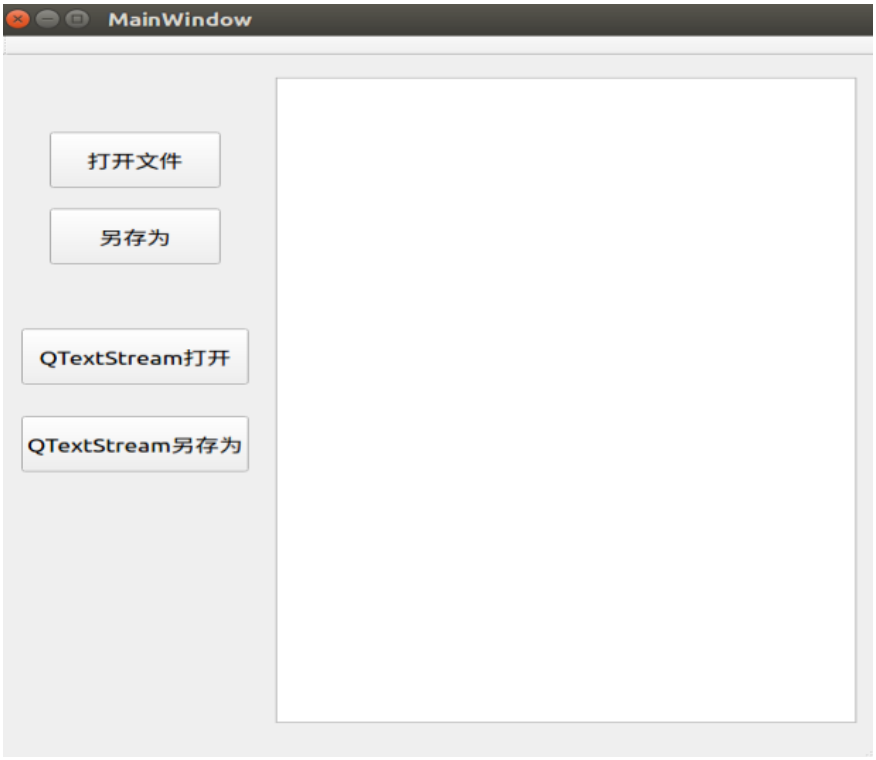
    while(!aFile.atEnd()){
        QByteArray line = aFile.readLine();
        ui->plainTextEdit->appendPlainText(line);
    }

    aFile.close();
}
void MainWindow::on_btn_save_clicked()
{
    QString aFileName = QFileDialog::getSaveFileName(this,
                                                     "另存为",

QDir::currentPath(),
                                                    "程序文件(*.h
*.cpp *.c);;文本文件(*.txt);;所有文件(*.*)");
    if(aFileName.isEmpty()){
        return;
    }
    QFile aFile(aFileName);
    if(!aFile.open(QIODevice::WriteOnly|QIODevice::Text)){
        return ;
    }
    QString str = ui->plainTextEdit->toPlainText();
    QByteArray strBytes = str.toUtf8();//转换为字节数组
    aFile.write(strBytes, strBytes.length());
    aFile.close();
}
```

4.1.2 QFile和QTextStream结合读写文本文件

QTextStream与IO读写设备相结合，为数据读写提供了一些方便的方法类，QTextStream可以与QFile、QTemporaryFile、QBuffer、QTcpSocket和QUdpSocket等IO设备类结合使用。




```
#include "mainwindow.h"
#include "ui_mainwindow.h"
#include <QDir>
#include <QFileDialog>
#include <QFile>
#include <unistd.h>
#include <fcntl.h>
#include <QDebug>

MainWindow::MainWindow(QWidget *parent) :
    QMainWindow(parent),
    ui(new Ui::MainWindow)
{
    ui->setupUi(this);
}

MainWindow::~MainWindow()
{
    delete ui;
}
```

```
void MainWindow::on_pushButton_clicked()
{
    QString aFileName = QFileDialog::getOpenFileName(this,
                                                     "打开一个文
件",

QDir::currentPath(),
                                                     "程序文件(*.h
*.cpp *.c);;文本文件(*.txt);;所有文件(*.*)");
    if(aFileName.isEmpty()){
        return;
    }

    QFile aFile(aFileName);
    if(!aFile.exists()){
        return ;
    }

    if(!aFile.open(QIODevice::ReadOnly | QIODevice::Text)){
        return ;
    }


    //ui->plainTextEdit->setPlainText(aFile.readAll());

    while(!aFile.atEnd()){
        QByteArray line = aFile.readLine();
        ui->plainTextEdit->appendPlainText(line);
    }

    aFile.close();

}

void MainWindow::on_btn_save_clicked()
{
    QString aFileName = QFileDialog::getSaveFileName(this,
                                                     "另存为",

QDir::currentPath(),
                                                     "程序文件(*.h
*.cpp *.c);;文本文件(*.txt);;所有文件(*.*)");
    if(aFileName.isEmpty()){
```

```
        return;
    }
    QFile aFile(aFileName);
    if(!aFile.open(QIODevice::WriteOnly|QIODevice::Text)){
        return ;
    }
    QString str = ui->plainTextEdit->toPlainText();
    QByteArray strBytes = str.toUtf8();
    aFile.write(strBytes, strBytes.length());
    aFile.close();

}

void MainWindow::on_btn_open_stream_clicked()
{
    QString aFileName = QFileDialog::getOpenFileName(this,
                                                     "打开一个文
件",

QDir::currentPath(),
                                                     "程序文件(*.h
*.cpp *.c);;文本文件(*.txt);;所有文件(*.*)");
    if(aFileName.isEmpty()){
        return;
    }

    QFile aFile(aFileName);
    if(!aFile.exists()){
        return ;
    }

    if(!aFile.open(QIODevice::ReadOnly | QIODevice::Text)){
        return ;
    }

    QTextStream aStream(&aFile);
    aStream.setAutoDetectUnicode(true);
    ui->plainTextEdit->setPlainText(aStream.readAll());
    aFile.close();

}

void MainWindow::on_btn_save_stream_clicked()
```

```
{
    QString aFileName = QFileDialog::getSaveFileName(this,
                                                     "另存为",

QDir::currentPath(),
                                                     "程序文件(*.h
*.cpp *.c);;文本文件(*.txt);;所有文件(*.*)");
    if(aFileName.isEmpty()){
        return;
    }
    QFile aFile(aFileName);
    if(!aFile.open(QIODevice::WriteOnly|QIODevice::Text)){
        return ;
    }
    QTextStream aStream(&aFile); //用文本流写入文件
    QString str = ui->plainTextEdit->toPlainText();
    aStream.setAutoDetectUnicode(true);
    aStream << str;
    aFile.close();
}
```

注意:QString与QByteArray

- 区别

QString是专门用来处理字符串的，除了能处理ASCII编码字符，还包括各国语言的编码，默认情况下QString会把所有数据当做utf-8编码来处理。QByteArray只是单纯用来处理数据的，除了能处理ASCII编码字符，其它复杂的编码不能处理，直接以字节流的方式来对待。

```
QString str("小马哥");
QByteArray byte("小马哥");
//str: "小马哥" byte: "\xE5\xB0\x8F\xE9\xA9\xAC\xE5\x93\xA5"
qDebug() << "str:" << str << "byte:" << byte << endl;
```

- QString转QByteArray

```
QString str("123abc小马哥");

QByteArray byte1 = str.toLatin1(); //按照ASCII编码转换，无法转换中文

QByteArray byte2 = str.toUtf8();  //按照Utf-8编码转换，可以转换中文
//byte1: "123abc???" byte2:
"123abc\xE5\xB0\x8F\xE9\xA9\xAC\xE5\x93\xA5"
qDebug() <<  "byte1:" << byte1 << "byte2:" << byte2;
```

- QByteArray转QString

```
QByteArray byte("123abc小马哥");

QString str(byte);
//:byte: "123abc\xE5\xB0\x8F\xE9\xA9\xAC\xE5\x93\xA5" str:
"123abc小马哥"
qDebug() << "byte:" << byte << "str:" << str;
```

- QString 转char*

  QString类型无法直接转换成char, *需要先转换成QByteArray, 然后再转char*

```
QString str("123abc小马哥");

QByteArray byte = str.toUtf8();

char* str1 = byte.data();

//或者
char *str1 = str.toStdString().c_str();
```

4.2 二进制文件读写

除了文本文件之外，其它需要按照一定的格式读写的文件都称为二进制文件。每种格式的二进制文件都有自己的格式定义，写入数据时按照一定的顺序写入，读出时也按照相应的顺序读出。Qt使用QFile和QDataStream进行二进制数据文件的读写。QFile负责文件的IO设备接口，即与文件的物理交互，QDataStream以数据流的方式读取文件内容或写入文件内容。




```
#include "mainwindow.h"
#include "ui_mainwindow.h"
#include <QDir>
#include <QFileDialog>
#include <QDebug>

MainWindow::MainWindow(QWidget *parent) :
    QMainWindow(parent),
    ui(new Ui::MainWindow)
{
    ui->setupUi(this);
}

MainWindow::~MainWindow()
{
    delete ui;
}



void MainWindow::on_btn_save_clicked()
{
    QString curPath = QDir::currentPath();
    QString dlgTitle = "保存文件";
    QString filter = "h文件(*.h);;C++文件(*.cpp);;所有文件(*.*)";
    QString filename = QFileDialog::getSaveFileName(this,
dlgTitle,
                                                    curPath,
filter);
```

```
    if(filename.isEmpty()){

        return;
      }

      QFile aFile(filename);
      if(!
     aFile.open(QIODevice::WriteOnly)//
     open默认打开二进制，所以文本打开需要加Text)
     {

        return ;
      }
      QDataStream out(&aFile);
      int num = 123;
      QString str1 = "abc";
      QString str2 = "def";
      out << str1;
      out << num;
      out << str2;

}     aFile.close();

void MainWindow::on_btn_read_clicked()
{
   QString aFileName = QFileDialog::getOpenFileName(this,
                                                     "打开一个文件",
                                                     "/tmp",
                                                     "程序文件(*.h
*.cpp *.c);;文本文件(*.txt);;所有文件(*.*)");
   if(aFileName.isEmpty()){
       return;
   }

   QFile aFile(aFileName);
   if(!aFile.exists()){
       return ;
   }

   if(!aFile.open(QIODevice::ReadOnly )){
       return ;
   }
```

```
    QDataStream in(&aFile);

    int num ;
    QString str1;
    QString str2;
    in >> str1;
    in >> num;
    in >> str2;

    QString text = QString("%1 %2
%3").arg(str1).arg(QString::asprintf("%d",num)).arg(str2);
    ui->plainTextEdit->setPlainText(text);
}
```

<<,在写入数据时调用的是 int QDataStream::writeRawData(const char *s, int len)

在写入字符串时调用的是QDataStream &QDataStream::writeBytes(const char *s, uint len)

该函数会先将len作为一个quint32类型写入数据流，然后再写入len个从指针s获取的数据。

输出>> 运算符使用的readRawData（）和readBytes（）和输入<<运算符的执行逻辑类似。

4.3 文件目录操作

Qt为文件和目录操作提供了一些类，利用这些类就可以方便地实现一些操作。Qt提供的与文件和目录操作相关的类包括以下几个：

- QCoreApplication, 用于提取应用程序路径、程序名等文件信息。
- QFile， 除了打开文件操作外，QFile还有复制文件、删除文件等功能
- QFileInfo,用于提取文件名、路径、后缀、大小等元数据信息。
- QDir， 用于提取目录或文件信息，获取一个目录下的文件或目录列表，创建或删除目录和文件，文件重命名等操作。
- QTemporaryDir和QTemporaryFile: 用于创建临时目录和临时文件。
- QFileSystemWatcher： 文件和目录监听类，监听目录下文件的添加、删除等变化，监听文件的修改变化。

这些类基本涵盖了文件操作需要的主要功能，有些功能还在某些类里重复出现，例如QFile和QDir都具有删除文件、判断文件是否存在的功能。

案例：读取指定目录的所有文件的信息




```
#include "widget.h"
#include "ui_widget.h"
#include <QFileDialog>
#include <QDir>
#include <QFileInfoList>

Widget::Widget(QWidget *parent) :
    QWidget(parent),
    ui(new Ui::Widget)
{
    ui->setupUi(this);
}

Widget::~Widget()
{
    delete ui;
}

void Widget::on_pushButton_clicked()
{
    QString dirName = QFileDialog::getExistingDirectory();

    if(dirName.isEmpty())
```

```
        return ;
    ui->lineEdit->setText(dirName);
    QDir dir(dirName);

    dir.setFilter(QDir::Files | QDir::Hidden | QDir::NoSymLinks);


    QFileInfoList list = dir.entryInfoList();

    for(int i=0; i<list.size(); i++){
        QFileInfo info = list.at(i);
        ui->plainTextEdit->appendPlainText(info.fileName() +
":\t" +

QString::asprintf("%lld",info.size()));
        //ui->plainTextEdit-
>appendPlainText(QString("%1:\t%2").arg(info.fileName()).arg(info
.size()));
    }
}
```

5. 数据库编程

5.1 数据库概述

数据库是指长期存储在计算机内的，有组织，可共享的数据的集合。数据库中的数据按一定的数学模型组织、描述和存储，具有较小的冗余，较高的数据独立性和易扩展性，并可为各种用户共享。

相关术语：

- DB , 数据库（database），存储在硬盘上数据的集合
- DBMS, 数据库管理系统， 操作管理数据库的大型软件，用户通过DBMS访问数据库（DB）中的数据。
- DBA， 数据库管理员
- RDB， 关系型数据库， 是指采用了关系模型来组织数据的数据库，其以行和列的形式存储数据， 关系型数据库这一系列的行和列被称为表，一组表组成了数据库。 主流的关系型数据库管理系统有Oracle、DB2、MySQL、Microsoft SQL Server、Microsoft Access、SQLite等多个品种。

WIN2016.bdqn - dbo.employee | SQLQuery1.sql - W...ministrator (54))*

编号	姓名	身份证号	职务	出生日期	基本工资	备注
3	黄蓉	10020030040...	DBA	1995-03-03 ...	10000.0000	NULL
2	杨过	11022033044...	运维工程师	1992-02-02 ...	9000.0000	NULL
4	小龙女	11111111122...	运维经理	1990-05-05 ...	15000.0000	技术全面
1	郭靖	11122233344...	运维工程师	1995-01-01 ...	8000.0000	NULL
6	洪七公	66055044033...	运维总监	1980-07-07 ...	36000.0000	NULL
5	段誉	66655544433...	DBA	1989-06-06 ...	9000.0000	NULL
7	黄药师	88077066055...	CTO	1975-11-11 ...	55000.0000	项目经验丰富

- 数据库操作语言SQL(Structed Query Language)，是一种针对数据库的结构化查询语言，用于实现数据库查询和程序设计。
- SQLite数据库

  是一个轻量级的开源数据库，源码完全公开不受版权限制，实现了自给自足、无服务器、零配置的SQL数据库引擎，也是最广泛使用的关系数据库管理系统。支持像Linux 、windows、Mac OS等众多平台。

  SQLite引擎是以静态库或共享库的方式连接到程序中的，而不同于客户-服务器数据库连接方式，所以主要的使用方法是在编程语言内的直接API调用，当前主流的编程语言，如C/C++、java、 python等都提供了sqlite操作的API接口。SQLite在资源消耗、实时性和可操作性有着出色的表现，整个数据库都在宿主机主机上存储的一个单一的文件中，通常sqlite的工作时在开始一个事务的时候锁定整个数据文件，从而允许多个进程或者线程安全访问。

  在ubuntu系统上安装Sqlite数据库管理系统。

```
sudo apt install sqlite3
```

  可以通过执行 sqlite3 ， 查看安装是否成功。正常启动如下图所示.

```
linux@ubuntu:~$ sqlite3
SQLite version 3.11.0 2016-02-15 17:29:24
Enter ".help" for usage hints.
Connected to a transient in-memory database.
Use ".open FILENAME" to reopen on a persistent database.
sqlite>
```

如上图所示，在sqlite3命令界面可以输入两种类型的指令：

- 自身配置和格式控制相关指令， 这类指令以"."开头
- SQL语句，实现对数据库的增删改查等操作，这类指令以";"结束

5.2 数据库操作指令

常用指令如下：

- .help, 帮助信息
- .database 当前关联的数据库名称
- .open 打开数据库
- .tables 显示当前数据库中的表名
- .schema 查看数据表结构
- .mode 设置显示模式
- .nullval 设置空白字段显示的字符串
- .headers on 显示数据表的表头

注意，以上配置当退出命令行时会失效，可以通过配置文件配置使其始终生效

```
vim ~/.sqliterc
    .mode tab
    .nullvalue NULL
    .head on

sqlite3 testDB.db
```

5.3 sql语句

```
/*创建数据库*/
.open student.db
/*创建数据表
  CREATE TABLE 表名(列名1 类型  [约束],
                    列名2 类型 [约束],
                    列名3 类型 [约束],
                    ...) ;
        常用类型：
                INTEGER（整型） TEXT(字符串) REAL(浮点数)
                BLOB(二进制，图像视频等) NUMERIC（精确数值类型，可以存储任意精度的数值）
                storage class，  是数据存储到磁盘时的实际类型。
                type affinity，创建表时给定的类型为建议类型（亲和类型）
                SQLite 没有提供专门用于存储日期和时间的数据类型。通常我们可以使用 TEXT、REAL 或者 INTEGER 存储日期和时间
                TEXT 存储日期和时间时使用 ISO8601 格式的字符串（“YYYY-MM-DD HH:MM:SS.SSS”）

        约束：
            PRIMARY KEY（主键），表示该列数据唯一，可以加快数据访问
```

```
            NOT NULL(非空), 该类数据不能为空
*/

CREATE TABLE scores(id INTEGER PRIMARY KEY NOT NULL,
                    name TEXT NOT NULL,
                    age INTEGER NOT NULL,
                    score REAL);

.tables
.schema scores
```

```
/*删除数据表 （慎用）
 DROP TABLE 表名;
*/
CREATE TABLE scores_1(id INTEGER PRIMARY KEY NOT NULL,
                    name text NOT NULL,
                    age INTEGER NOT NULL,
                    score REAL);
.tables
DROP TABLE scores_1;
.tables
```

```
/*向数据库表中插入数据
  方式一，表中某几类插入数据
    INSERT INTO 表名(列名1， 列名2，...) VALUES(数值一，数值二，...);
  方式二， 表中所有列按列表顺序插入数据
    INSERT INTO 表名 VALUES(数值一，数值二，...);
*/
INSERT INTO scores(id, name, age) VALUES(1, "张飞", 21);
INSERT INTO scores VALUES(2, "赵云", 18, 98);
INSERT INTO scores(name, id, age) VALUES("关羽", 3, 25);
SELECT * FROM scores;
SELECT name,age FROM scores;
```

```
/*从数据库表中删除数据
  DELETE FROM 表名 WHERE 条件表达式
        没有条件表达式删除表中所有数据
        多个条件可以用 and 或者 or 连接
*/
DELETE FROM scores where age>20;
SELECT * FROM scores;
DELETE FROM scores where name="赵云" and age=19;
SELECT * FROM scores;
DELETE FROM scores where name="赵云" or  age=19;
SELECT * FROM scores;
```

```
/*修改数据表中的数据
  UPDATE 表名 SET 列名1=新值, 列名2=新值,  ... WHERE 条件表达式
        没有条件表达式指定的所有列数据都被修改
        多个条件可以用 and 或者 or 连接
*/
INSERT INTO scores(id, name, age) VALUES(1, "张飞", 21);
INSERT INTO scores VALUES(2, "赵云", 18, 98);
INSERT INTO scores(name, id, age) VALUES("关羽", 3, 25);
select * from scores;
UPDATE scores SET age=age+100;
select * from scores;
UPDATE scores SET score=100 WHERE score is null;
select * from scores;
```

```
/*查询操作
  SELECT 列名1，列名2, ... FROM 表名;
  SELECT 列名1，列名2, ... FROM 表名 WHERE 条件表达式;
  SELECT 列名1，列名2, ... FROM 表名 ORDER BY 列名 排列方式
  SELECT 列名1，列名2, ... FROM 表名 WHERE 条件表达式 ORDER BY 列名 排
列方式;
*/

.open testDB.db
select * from employee;
SELECT * FROM employee where title="运营总监" ORDER BY id  ASC;
SELECT * FROM employee where title="运营总监" ORDER BY id  DESC;

/*模糊查询
 SELECT 列名1，列名2, ... FROM 表名 WHERE 列名 LIKE 模糊匹配条件;
    模糊匹配通配符
       %, 代表零个、一个或者多个数字或字符
```

```
        _, 代表一个单一的数字或字符
*/
select * from employee where ein like "1%" ;
select * from employee where name like "黄%" ;
select * from employee where name like "黄_" ;
select * from employee where ein  like "1_2%" ;
```

5.4 Qt中使用SQLite

我们更多的时候是通过程序操作SQLite数据库，在Qt中不论我们连接的何种类型的关系型数据库，在我们使用的时候其操作流程是一致的：

- 创建数据库实例并初始化
- 连接数据库
- 对数据库进行一系列的添、删、查、改操作（编写并执行SQL语句）
- 关闭数据库

Qt中和数据库操作最为紧密相关的类有三个：

- QSqlDatabase, 通过这个类添加/删除/复制/关闭数据库实例

```
/*连接PSQL数据*/
//创建数据库实例（加载对应的驱动）
QSqlDatabase db = QSqlDatabase::addDatabase("QPSQL");
db.setHostName("acidalia");
db.setDatabaseName("customdb");
db.setUserName("mojito");
db.setPassword("J0a1m8");
//连接数据库
bool ok = db.open();
```

```
/*连接SQLite数据库*/
//创建数据库实例（加载对应的驱动）， 加载的共享库位于/opt/Qt5.12.0/5.12.0/gcc_64/plugins/sqldrivers/
QSqlDatabase db = QSqlDatabase::addDatabase("QSQLITE");
//设置要使用的数据库名称
db.setDatabaseName("customdb");
//连接数据库
bool ok = db.open();
```

  注意： 使用Qt数据库模块需要在工程文件中添加 "QT += sql"。

- QSqlQuery, 数据库操作类

```
/*查询*/
QSqlQuery query("SELECT * FROM artist");
int fieldNo = query.record().indexOf("country");
while (query.next()) {
    QString country = query.value(fieldNo).toString();
    doSomething(country);
}
/*插入*/
QSqlQuery query;
query.prepare("INSERT INTO person (id, forename, surname) "
              "VALUES (:id, :forename, :surname)");
query.bindValue(":id", 1001);
query.bindValue(":forename", "Bart");
query.bindValue(":surname", "Simpson");
query.exec();

/*也可以自行拼接字符串*/
QSqlQuery query;
query.exec("INSERT INTO employee (id, name, salary) "
           "VALUES (1001, 'Thad Beaumont', 65000)");
```

- QSqlQueryModel，执行SQL语句和遍历结果集的高级接口。它构建在底层QSqlQuery之上，可以用来为视图类(如QTableView)提供数据。

The QSqlQueryModel class provides a read-only data model for SQL result sets.

```
QSqlQueryModel *model = new QSqlQueryModel;
model->setQuery("SELECT name, salary FROM employee");


QTableView *view = new QTableView;
view->setModel(model);
view->show();
```

QSqlQueryModel如果想单独获取查询记录可以使用它的record函数，它有两个重载版本。其中无参版本返回字段信息和一条空记录，带参版本返回字段信息和对应的一条记录。

QSqlRecord QSqlQueryModel::**record**(**int** *row*) const

Returns the record containing information about the fields of the current query. If *row* is the index of a valid row, the record will be populated with values from that row.

If the model is not initialized, an empty record will be returned.

See also QSqlRecord::isEmpty().

QSqlRecord QSqlQueryModel::**record**() const

This is an overloaded function.

Returns an empty record containing information about the fields of the current query.

If the model is not initialized, an empty record will be returned.

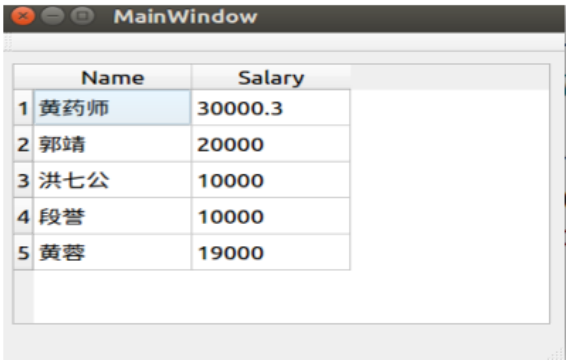
```
rec = model.record(2);
for(int i=0; i<rec.count(); i++){
    //qDebug()<<rec.fieldName(i);
    qDebug()<<rec.field(i).type()<<": "<< rec.field(i).name();
    qDebug()<<rec.value(i);
}
```

Qt中和数据库操作需要关注的三个类：

- QSqlRecord，数据库记录（通常是数据库中表或视图中的一行）的功能和特征。
- QSqlField，数据库表或视图中单个列的特征，例如数据类型和列名（单个字段信息）
- QSqlError，数据操作失败可以通过这个类获取相关的错误信息

5.5 编程实例

5.5.1 数据库相关类的简单使用




```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>
#include <QSqlDatabase>
#include <QSqlQueryModel>

QT_BEGIN_NAMESPACE
namespace Ui { class MainWindow; }
QT_END_NAMESPACE

class MainWindow : public QMainWindow
{
    Q_OBJECT
```

```
public:
    MainWindow(QWidget *parent = nullptr);
    ~MainWindow();

private:
    Ui::MainWindow *ui;
    QSqlDatabase db;
    QSqlQueryModel model;
};
#endif // MAINWINDOW_H
```

```
#include "mainwindow.h"
#include "ui_mainwindow.h"
#include <QTableView>
#include <QVBoxLayout>

MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
    , ui(new Ui::MainWindow)
{
    ui->setupUi(this);

    /*加载数据库驱动*/
    db = QSqlDatabase::addDatabase("QSQLITE");
    /*设置要使用的数据库*/
    db.setDatabaseName("./testDB.db");
    /*连接数据库*/
    db.open();

    /*执行sql语句*/
    model.setQuery("SELECT name, salary FROM employee;");

    /*显示查询结果*/
    QTableView *view = new QTableView(this);
    view->setModel(&model);

    /*组件view的显示*/
    QVBoxLayout * vb = new QVBoxLayout(centralWidget());
    vb->addWidget(view);
```

```
    ui->centralwidget->setLayout(vb);

}

MainWindow::~MainWindow()
{
    delete ui;
}
```

5.5.2 相关类出错信息的捕获

- 出错处理问题

```
#include "mainwindow.h"
#include "ui_mainwindow.h"
#include <QTableView>
#include <QVBoxLayout>
#include <QMessageBox>
#include <QSqlError>
#include <QDebug>

MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
    , ui(new Ui::MainWindow)
{
    ui->setupUi(this);

    /*加载数据库驱动*/
    db = QSqlDatabase::addDatabase("QSQLITE");
    if(!db.isValid()){
        QMessageBox::critical(this, "严重错误", "driver not
loaded");
        throw "加载数据库驱动失败";
    }
    /*设置要使用的数据库*/
    db.setDatabaseName("./testDB1.db");
    /*连接数据库*/
    bool ok = db.open();
    if(!ok){
        QMessageBox::critical(this, "严重错误",
db.lastError().text());
        throw "连接数据库失败";
    }
```

```
    /*执行sql语句*/
    model.setQuery("SELECT name, salary FROM employee;");
    if(model.lastError().isValid()){
        qDebug()<<model.lastError();
    }

    /*显示查询结果*/
    QTableView *view = new QTableView(this);
    view->setModel(&model);

    /*组件view的显示*/
    QVBoxLayout * vb = new QVBoxLayout(centralWidget());
    vb->addWidget(view);
    ui->centralwidget->setLayout(vb);

}

MainWindow::~MainWindow()
{
    delete ui;
}
```

5.5.3 获取单条记录

- QSqlQueryModel

```
//列信息
QSqlRecord QSqlQueryModel::record() const
//第row条记录
QSqlRecord QSqlQueryModel::record(int row) const
//查询到的数量
virtual int rowCount(const QModelIndex &parent = QModelIndex())
const override
```

- QSqlQuery

```
//列信息
QSqlRecord QSqlQuery::record() const
//判断是否存在下条记录
bool QSqlQuery::next()
```

```
//返回某条记录中索引为index的子弹值
QVariant QSqlQuery::value(int index) const
//查询到的数量
if(db.driver()->hasFeature(QSqlDriver::QuerySize))
{
    qDebug()<<query.size() ;
}
else
{
    query.last();
    qDebug()<<query.at() + 1 ;
}
```

```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>
#include <QSqlDatabase>
#include <QSqlQueryModel>

QT_BEGIN_NAMESPACE
namespace Ui { class MainWindow; }
QT_END_NAMESPACE

class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    MainWindow(QWidget *parent = nullptr);
    ~MainWindow();

private:
    Ui::MainWindow *ui;
    QSqlDatabase db;
    QSqlQueryModel model;
};
#endif // MAINWINDOW_H
```

```
#include "mainwindow.h"
#include "ui_mainwindow.h"
#include <QTableView>
```

```
#include <QVBoxLayout>
#include <QMessageBox>
#include <QSqlError>
#include <QDebug>
#include <QSqlRecord>
#include <QSqlQuery>
#include <QSqlField>

MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
    , ui(new Ui::MainWindow)
{
    ui->setupUi(this);

    /*加载数据库驱动*/
    db = QSqlDatabase::addDatabase("QSQLITE");
    if(!db.isValid()){
        QMessageBox::critical(this, "严重错误", "driver not
loaded");
        throw "加载数据库驱动失败";
    }
    /*设置要使用的数据库*/
    db.setDatabaseName("./testDB.db");
    /*连接数据库*/
    bool ok = db.open();
    if(!ok){
        QMessageBox::critical(this, "严重错误",
db.lastError().text());
        throw "连接数据库失败";
    }

    /*执行sql语句*/
    model.setQuery("SELECT name, salary FROM employee;");
    if(model.lastError().isValid()){
        qDebug()<<model.lastError();
    }

    /*显示查询结果*/
    QTableView *view = new QTableView(this);
    view->setModel(&model);

    /*组件view的显示*/
    QVBoxLayout * vb = new QVBoxLayout(centralWidget());
```

```
    vb->addWidget(view);
    ui->centralwidget->setLayout(vb);

    /*通过QSqlQueryModel获取单条记录*/
    //QSqlRecord rec = model.record(2);
    //qDebug()<<rec.value(0).toString();
    //qDebug()<<rec.value(1).toDouble();

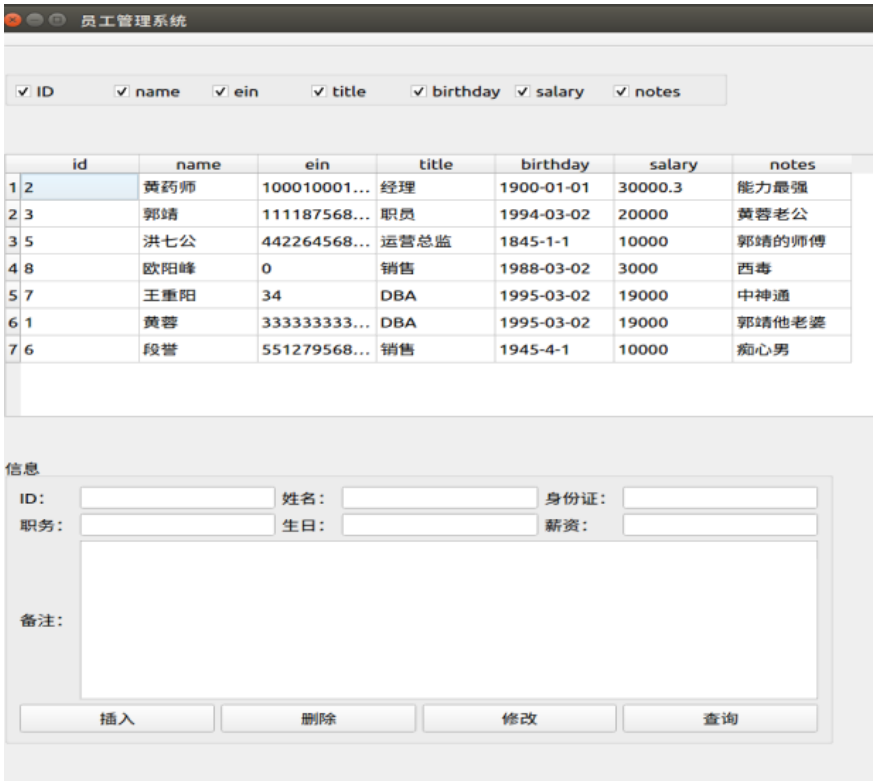
    /*通过QSqlQuery获取单条记录*/
    QSqlQuery query("select * from employee");
    if(!query.exec()){
        qDebug()<<query.lastError().text();
        throw query.lastError().text();

    }
    QSqlRecord rec = query.record();
    qDebug()<<rec.count();
    for(int i=0; i<rec.count(); i++){
        qDebug()<<rec.field(i).type() << ": " <<
rec.field(i).name();
    }

    while(query.next()){
        qDebug()<<query.value(0).toInt() <<"\t"
                <<query.value(1).toString()<<"\t"
                <<query.value(2).toInt() << "\t"
                << query.value(3).toString() << "\t"
                << query.value(4).toString() << "\t"
                <<query.value(5).toDouble() <<"\t"
                <<query.value(6).toString();
    }
}

MainWindow::~MainWindow()
{
    delete ui;
}
```

5.6 员工管理系统




```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>
#include <QSqlDatabase>
#include <QSqlQueryModel>

namespace Ui {
class MainWindow;
}

class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    explicit MainWindow(QWidget *parent = nullptr);
    ~MainWindow();

private slots:
    void on_cb_id_stateChanged(int arg1);

    void on_cb_name_stateChanged(int arg1);
```

```
    void on_cb_ein_stateChanged(int arg1);

    void on_cb_title_stateChanged(int arg1);

    void on_cb_birthday_stateChanged(int arg1);

    void on_cb_salary_stateChanged(int arg1);

    void on_cb_notes_stateChanged(int arg1);

    void on_btn_insert_clicked();

    void on_tableView_pressed(const QModelIndex &index);



    void on_btn_delete_clicked();

    void on_btn_modify_clicked();

    void on_btn_select_clicked();

private:
    Ui::MainWindow *ui;
    QSqlDatabase db;
    QSqlQueryModel model;
    QString items;

    void select_items(void);
};

#endif // MAINWINDOW_H
```

```
#include "mainwindow.h"
#include "ui_mainwindow.h"
#include <QDebug>
#include <QSqlError>
#include <QSqlQuery>
#include <QModelIndex>
#include <QSqlRecord>
#include <QMessageBox>
```

```
MainWindow::MainWindow(QWidget *parent) :
    QMainWindow(parent),
    ui(new Ui::MainWindow)
{
    ui->setupUi(this);
    db = QSqlDatabase::addDatabase("QSQLITE");
    db.setDatabaseName("./testDB.db");
    db.open();

    items = QString("id, name,ein,title,birthday,salary,notes");
    model.setQuery(QString("SELECT %1 FROM
employee").arg(items));
    //model.setHeaderData(0, Qt::Horizontal, tr("Name"));
    //model.setHeaderData(1, Qt::Horizontal, tr("Salary"));
    ui->tableView->setModel(&model);
}

MainWindow::~MainWindow()
{
    delete ui;
}
void MainWindow::select_items(void){
    items.clear();

    if(ui->cb_id->isChecked())
        items += ui->cb_id->text() + ",";
    if(ui->cb_name->isChecked())
        items += ui->cb_name->text()+ ",";
    if(ui->cb_ein->isChecked())
        items += ui->cb_ein->text()+ ",";
    if(ui->cb_title->isChecked())
        items += ui->cb_title->text()+ ",";
    if(ui->cb_birthday->isChecked())
        items += ui->cb_birthday->text()+ ",";
    if(ui->cb_salary->isChecked())
        items += ui->cb_salary->text()+ ",";
    if(ui->cb_notes->isChecked())
        items += ui->cb_notes->text()+ ",";

    if(items.endsWith(','))
        items[items.size()-1] = ' ';
    qDebug()<<items;
```

```
    model.setQuery(QString("SELECT %1 FROM
employee").arg(items));

    if (model.lastError().isValid())
         qDebug() << model.lastError();
    ui->tableView->setModel(&model);


}
void MainWindow::on_cb_id_stateChanged(int arg1)
{
    Q_UNUSED(arg1);
    select_items();

}

void MainWindow::on_cb_name_stateChanged(int arg1)
{
    Q_UNUSED(arg1);
    select_items();
}

void MainWindow::on_cb_ein_stateChanged(int arg1)
{
    Q_UNUSED(arg1);
    select_items();
}

void MainWindow::on_cb_title_stateChanged(int arg1)
{
    Q_UNUSED(arg1);
    select_items();
}

void MainWindow::on_cb_birthday_stateChanged(int arg1)
{
    Q_UNUSED(arg1);
    select_items();
}

void MainWindow::on_cb_salary_stateChanged(int arg1)
{
    Q_UNUSED(arg1);
```

```
    select_items();
}

void MainWindow::on_cb_notes_stateChanged(int arg1)
{
    Q_UNUSED(arg1);
    select_items();
}

void MainWindow::on_btn_insert_clicked()
{
    /*获取用户输入数据*/
    int id = ui->le_id->text().toInt();
    QString name = ui->le_name->text();
    qlonglong ein = ui->le_ein->text().toLongLong();
    QString title = ui->le_title->text();
    QString birthday = ui->le_birthday->text();
    double salary = ui->le_salary->text().toDouble();
    QString note = ui->plainTextEdit->toPlainText();


    /*插入数据库*/
    QSqlQuery query;
    query.prepare("INSERT INTO employee "
                  "VALUES (:id, :name, :ein, :title, :birthday,
:salary, :notes)");
    query.bindValue(0, id);
    query.bindValue(1, name);
    query.bindValue(2, ein);
    query.bindValue(3, title);
    query.bindValue(4, birthday);
    query.bindValue(5, salary);
    query.bindValue(6, note);

    if(!query.exec()){
      qDebug()<<query.lastError().text();
      //throw query.lastError().text();
      QMessageBox::critical(this, "插入失败",
query.lastError().text());
      return ;
    }

    /*刷新*/
```

```
    select_items();
}

void MainWindow::on_tableView_pressed(const QModelIndex &index)
{
    //qDebug()<<"pressed:"<< index.data().toInt();  //获取点击列具体内容


    int row = index.row();
    QSqlRecord rec = model.record(row);
    ui->le_id->setText(rec.value(0).toString());
    ui->le_name->setText(rec.value(1).toString());
    ui->le_ein->setText(rec.value(2).toString());
    ui->le_title->setText(rec.value(3).toString());
    ui->le_birthday->setText(rec.value(4).toString());
    ui->le_salary->setText(rec.value(5).toString());
    ui->plainTextEdit->setPlainText(rec.value(6).toString());

}


void MainWindow::on_btn_delete_clicked()
{
    bool ok;
    /*获取用户输入数据*/
    int id = ui->le_id->text().toInt(&ok);
    if(!ok){
        QMessageBox::warning(this, "警告", "ID 字段不合法");
        return;
    }
    QSqlQuery query;
    //query.prepare("delete from employee where id=:id");
    //query.bindValue(":id", id);

    QString del = QString("delete from employee where id=%1").arg(id);
    if(!query.exec(del)){
        QMessageBox::critical(this, "删除失败", query.lastError().text());
        return;
    }
    /*刷新*/
```

```
    select_items();

}

void MainWindow::on_btn_modify_clicked()
{
    bool ok;
    /*获取用户输入数据*/
    int id = ui->le_id->text().toInt(&ok);
    if(!ok){
        QMessageBox::warning(this, "警告", "ID 字段不合法");
        return;
    }


    QSqlQuery query;
    QString del = QString("delete from employee where
id=%1").arg(id);
    if(!query.exec(del)){
        QMessageBox::critical(this, "修改失败",
query.lastError().text());
        return;
    }
    /*插入数据库*/
    /*获取用户输入数据*/
    QString name = ui->le_name->text();
    qlonglong ein = ui->le_ein->text().toLongLong();
    QString title = ui->le_title->text();
    QString birthday = ui->le_birthday->text();
    double salary = ui->le_salary->text().toDouble();
    QString note = ui->plainTextEdit->toPlainText();

    query.prepare("INSERT INTO employee "
                  "VALUES (:id, :name, :ein, :title, :birthday,
:salary, :notes)");
    query.bindValue(0, id);
    query.bindValue(1, name);
    query.bindValue(2, ein);
    query.bindValue(3, title);
    query.bindValue(4, birthday);
    query.bindValue(5, salary);
    query.bindValue(6, note);
```

```
    if(!query.exec()){
      qDebug()<<query.lastError().text();
      //throw query.lastError().text();
      QMessageBox::critical(this, "修改失败",
query.lastError().text());
      return ;
    }

    /*刷新*/
    select_items();

}

void MainWindow::on_btn_select_clicked()
{
    bool ok;
    /*获取用户输入数据*/
    int id = ui->le_id->text().toInt(&ok);
    if(!ok){
        QMessageBox::warning(this, "警告", "ID 字段不合法");
        return;
    }

    model.setQuery(QString("SELECT * FROM employee where
id=%1").arg(id));
}
```

其实到这还有大量功能可以添加，例如更加严格的出错处理，条件的查询等等，大家可以自行丰富功能。

6. Qt事件处理

6.1 Qt事件概述

事件(event)是由窗口系统（操作系统）或者Qt自身产生的，用以响应所发生的各类事件。QT程序是事件驱动的, 程序的每个动作都是由内部某个事件所触发。QT事件的发生和处理成为程序运行的主线，存在于程序整个生命周期。

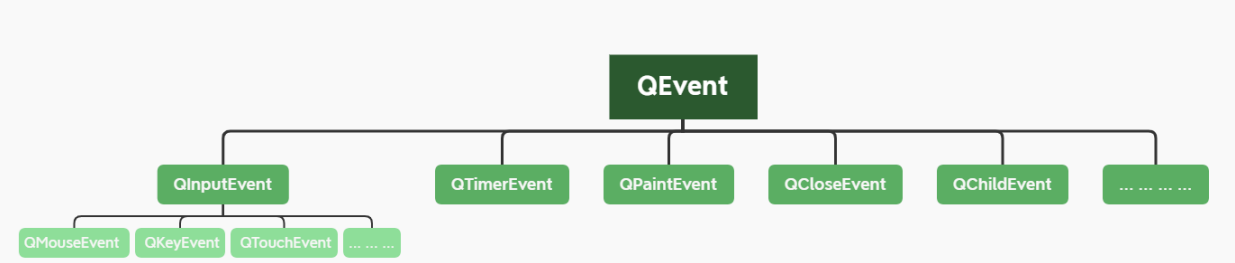
常见的事件如：

- 键盘或鼠标事件， 用户按下或者松开键盘或鼠标上的按键，就可以产生一个键盘或者鼠标事件。

- 绘制事件，某个窗口第一次显示时就会产生一个绘制事件，用来告诉窗口需要重新绘制它本身，从而使得该窗口可见。
- Qt事件，Qt自身也会产生很多事件，比如QObject::startTimer()会触发QTimerEvent事件。

6.1.1 Qt事件的表示

QT将系统产生的消息转化为QT事件，QT事件被封装为对象，所有的QT事件均继承抽象类QEvent，用于描述程序内部或外部发生的动作。



Public Functions

	QEvent(QEvent::Type *type*)
virtual	**~QEvent**()
void	**accept**() 接收该消息
void	**ignore**() 不接收，继续向父窗口传递
bool	**isAccepted**() const
void	**setAccepted**(bool *accepted*)
bool	**spontaneous**() const
QEvent::Type	**type**() const

特殊情况： 在复写`closeEvent`时，注意如果调用`event->ignore()`，意味着关闭事件没有被处理，**窗口不关闭**；不调用（默认）或者显式调用了另外一个函数`event->accept()`则代表接受事件， **窗口允许被关闭。**

enum QEvent::**Type**

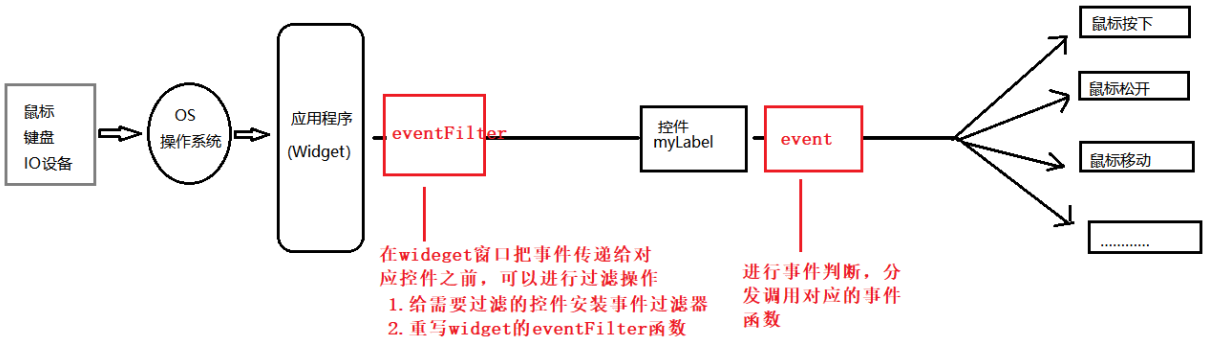
This enum type defines the valid event types in Qt. The event types and the specialized classes for each type are as follows:

Constant	Value	Description
QEvent::None	0	Not an event.
QEvent::ActionAdded	114	A new action has been added (QActionEvent).
QEvent::ActionChanged	113	An action has been changed (QActionEvent).
QEvent::ActionRemoved	115	An action has been removed (QActionEvent).
QEvent::ActivationChange	99	A widget's top-level window activation state has changed.
QEvent::ApplicationActivate	121	This enum has been deprecated. Use ApplicationStateChange instead.
QEvent::ApplicationActivated	ApplicationActivate	This enum has been deprecated. Use ApplicationStateChange instead.
QEvent::ApplicationDeactivate	122	This enum has been deprecated. Use ApplicationStateChange instead.
QEvent::ApplicationFontChange	36	The default application font has changed.
QEvent::ApplicationLayoutDirectionChange	37	The default application layout direction has changed.
QEvent::ApplicationPaletteChange	38	The default application palette has changed.
QEvent::ApplicationStateChange	214	The state of the application has changed.
QEvent::ApplicationWindowIconChange	35	The application's icon has changed.
QEvent::ChildAdded	68	An object gets a child (QChildEvent).
QEvent::ChildPolished	69	A widget child gets polished (QChildEvent).
QEvent::ChildRemoved	71	An object loses a child (QChildEvent).
QEvent::Clipboard	40	The clipboard contents have changed.
QEvent::Close	19	Widget was closed (QCloseEvent).
QEvent::CloseSoftwareInputPanel	200	A widget wants to close the software input panel (SIP).

6.1.2 Qt事件的处理顺序

谁来接受和处理事件：答案是QObject。QObject 类是整个Qt对象模型的心脏，事件处理机制是QObject三大职责（内存管理、内省(intropection)与事件处理制）之一。任何一个想要接受并处理事件的对象均须继承自QObject，可以选择重载QObject::event()函数或事件的处理权转给父窗口。

Qt框架内部为我们提供了一系列的事件处理机制，当窗口事件产生之后，事件会经过：事件派发 -> 事件过滤->事件分发->事件处理几个阶段。Qt窗口中对于产生的一系列事件都有默认的处理动作，如果我们有特殊需求就需要在合适的阶段重写事件的处理动作。



每一个Qt应用程序都对应一个唯一的 QApplication应用程序对象，然后调用这个对象的exec()函数，这样Qt框架内部的事件检测就开始了（程序将进入事件循环来监听应用程序的事件）。

```
#include "mainwindow.h"
#include <QApplication>

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    MainWindow w;
    w.show();

    return a.exec();//开启事件循环
}
```

事件在Qt中产生之后的分发过程是这样的:

- 当事件产生之后，Qt使用用应用程序对象调用notify()函数将事件发送到指定的窗口

```
[override virtual] bool QApplication::notify(QObject *receiver, QEvent *e)
```

- 注意，事件在发送之前可以通过事件过滤器进行过滤，默认不对任何产生的事件进行过滤。

```
[virtual] bool QObject::eventFilter(QObject *watched, QEvent *event)
```

- 当事件发送到指定窗口之后，窗口的事件分发器会对收到的事件进行分类

```
[virtual] bool QObject::event(QEvent *e) //QWidget中重写了该方法
```

- 事件分发器会将分类之后的事件（鼠标事件、键盘事件、绘图事件。。。）分发给对应的事件处理函数进行处理，每个事件处理函数都有默认的处理动作（我们也可以重写这些事件处理器函数），比如：鼠标事件。

  QWidget类是Qt中所有窗口类的基类，在这个类里边定义了很多事件处理器函数，它们都是受保护的虚函数。 如下图所示。

Protected Functions

virtual void	**actionEvent**(QActionEvent *event)
virtual void	**changeEvent**(QEvent *event)
virtual void	**closeEvent**(QCloseEvent *event)
virtual void	**contextMenuEvent**(QContextMenuEvent *event)
void	**create**(WId window = 0, bool initializeWindow = true, bool destroyOldWindow = true)
void	**destroy**(bool destroyWindow = true, bool destroySubWindows = true)
virtual void	**dragEnterEvent**(QDragEnterEvent *event)
virtual void	**dragLeaveEvent**(QDragLeaveEvent *event)
virtual void	**dragMoveEvent**(QDragMoveEvent *event)
virtual void	**dropEvent**(QDropEvent *event)
virtual void	**enterEvent**(QEvent *event)
virtual void	**focusInEvent**(QFocusEvent *event)
bool	**focusNextChild**()
virtual bool	**focusNextPrevChild**(bool next)
virtual void	**focusOutEvent**(QFocusEvent *event)
bool	**focusPreviousChild**()
virtual void	**hideEvent**(QHideEvent *event)
virtual void	**inputMethodEvent**(QInputMethodEvent *event)
virtual void	**keyPressEvent**(QKeyEvent *event)
virtual void	**keyReleaseEvent**(QKeyEvent *event)
virtual void	**leaveEvent**(QEvent *event)
virtual void	**mouseDoubleClickEvent**(QMouseEvent *event)
virtual void	**mouseMoveEvent**(QMouseEvent *event)
virtual void	**mousePressEvent**(QMouseEvent *event)
virtual void	**mouseReleaseEvent**(QMouseEvent *event)
virtual void	**moveEvent**(QMoveEvent *event)
virtual bool	**nativeEvent**(const QByteArray &eventType, void *message, long *result)
virtual void	**paintEvent**(QPaintEvent *event)
virtual void	**resizeEvent**(QResizeEvent *event)

当然各个子组件中可以覆盖这些虚函数

各个子组件中可以覆盖这些虚函数，例如，QPushButton中

```
//
/opt/Qt/6.2.4/Src/qtbase/src/widgets/widgets/qabstractbutton.cpp
```

```
void QAbstractButton::mouseReleaseEvent(QMouseEvent *e)
{
    Q_D(QAbstractButton);

    if (e->button() != Qt::LeftButton) {
        e->ignore();
        return;
    }

    d->pressed = false;

    if (!d->down) {
        // refresh is required by QMacStyle to resume the default
button animation
        d->refresh();
        e->ignore();
        return;
    }

    if (hitButton(e->pos())) {
        d->repeatTimer.stop();
        d->click(); //发射 click()信号
        e->accept();
    } else {
        setDown(false);
        e->ignore();
    }
}
```

6.2 覆盖过滤器eventFilter

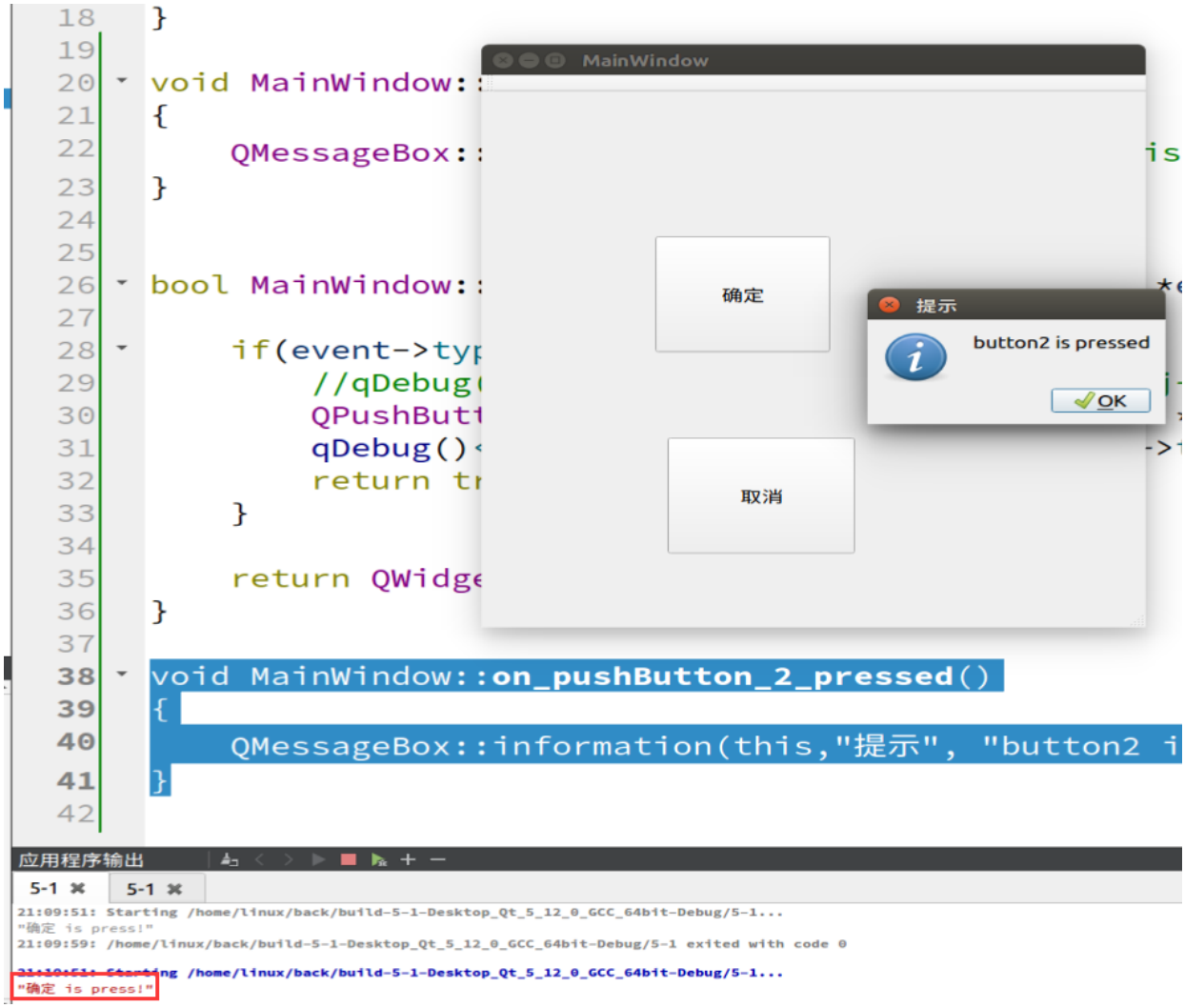
实现一个事件过滤包括两个步骤：

1)在目标对象上调用installEventFilter()，注册监视对象。

2)在监视对象的eventFilter()函数中处理目标对象的事件。

```
void QObject::installEventFilter(QObject *filterObj)
    filterObj, 实现了eventFilter(QObject *obj, QEvent *event)的对象
//在一个对象上安装事件过滤器filterObj,这样发往monitoredObj对象的所有事件，
都会调用filterObj.eventFilter()进行过滤
monitoredObj->installEventFilter(filterObj);
```

实例一：




```
#include "mainwindow.h"
#include "ui_mainwindow.h"
#include <QMessageBox>
#include <QDebug>

MainWindow::MainWindow(QWidget *parent) :
    QMainWindow(parent),
    ui(new Ui::MainWindow)
{
    ui->setupUi(this);
    //给pushbutton添加过滤器
    ui->pushButton->installEventFilter(this);
}

MainWindow::~MainWindow()
{
    delete ui;
}
```

```
void MainWindow::on_pushButton_pressed()
{
    QMessageBox::information(this,"提示", "button is pressed");
}

//过滤器中覆盖默认的eventFilter
bool MainWindow::eventFilter(QObject *obj, QEvent *event){
    if(event->type() == QEvent::MouseButtonPress){
        qDebug()<<QString("%1 is press!").arg(obj->objectName());
        QPushButton *btn = static_cast<QPushButton *>(obj);
        qDebug()<<QString("%1 is press!").arg(btn->text());
        return true;
    }
   return QWidget::eventFilter(obj,event);
}
void MainWindow::on_pushButton_2_pressed()
{
    QMessageBox::information(this,"提示", "button2 is pressed");
}
```

案例二




```
#ifndef MAINWINDOW_H
```

```h
#define MAINWINDOW_H

#include <QEvent>
#include <QMainWindow>
#include <QStringList>

QT_BEGIN_NAMESPACE
namespace Ui { class MainWindow; }
QT_END_NAMESPACE

class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    MainWindow(QWidget *parent = nullptr);
    ~MainWindow();

    bool eventFilter(QObject *obj, QEvent *event);
private slots:
    void on_btn_open_clicked();

    void on_btn_prev_clicked();

    void on_btn_next_clicked();

private:
    Ui::MainWindow *ui;
    QStringList files;
    int index;
};
#endif // MAINWINDOW_H
```

```cpp
#include "mainwindow.h"
#include "ui_mainwindow.h"
#include <QFileDialog>
#include <QKeyEvent>
#include <QPixmap>

MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
    , ui(new Ui::MainWindow)
```

```
{
    ui->setupUi(this);

    index = 0;

    /*注册监视对象*/
    ui->lb_pix->installEventFilter(this);

    ui->lb_pix->setFocus();
}

MainWindow::~MainWindow()
{
    delete ui;
}


void MainWindow::on_btn_open_clicked()
{
    files = QFileDialog::getOpenFileNames();
    QPixmap pix(files[index]);
    ui->lb_pix->setPixmap(pix);
    ui->lb_pix->setFocus();//事件是发送给被聚焦的组件。鼠标选择时
}   就完成了聚焦，所以鼠标点击按钮时，鼠标点击事件才能正确的发送给鼠标所
    指的那个按钮，同理，要让按键事件发送给安有过滤器的label标签，则要让聚
    焦一直聚焦label
void MainWindow::on_btn_prev_clicked()
{
    if(index == 0){
        index = files.length() - 1;
    }
    else{
        index--;
    }

    QPixmap pix(files[index]);
    ui->lb_pix->setPixmap(pix);
    ui->lb_pix->setFocus();
}


void MainWindow::on_btn_next_clicked()
{
```

```
    index++;
    if(index == files.length()){
        index = 0;
    }
    QPixmap pix(files[index]);
    ui->lb_pix->setPixmap(pix);
    ui->lb_pix->setFocus();
}

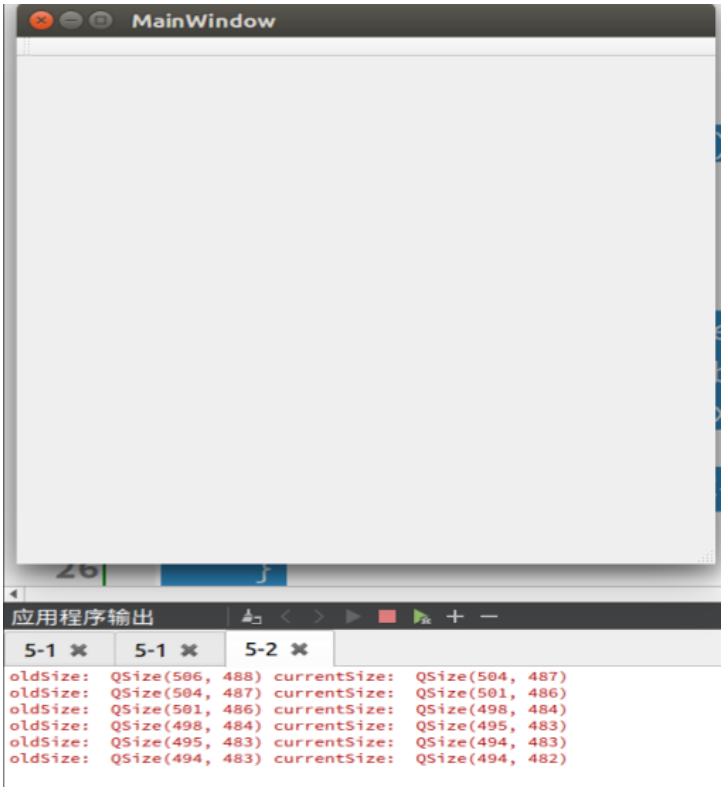
bool MainWindow::eventFilter(QObject *obj, QEvent *event){

    if(event->type()== QEvent::KeyPress){
        QKeyEvent *keyEvent = static_cast<QKeyEvent *>(event);

        if(keyEvent->key() == Qt::Key_Left){
            on_btn_prev_clicked();
        }
        if(keyEvent->key() == Qt::Key_Right){
            on_btn_next_clicked();
        }
    }
    return QWidget::eventFilter(obj, event);
}
```

6.3 覆盖事件处理函数

- 实例一： 覆盖closeEvent　resizeEvent



```h
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>

namespace Ui {
class MainWindow;
}

class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    explicit MainWindow(QWidget *parent = nullptr);
    ~MainWindow();

private:
    Ui::MainWindow *ui;
protected:
    void closeEvent(QCloseEvent *event);
    void resizeEvent(QResizeEvent *event);
};

#endif // MAINWINDOW_H
```

```cpp
#include "mainwindow.h"
#include "ui_mainwindow.h"
#include <QCloseEvent>
#include <QMessageBox>
#include <QResizeEvent>
#include <QDebug>

MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
    , ui(new Ui::MainWindow)
{
    ui->setupUi(this);
}

MainWindow::~MainWindow()
{
```

```
    delete ui;
}

void MainWindow::closeEvent(QCloseEvent *ev){
    QMessageBox::Button btn = QMessageBox::question(this, "关闭窗口", "您确定要关闭窗口吗?");
    if(btn == QMessageBox::Yes)
    {
        // 接收并处理这个事件
        ev->accept();
    }
    else
    {
        // 忽略这个事件
        ev->ignore();
    }
}

void MainWindow::resizeEvent(QResizeEvent *ev){
    qDebug() << "oldSize: " << ev->oldSize()
             << "currentSize: " << ev->size();
}
```

- 实例二： 自定义按钮




```
#ifndef MYBUTTON_H
#define MYBUTTON_H
```

```
#include <QWidget>

class MyButton : public QWidget
{
    Q_OBJECT
public:
    explicit MyButton(QWidget *parent = nullptr);

    void setImage(QString normal, QString hover, QString
pressed);

protected:
    void mousePressEvent(QMouseEvent* ev);
    void mouseReleaseEvent(QMouseEvent* ev);

    void enterEvent(QEnterEvent *event);
    void leaveEvent(QEvent* ev);
    void paintEvent(QPaintEvent* ev);

signals:
    void clicked();

private:
    QPixmap m_normal;
    QPixmap m_press;
    QPixmap m_hover;
    QPixmap m_current;
};

#endif // MYBUTTON_H
```

```
#include "mybutton.h"

#include <QPainter>
#include <QDebug>

MyButton::MyButton(QWidget *parent) : QWidget(parent)
{

}
```

```
void MyButton::setImage(QString normal, QString hover, QString
pressed)
{
    // 加载图片
    m_normal.load(normal);
    m_hover.load(hover);
    m_press.load(pressed);
    m_current = m_normal;
    // 设置按钮和图片大小一致
    setFixedSize(m_normal.size());
}

void MyButton::mousePressEvent(QMouseEvent *ev)
{
    Q_UNUSED(ev)
    // 鼠标被按下，发射这个自定义信号
    //emit clicked();
    m_current = m_press;
    update();
}

void MyButton::mouseReleaseEvent(QMouseEvent *ev)
{
    Q_UNUSED(ev)
    m_current = m_normal;
    update();
}

void MyButton::enterEvent(QEnterEvent *event)
{
    Q_UNUSED(event)
    qDebug()<<"enter";
    m_current = m_hover;
   update();//产生重绘事
}  件。调用painteven函数



void MyButton::leaveEvent(QEvent *ev)
{
    Q_UNUSED(ev)
    qDebug()<<"leave";
    m_current = m_normal;
```

```
    update();
}

void MyButton::paintEvent(QPaintEvent *ev)
{
    Q_UNUSED(ev)
    QPainter p(this);
    p.drawPixmap(rect(), m_current);
}
```

```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>

namespace Ui {
class MainWindow;
}

class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    explicit MainWindow(QWidget *parent = nullptr);
    ~MainWindow();

private:
    Ui::MainWindow *ui;
private slots:
    void btn_slot_func();
};

#endif // MAINWINDOW_H
```

```
#include "mainwindow.h"
#include "ui_mainwindow.h"
#include <QMessageBox>

MainWindow::MainWindow(QWidget *parent) :
    QMainWindow(parent),
    ui(new Ui::MainWindow)
```

```
{
    ui->setupUi(this);

    // 给自定义按钮设置图片
    ui->button->setImage(":/images/1.png", ":/images/2.png",
":/images/3.png");
    // 处理自定义按钮的鼠标点击事件
  connect(ui->button, SIGNAL(clicked()), this,
SLOT(btn_slot_func()));


   //由于自定义组件继承自QWidget，没有click（）信号的定义，所以不能直接
用转到槽的图形化方式，而要手动自定义信号以及手动连接信号与槽


}
    delete ui;
}
MainWindow::~MainWindow()
{
void MainWindow::btn_slot_func(){

    QMessageBox::critical(this, "Error", "嘿嘿");
}
```

注:

QPixmap,专注于显示，效率更高

QImage，更多用于图片的传输，像素级的图片修改。

```
//方式一
QPixmap pix;
pix.load(指定图片路径) ;
//方式二
QPixmap pix(指定图片路径);
//方式三
QFile imageFile(指定图片路径);
if (!imageFile.open(QIODevice::ReadOnly))//可读
{
    return;
}
QByteArray Image = imageFile.readAll();
QPixmap pix;
pix.loadFromData(Image);
```

6.4 综合编程实例:飞舞的雪花

一个雪花


```
#ifndef SNOW_H
#define SNOW_H

#include <QWidget>
#include <QPixmap>

class snow : public QWidget
{
    Q_OBJECT
public:
    explicit snow(QWidget *parent = nullptr);

signals:

protected:
    virtual void paintEvent(QPaintEvent *event);
    virtual void timerEvent(QTimerEvent *event);
private:
    QPixmap m_pix;
};

#endif // SNOW_H
```

```
#include "snow.h"
```

```
#include <QGuiApplication>
#include <QPainter>

snow::snow(QWidget *parent)
    : QWidget{parent}
{
    m_pix.load(":/images/snow-5.png");
    setFixedSize(m_pix.size());
    startTimer(QRandomGenerator::global()->bounded(300,800));
}

void snow::paintEvent(QPaintEvent *event)
{
    QPainter p(this);
    p.drawPixmap(rect(),m_pix);
}

void snow::timerEvent(QTimerEvent *event)
{
    int stepX = QRandomGenerator::global()->bounded(10, 15);
    int stepY = QRandomGenerator::global()->bounded(10,30);

    int curX = this->geometry().topLeft().x() + stepX;
    int curY = this->geometry().topLeft().y() + stepY;

    move(curX, curY);
    update();
}
```

```
#include "mainwindow.h"
#include "ui_mainwindow.h"
#include "snow.h"
#include <QRect>
#include <QScreen>
#include <QRandomGenerator>

MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
    , ui(new Ui::MainWindow)
{
    ui->setupUi(this);
```

```
    snow *win = new snow(this);
}

MainWindow::~MainWindow()
{
    delete ui;
}
```

去除主窗口边框 最大化窗口 多个雪花

```
#include "snow.h"

#include <QGuiApplication>
#include <QPainter>
#include <QRandomGenerator>
#include <QScreen>

snow::snow(QWidget *parent)
    : QWidget{parent}
{
    m_pix.load(":/images/snow-5.png");
    setFixedSize(m_pix.size());
    startTimer(QRandomGenerator::global()->bounded(300,800));
}

void snow::paintEvent(QPaintEvent *event)
{
    QPainter p(this);
    p.drawPixmap(rect(),m_pix);
}

void snow::timerEvent(QTimerEvent *event)
{
    int stepX = QRandomGenerator::global()->bounded(10, 15);
    int stepY = QRandomGenerator::global()->bounded(10,30);

    int curX = this->geometry().topLeft().x() + stepX;
    int curY = this->geometry().topLeft().y() + stepY;

    QRect screenRect = QGuiApplication::primaryScreen()-
>geometry();
    if(curX >= screenRect.right()){
```

```
        curX = screenRect.left();
    }
    if(curY >= screenRect.bottom()){
        curY = screenRect.top();
    }
    move(curX, curY);
    update();
}
```

```
#include "mainwindow.h"
#include "ui_mainwindow.h"
#include "snow.h"
#include <QRect>
#include <QScreen>
#include <QRandomGenerator>

MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
    , ui(new Ui::MainWindow)
{
    ui->setupUi(this);

    /*去除边框*/
    setWindowFlags(windowFlags()|Qt::FramelessWindowHint);
    /*全屏显示*/
    showMaximized();

    /*构造多个雪花*/
    QRect screenRect = QGuiApplication::primaryScreen()-
>geometry();
    for(int i=0; i<100; i++){
        snow *win = new snow(this);

        win->move(QRandomGenerator::global()->bounded(0,
screenRect.right()),
                  QRandomGenerator::global()->bounded(0,
screenRect.bottom()));
        win->show();
    }
}

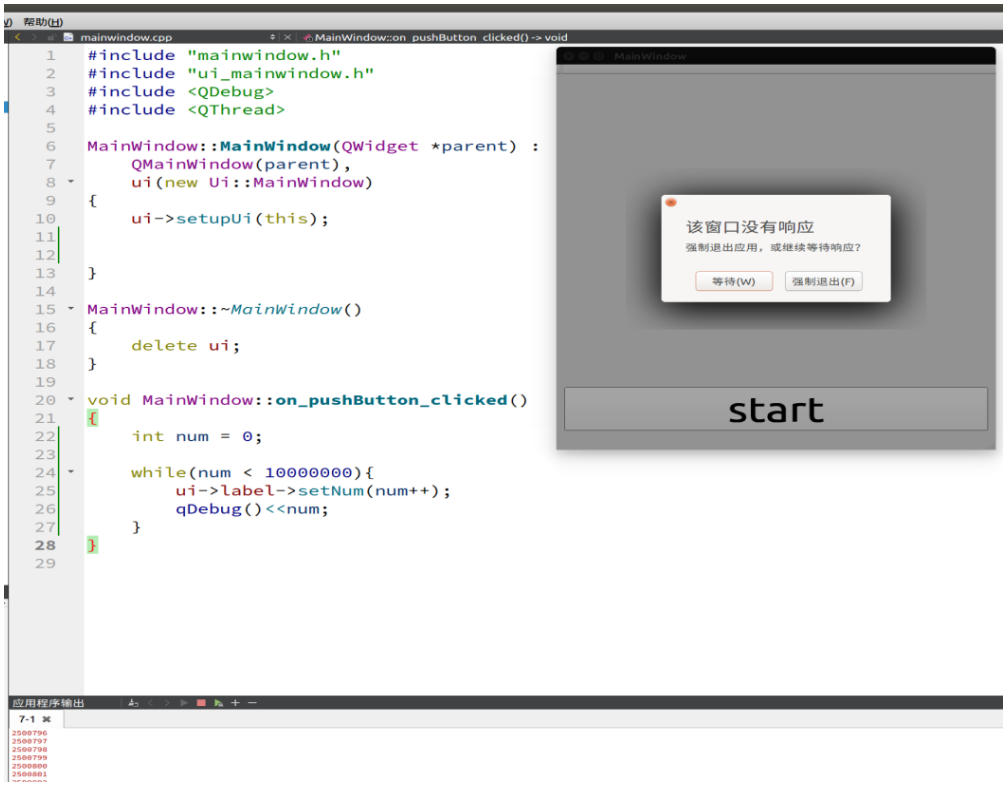
MainWindow::~MainWindow()
```

```
{
    delete ui;
}
```



7. 多线程编程

一个应用程序一般只有一个线程，一个线程内的操作时顺序执行的， 如果有某个比较消耗时间的计算或操作，比如网络通信中的文件传输，在一个线程内操作时，用户界面就可能会冻结而不能及时响应，出现假死现象。



这种情况下就需要使用多线程，其中一个线程处理窗口事件，其他线程进行逻辑运算，多个线程各司其职，不仅可以提高用户体验还可以提升程序的执行效率。

Qt为多线程操作提供了完整的支持。QThread是线程类，是实现多线程操作的核心类。Qt中提供了QMutex、QwaitCondition、QMutexLocker、QReadWriteLock、QSemaphore等多种类用于实现线程之间的同步。

Qt中主线程负责界面显示和窗口控件的数据更新，子线程(工作线程)负责逻辑业务处理和数据计算，子线程(工作线程)不能对窗口有任何操作，子线程(工作线程)可通过信号槽来将数据传递给主线程。

7.1 多线程的创建

Qt中实现了四种创建多线程的方式，对于初学者来说，主要掌握两种常见方式即可。

- 继承QThread
- 继承QObject
- 继承QRunnable，配合QThreadPool实现多线程
- 使用QtConcurrent::run()

7.1.1继承QThread

QThread Class

The QThread class provides a platform-independent way to manage threads. More...

Header: #include <QThread>
qmake: QT += core
Inherits: QObject

- List of all members, including inherited members
- Obsolete members

Public Types

enum Priority { IdlePriority, LowestPriority, LowPriority, NormalPriority, ..., InheritPriority }

Public Functions

QThread是QObject的子类，所以可以使用信号与槽机制。其主要成员函数

Public Functions

	QThread(QObject *parent = nullptr)
virtual	~QThread()
QAbstractEventDispatcher *	eventDispatcher() const
void	exit(int returnCode = 0) 线程退出函数
bool	isFinished() const
bool	isInterruptionRequested() const
bool	isRunning() const
int	loopLevel() const
QThread::Priority	priority() const
void	requestInterruption()
void	setEventDispatcher(QAbstractEventDispatcher *eventDispatcher)
void	setPriority(QThread::Priority priority)
void	setStackSize(uint stackSize)
uint	stackSize() const
bool	wait(unsigned long time = ULONG_MAX)

Signals

void **finished**() 线程中执行的任务完成了, 发出该信号
void **started**() 开始工作之前发出该信号
• 2 signals inherited from QObject

Public Slots

void **exit**(int *returnCode* = 0) 退出线程
void **quit**()
void **start**(QThread::Priority *priority* = InheritPriority) 启动线程
void **terminate**() 终止线程

Static Public Members

QThread * **create**(Function &&*f*, Args &&... *args*)
QThread * **create**(Function &&*f*)
QThread * **currentThread**()
Qt::HANDLE **currentThreadId**()
int **idealThreadCount**()
void **msleep**(unsigned long *msecs*)
void **sleep**(unsigned long *secs*)
const QMetaObject **staticMetaObject**
void **usleep**(unsigned long *usecs*)
void **yieldCurrentThread**()

Protected Functions

int **exec**()
virtual void **run**() 子线程中需要执行的逻辑，放到run函数中
• 9 protected functions inherited from QObject

编程步骤：

- 创建一个QThread类的子类

```
class MyThread:public QThread
{
    ......
}
```

- 覆盖父类的 run() 方法，在该函数内部编写子线程要处理的具体的业务流程

```
class MyThread:public QThread
{
    ......
 protected:
    void run()
    {
        ........
    }
}
```

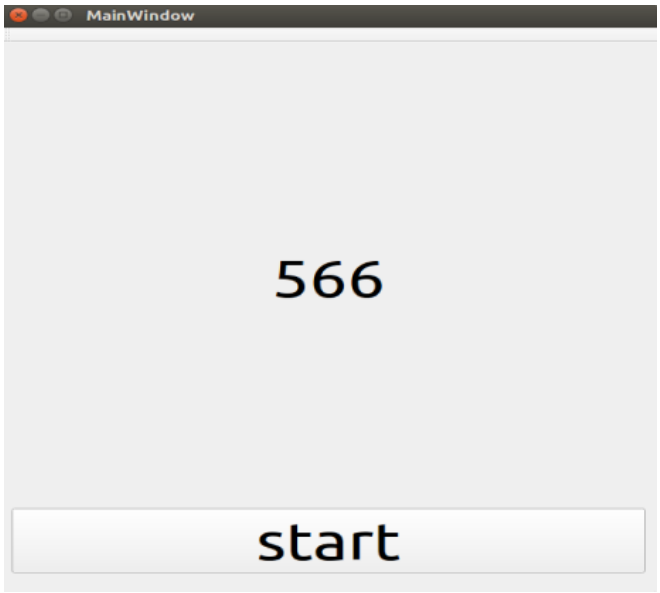
- 在主线程中创建子线程对象

```
MyThread * subThread = new MyThread;
```

- 调用 start() 方法,启动子线程

```
subThread->start();
```

编程实例:




```
#ifndef MYTHREAD_H
#define MYTHREAD_H

#include <QObject>
#include <QThread>

class MyThread : public QThread
{
    Q_OBJECT
public:
    explicit MyThread(QObject *parent = nullptr);
protected:
```

```
    void run();

signals:
    void curNum(int num);
public slots:
};

#endif // MYTHREAD_H
```

```
#include "mythread.h"
#include <QDebug>

MyThread::MyThread(QObject *parent) : QThread (parent)
{

}
void MyThread::run(){
    qDebug()<<"子线程开始运行...";

    int num = 0;

    while(num < 100000){

        emit curNum(num++);

        QThread::msleep(10);
    }

    qDebug()<<"子线程结束.";
}
```

```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>
#include "mythread.h"

namespace Ui {
class MainWindow;
}

class MainWindow : public QMainWindow
{
```

```
    Q_OBJECT

public:
    explicit MainWindow(QWidget *parent = nullptr);
    ~MainWindow();

private slots:
    void on_pushButton_clicked();
    void showNum(int num);

private:
    Ui::MainWindow *ui;
    MyThread *work;
};

#endif // MAINWINDOW_H
```

```
#include "mainwindow.h"
#include "ui_mainwindow.h"
#include <QDebug>
#include <QThread>
#include "mythread.h"

MainWindow::MainWindow(QWidget *parent) :
    QMainWindow(parent),
    ui(new Ui::MainWindow)
{
    ui->setupUi(this);

    work = new MyThread;

    connect(work, SIGNAL(curNum(int)), this, SLOT(showNum(int)));

}

MainWindow::~MainWindow()
{
    delete ui;
    delete work;
}

void MainWindow::on_pushButton_clicked()
{
```

```
    work->start();
}
void MainWindow::showNum(int num){
    ui->label->setNum(num);
}
```

这种在程序中添加子线程的方式是非常简单的，但是也有弊端，假设要在一个子线程中处理多个任务，所有的处理逻辑都需要写到run()函数中，这样该函数中的处理逻辑就会变得非常混乱，不太容易维护。

7.1.2 使用moveToThread

编程步骤：

- 创建一个新的类，让这个类从QObject派生

```
class MyWork:public QObject
{
    .......
}
```

- 在这个类中添加一个公共的成员函数，函数体就是我们要子线程中执行的业务逻辑

```
class MyWork:public QObject
{
public:
    .......
    // 函数名自己指定，叫什么都可以，参数可以根据实际需求添加
    void working();
}
```

- 在主线程中创建一个QThread对象

```
QThread* sub = new QThread;
```

- 在主线程中创建工作的类对象

```
MyWork* work = new MyWork;
```

- 调用QObject类提供的moveToThread()方法将MyWork对象移动到创建的子线程对象中

```
work->moveToThread(sub);    // 移动到子线程中工作
```

- 启动子线程，调用 start(), 这时候线程启动了, 但是移动到线程中的对象并没有工作
- 调用MyWork类对象的工作函数，让这个函数开始执行，这时候是在移动到的那个子线程中运行的

编程实例：




```
#ifndef MYWORK_H
#define MYWORK_H

#include <QObject>

class MyWork : public QObject
{
    Q_OBJECT
public:
    explicit MyWork(QObject *parent = nullptr);
    //要处理的业务逻辑
    void working();

signals:
    void curNum(int num);

public slots:

};

#endif // MYWORK_H
```

```cpp
#include "mywork.h"
#include <QDebug>
#include <QThread>

MyWork::MyWork(QObject *parent) : QObject(parent)
{

}
void MyWork::working(){
    qDebug()<<"子线程开始运行...";

    int num = 0;

    while(num < 100000){

        emit curNum(num++);

        QThread::msleep(10);
    }

    qDebug()<<"子线程结束.";
}
```

```cpp
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>
#include "mywork.h"
#include <QThread>

namespace Ui {
class MainWindow;
}

class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    explicit MainWindow(QWidget *parent = nullptr);
    ~MainWindow();

private slots:
```

```
    void showNum(int num);

private:
    Ui::MainWindow *ui;
    MyWork *work;
    QThread* sub;
};

#endif // MAINWINDOW_H
```

```
#include "mainwindow.h"
#include "ui_mainwindow.h"
#include <QDebug>
#include <QThread>
#include "mywork.h"

MainWindow::MainWindow(QWidget *parent) :
    QMainWindow(parent),
    ui(new Ui::MainWindow)
{
    ui->setupUi(this);

    work = new MyWork;
    sub = new QThread;

    work->moveToThread(sub);
    //此时子线程运行 但没有执行working（）
    sub->start();

    connect(ui->pushButton, &QPushButton::clicked, work,
&MyWork::working);
    connect(work, SIGNAL(curNum(int)), this, SLOT(showNum(int)));

}

MainWindow::~MainWindow()
{
    delete ui;

    delete work;
}
```

```
void MainWindow::showNum(int num){
    ui->label->setNum(num);
}
```

使用这种多线程方式，假设有多个不相关的业务流程需要被处理，那么就可以创建多个类似于MyWork的类，将业务流程放入类的公共成员函数中，然后将这个业务类的实例对象通过moveToThread()移动到对应的子线程中就可以了，这样可以让编写的程序更加灵活，可读性更强，更易于维护。

7.2 线程间同步

7.2.1 线程同步基础

使用多线程的主要想法是希望它们可以尽可能并发执行提高效率， 在实际多线程应用编程中，由于多个线程的存在，线程间可能需要访问同一个变量，或者一个线程需要等待另外一个线程完成某个操作之后才产生相应的动作，这时候就要考虑线程间的同步、互斥问题了。

以下程序如果不考虑互斥同步问题，会导致程序执行结果异常。

```
#ifndef MYTHREAD_H
#define MYTHREAD_H

#include <QThread>

class Mythread : public QThread
{
    Q_OBJECT
public:
    explicit Mythread(QObject *parent = nullptr);

signals:

public slots:

protected:
    virtual void run();
};

#endif // MYTHREAD_H
```

```
#include "mythread.h"
#include <QDebug>

extern int g_val;

Mythread::Mythread(QObject *parent) : QThread(parent)
{
    qDebug()<<"Main thread ID:" << QThread::currentThreadId();
}

void Mythread::run(){
    int i = 0;

    while(i<100000){
        g_val++;
        i++;
        //QThread::msleep(10);
    }

    qDebug()<<"work thread :"<< QThread::currentThreadId() <<
"finished g_val="<<g_val;
}
```

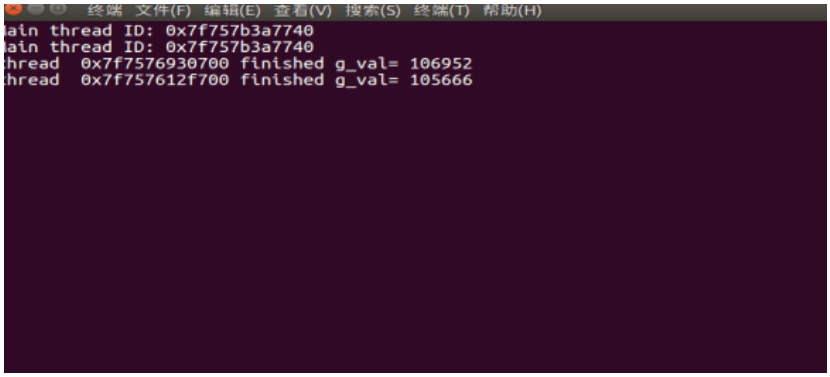
```
#include <QCoreApplication>
#include "mythread.h"

int g_val = 0;
int main(int argc, char *argv[])
{
    QCoreApplication a(argc, argv);

    Mythread t1, t2;

    t1.start();
    t2.start();

    return a.exec();
}
```



注意：以上实验现象不唯一。

Qt中提供给我们用于线程间同步的类主要包括：QMutex QReadWriteLock QWaitCondition QSemaphore.

7.2.2 QMutex

是强制互斥的基本类。线程锁定互斥锁是为了访问共享资源。如果第二个线程在互斥锁已经锁定时试图锁定互斥锁，则第二个线程将进入休眠状态，直到第一个线程完成其任务并解锁互斥锁。

```
//需要包含的头文件
#include <QMutex>
//定义互斥量
QMutex mutex;
//加锁操作
mutex.lock();

/*访问共享资源*/

//解锁操作
mutex.unlock();
```

前面程序中g_val作为工作线程中共享的数据，可以使用以上加锁方式访问。

```
#include "mythread.h"
#include <QDebug>
#include <QMutex>

extern int g_val;
QMutex mutex; //特别注意多个线程使用的是同一把锁

Mythread::Mythread(QObject *parent) : QThread(parent)
{
    qDebug()<<"Main thread ID:" << QThread::currentThreadId();
}
```

```
void Mythread::run(){
    int i = 0;

    while(i<100000000){
        mutex.lock();
        g_val++;
        mutex.unlock();
        i++;
        //QThread::msleep(10);
    }

    qDebug()<<"thread "<< QThread::currentThreadId() << "finished
g_val="<<g_val;

}
```

7.2.3 QReadWriteLock

类似于QMutex，除了它区分了“读”和“写”访问。当一段数据没有被写入时，多个线程同时从中读取是安全的。QMutex强制多个读取线程轮流读取共享数据，而QReadWriteLock允许同时读取，从而提高了并行性。（读锁共享，写锁互斥）

例如，假设有一个数据采集程序，一个线程负责采集数据到缓冲区，一个线程负责读取缓冲区的数据并显示，另一线程负责读取缓冲区数据并保存到文件。

```
//需要包含的头文件
#include <QReadWriteLock>
int buffer[100]={0};
QReadWriteLock lock;

void WriteThread::run()
{
    ...
    lock.lockForWrite();
    get_data_write_buffer(); //写共享资源
    lock.unlock();
    ...
}

void ShowThread::run()
{
    ...
```

```
    lock.lockForRead();
    show_file(); //读共享资源
    lock.unlock();
    ...
}
void SaveThread::run()
{
    ...
    lock.lockForRead();
    save_buffer_tofile(); //读共享资源
    lock.unlock();
    ...
}
```

这样如果WriteThread没有以lockForWrite形式锁定 lock，ShowThread和SaveThread就可以同时访问buffer。

7.2.4 QWaitCondition

同步线程不是通过强制互斥，而是通过提供一个条件变量。其他原语使线程等待资源被解锁，而QWaitCondition使线程等待特定条件被满足。为了让等待的线程继续，可以调用wakeOne()随机唤醒一个线程，或者调用wakeAll()同时唤醒所有线程。

```
//解锁互斥量lockedMutex，并阻塞等待唤醒条件； 被唤醒后锁定lockedMutex并退出该函数
bool wait(QMutex *lockedMutex, unsigned long time = ULONG_MAX)
bool wait(QReadWriteLock *lockedReadWriteLock, unsigned long time = ULONG_MAX)

//唤醒所有处于等待状态的线程， 唤醒的顺序不确定
void wakeAll();
//唤醒一个处于等待状态的线程，唤醒哪个不确定
void wakeOne()
```

上面的例子中，如果采集线程没有向buffer中写入数据，而显示和保存线程就显示或者向文件中写入，那操作的就是“脏数据”。逻辑上应该是采集线程向buffer中写入了数据，其它两个线程再去读，这时候就可以使用QWaitCondition来实现了。

```
//需要包含的头文件
#include <QReadWriteLock>
#include <QWaitCondition>
```

```
static int buffer[100]={0};
static QReadWriteLock lock;
static QWaitCondition newDataAvailable;

void WriteThread::run()
{
    ...
    lock.lockForWrite();
    get_data_write_buffer(); //写共享资源
    lock.unlock();
    newDataAvailable.wakeAll();
    ...
}

void ShowThread::run()
{
    ...
    lock.lockForRead();
    newDataAvailable.wait(&lock);
    show_file(); //读共享资源
    lock.unlock();
    ...
}
void SaveThread::run()
{
    ...
    lock.lockForRead();
    newDataAvailable.wait(&lock);
    save_buffer_tofile(); //读共享资源
    lock.unlock();
    ...
}
```

7.2.5 QSemaphore

是QMutex的一个推广,它保护了一定数量的相同资源。相反，QMutex只保护一个资源。比如信号量的一个典型应用：同步生产者和消费者之间对循环缓冲区的访问。

Semaphore提供两种基本的操作，acquire() and release()。

```
//尝试获得n个资源， 如果没有这么多资源， 线程阻塞直到有n个资源可用
void acquire(int n = 1);
//返回可用资源个数，这个是一个非负整数，为0代表当前没有资源可用
int available() const;
//释放n个资源，增加可用资源个数
void release(int n = 1);
```

```
#include <QtCore/QCoreApplication>
#include <QSemaphore>
#include <QThread>
#include <iostream>
#include <QRandomGenerator>

static int DataSize=100;
static QSemaphore goods (0);

//生产者
class Producer:public QThread
{
protected:
    void run();

};
void Producer::run(){
    for (int i = 0;i < DataSize;++i)
    {
        goods.release(1);
        std::cerr<<"P";
        QThread::msleep(static_cast<unsigned long>
(QRandomGenerator::global()->bounded(100,150)));
    }
}

//消费者
class Consumer:public QThread
{
protected:
    void run();

};
void Consumer::run(){
```

```
    for (int i = 0;i < DataSize;++i)
    {
        goods.acquire(1);
        std::cerr<<"C";
        QThread::msleep(static_cast<unsigned long>
(QRandomGenerator::global()->bounded(100,200)));
    }
}

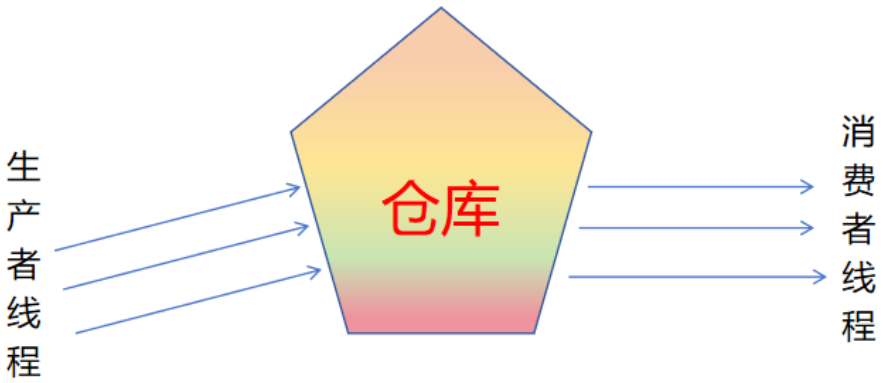
int main(int argc, char *argv[])
{
    QCoreApplication a(argc, argv);

    Producer producer;
    Consumer consumer;
    producer.start();
    consumer.start();
    producer.wait();

    consumer.wait();

    return a.exec();
}
```

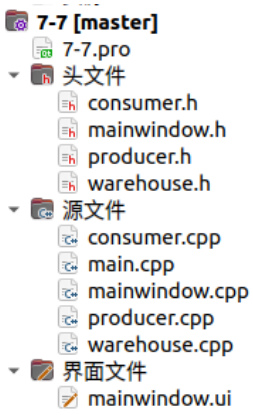
7.3综合实例



实验效果如下图所示：



对应的代码：




```
#ifndef WAREHOUSE_H
#define WAREHOUSE_H
#include <QMutex>
#include <QWaitCondition>
#include <QStringList>

class WareHouse
{
public:
    WareHouse(int capacity=5);
    ~WareHouse();
private:
    char *buffer ; //记录仓库的存储空间
    int capacity; //仓库容量
    int size; //现存商品数量
    int readPos, writePos; //读写位置
```

```
public:
    QMutex mutex;
    QWaitCondition notEmpty;
    QWaitCondition notFull;

    bool full();
    bool empty();

    void put_goods(char c); //向仓库中放商品
    char get_goods(); //从仓库中取商品
    /*获取仓库中所有的商品 但不删除商品*/
    QStringList peek_goods();

};

#endif // WAREHOUSE_H
```

```
#include "warehouse.h"

WareHouse::WareHouse(int cap)
{
    capacity = cap;

    buffer = new char[cap];

    size = 0;
    readPos = 0;
    writePos = 0;
}

WareHouse::~WareHouse()
{
    delete []buffer;
}

bool WareHouse::full()
{
    return size == capacity;
}

bool WareHouse::empty()
{
```

```
    return !size;
}

void WareHouse::put_goods(char c)
{
    buffer[writePos++] = c;
    size++;

    if(writePos == capacity)
        writePos = 0;
}

char WareHouse::get_goods()
{
    char c = buffer[readPos++];
    size--;

    if(readPos == capacity)
        readPos=0;
    return c;
}

QStringList WareHouse::peek_goods()
{
    QStringList list;

    int pos = readPos;
    QString c;

    for(int i=0; i<size; i++){
        c = buffer[pos++];
        if(pos==capacity)
            pos = 0;
        list << c;
    }
    return list;
}
```

```
#ifndef PRODUCER_H
#define PRODUCER_H

#include <QThread>
```

```
class Producer : public QThread
{
    Q_OBJECT
public:
    explicit Producer(char goods, QObject *parent = nullptr);

signals:
    void product_refresh(const QString &);
protected:
    void run();
private:
    char start_goods; //a  b  c     ; A B C ...

};

#endif // PRODUCER_H
```

```
#include "producer.h"
#include "warehouse.h"
#include <QDebug>
#include <QDateTime>
#include <QRandomGenerator>

extern WareHouse house;

Producer::Producer(char goods, QObject *parent)
    : QThread{parent}
{
    start_goods = goods;
}

void Producer::run(){
    char goods = start_goods;

    while(1){
        /*获取仓库使用权限*/
        house.mutex.lock();

        while(house.full()){ //仓库满
            qDebug()<<"生产者线程: " <<QThread::currentThreadId()
<<" 阻塞等待仓库不满";
            house.notFull.wait(&house.mutex);
```

```
        }

        house.put_goods(goods);

        house.mutex.unlock(); //释放仓库的使用权力

        house.notEmpty.wakeAll(); //唤醒因仓库空而睡眠的消费者线程


        emit product_refresh(QString("线程%1: %2: 生产:%3")
                                .arg(*(static_cast<int *>
(QThread::currentThreadId())))

.arg(QDateTime::currentDateTime().toString("hh:mm:ss"))
                                .arg(goods));

        goods++;
        if(goods >= start_goods+26)
            break;

        QThread::sleep(QRandomGenerator::global()->bounded(1,2));
    }
}
```

```
#ifndef CONSUMER_H
#define CONSUMER_H

#include <QThread>

class Consumer : public QThread
{
    Q_OBJECT
public:
    explicit Consumer(QObject *parent = nullptr);

signals:
    void consumer_refresh(const QString &);
protected:
    void run();
};

#endif // CONSUMER_H
```

```
#include "consumer.h"
#include "warehouse.h"
#include <QDebug>
#include <QDateTime>
#include <QRandomGenerator>

extern WareHouse house;
Consumer::Consumer(QObject *parent)
    : QThread{parent}
{

}
void Consumer::run(){

    while(1){
        house.mutex.lock();//获取仓库的访问权限

        while(house.empty()){
            qDebug()<<"消费线程: "<<QThread::currentThreadId() <<
" 因仓库空而阻塞";
            house.notEmpty.wait(&house.mutex);
        }
        char goods = house.get_goods();

        house.mutex.unlock();//释放仓库的使用权限

        house.notFull.wakeAll();

        emit consumer_refresh(QString("线程%1: %2: 消费: %3")
                                  .arg(*(static_cast<int *>
(QThread::currentThreadId())))

.arg(QDateTime::currentDateTime().toString("hh:mm:ss"))
                                  .arg(goods));
        QThread::sleep(QRandomGenerator::global()->bounded(2,3));
    }
}
```

```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H
```

```
#include <QMainWindow>
#include "producer.h"
#include "consumer.h"
#include "warehouse.h"
#include <QStringList>

QT_BEGIN_NAMESPACE
namespace Ui { class MainWindow; }
QT_END_NAMESPACE

class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    MainWindow(QWidget *parent = nullptr);
    ~MainWindow();

private:
    Ui::MainWindow *ui;

    Consumer c1,c2,c3;
    Producer *p1, *p2;
protected slots:
    void on_product_refresh(const QString &);
    void on_consumer_refresh(const QString &);

private:
    void show_goods(QStringList &);
};
#endif // MAINWINDOW_H
```

```
#include "mainwindow.h"
#include "ui_mainwindow.h"
#include <QDateTime>
#include "warehouse.h"
#include <QThread>

extern WareHouse house;

MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
```

```
    , ui(new Ui::MainWindow)
{
    ui->setupUi(this);

    p1 = new Producer('a');
    p2 = new Producer('A');
    /*信号和槽的连接*/
    connect(&c1, &Consumer::consumer_refresh, this,
&MainWindow::on_consumer_refresh);
    connect(&c2, &Consumer::consumer_refresh, this,
&MainWindow::on_consumer_refresh);
    connect(&c3, &Consumer::consumer_refresh, this,
&MainWindow::on_consumer_refresh);
    connect(p1, &Producer::product_refresh, this,
&MainWindow::on_product_refresh);
    connect(p2, &Producer::product_refresh, this,
&MainWindow::on_product_refresh);

    /*启动工作线程*/
    c1.start();
    c2.start();
    c3.start();
    p1->start();
    p2->start();

}

MainWindow::~MainWindow()
{
    delete ui;
    /*停止线程*/
    c1.terminate();
    c2.terminate();
    c3.terminate();
    p1->terminate();
    p2->terminate();

    c1.wait();
    c2.wait();
    c3.wait();
    p1->wait();
    p2->wait();
```

```
    delete p1;
    delete p2;

}

void MainWindow::on_product_refresh(const QString &info)
{
    ui->pte_producer->appendPlainText(info);
    house.mutex.lock();
    QStringList list = house.peek_goods();
    house.mutex.unlock();
    show_goods(list);
}

void MainWindow::on_consumer_refresh(const QString &info)
{
    ui->pte_consumer->appendPlainText(info);
    house.mutex.lock();
    QStringList list = house.peek_goods();
    house.mutex.unlock();
    show_goods(list);
}

void MainWindow::show_goods(QStringList &list)
{
    QString goods_list;

    for(int i=0; i<list.size(); i++){
        goods_list += list[i];
        goods_list += " ";
    }

    ui->pte_warehouse->appendPlainText(QString("%1: 现有商品: %2")

.arg(QDateTime::currentDateTime().toString("hh:mm:ss"))
                                                .arg(goods_list));
}
```

```
#include "mainwindow.h"
#include <QApplication>

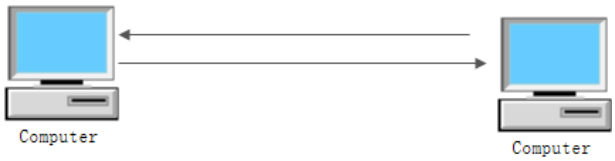
/*简单起见 搞成全局变量 实则可以以局部变量形式出现， 创建其他对象时作为参数传递*/
```

```
WareHouse house(10);

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    MainWindow w;
    w.show();

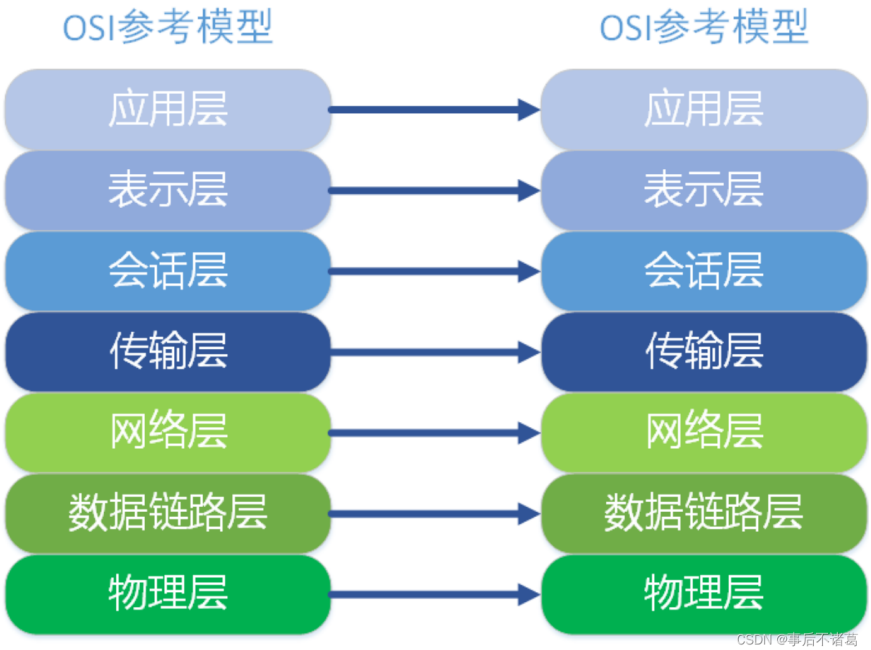

    return a.exec();
}
```

8.网络编程

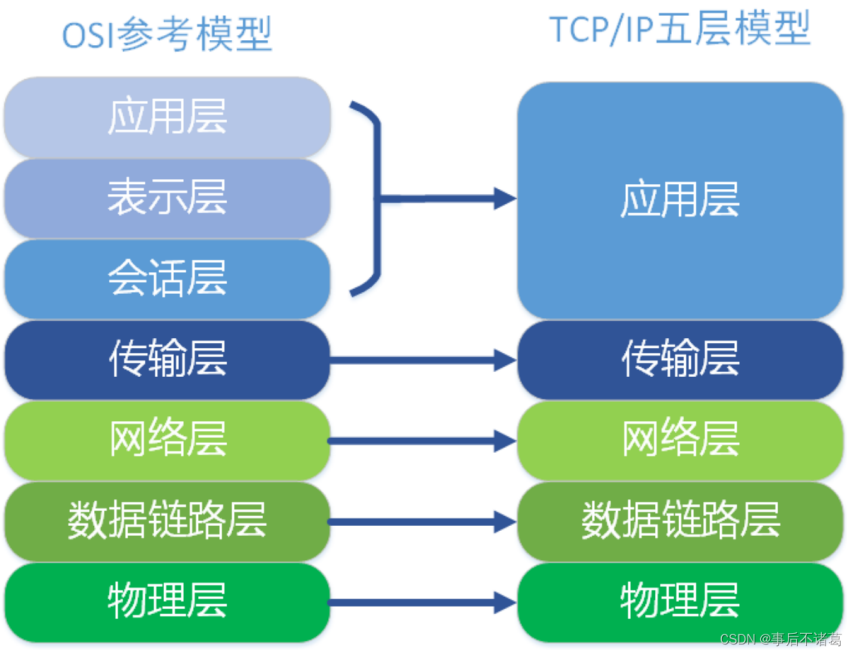


网络整个通信过程非常复杂，对于这种比较复杂的问题，我们通常采用分层来解决，每一层负责自己的工作，以及定义后相互使用的接口。采用这种方式，将一个大的问题细分为小的、目标更加确定的小问题。

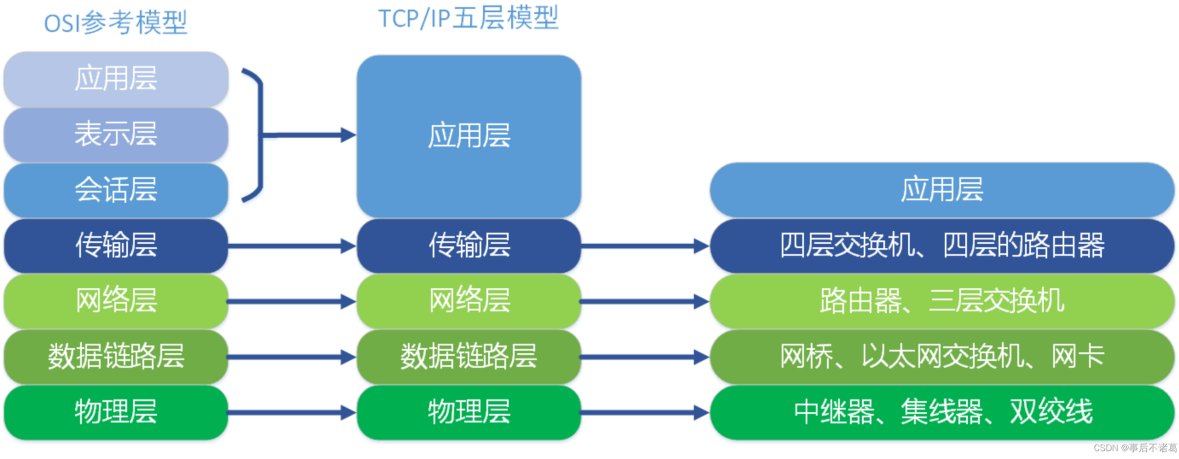
- 计算机网络的分层体系
  - OSI（Open Systems Interconnection Reference Model）七层模型



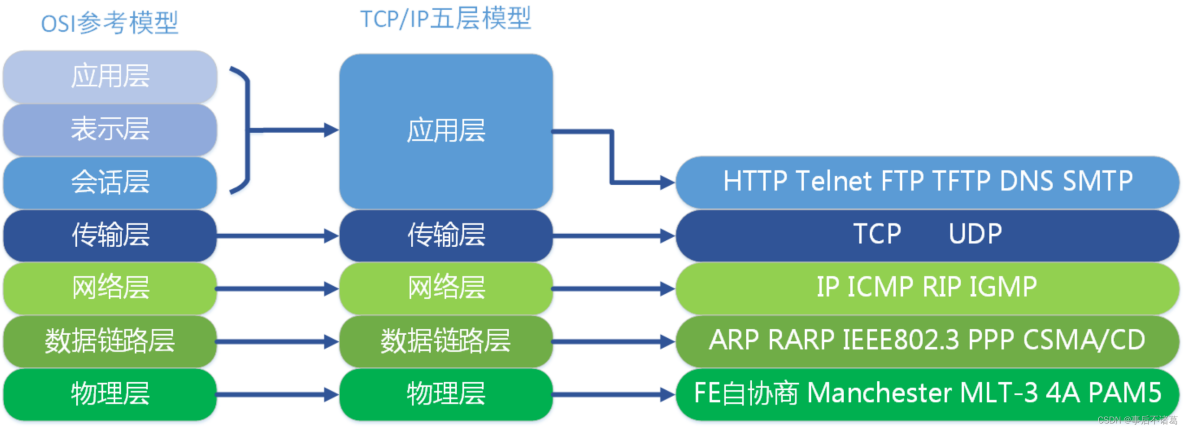
  - TCP/IP网络模型（五层模型）



- 各层级中的设备



- 各层级中的协议



Qt Network模块提供了用于编写TCP/IP网络应用程序的各种类，如用于TCP通信的**QTcpSocket** 和**QTcpServer**，用于UDP通信的**QUdpSocket**，还有用于网络承载管理的类，以及基于SSL协议的用于网络安全通信的类。

8.1 主机信息查询

QHostInfo类可以根据主机名获取主机的IP地址，或者通过IP地址获取主机名。QHostInfo类的静态函数localHostName（）可获取本机的主机名，静态函数fromName()可以通过主机名获取IP地址，静态函数lookupHost()可通过一个主机名以异步方式查找这个主机的IP地址。

```
                                QHostInfo(int id = -1)
                                QHostInfo(const QHostInfo &other)
                                ~QHostInfo()
          QList<QHostAddress>   addresses() const      返回对应的IP列表   /etc/hosts文件
QHostInfo::HostInfoError        error() const
                    QString     errorString() const
                    QString     hostName() const
                        int     lookupId() const
                       void     setAddresses(const QList<QHostAddress> &addresses)
                       void     setError(QHostInfo::HostInfoError error)
                       void     setErrorString(const QString &str)
                       void     setHostName(const QString &hostName)
                       void     setLookupId(int id)
                       void     swap(QHostInfo &other)
                QHostInfo &     operator=(const QHostInfo &other)
                QHostInfo &     operator=(QHostInfo &&other)
```

Static Public Members

```
                       void     abortHostLookup(int id)
                  QHostInfo     fromName(const QString &name)      根据主机名称获取对应IP
                    QString     localDomainName()
                    QString     localHostName()  获取本机名称    /etc/hosts文件
                        int     lookupHost(const QString &name, QObject *receiver, const char *member)   根据域名获取IP
                        int     lookupHost(const QString &name, const QObject *receiver, PointerToMemberFunction function)
                        int     lookupHost(const QString &name, Functor functor)
                        int     lookupHost(const QString &name, const QObject *context, Functor functor)
```

实例程序：




```
#include "mainwindow.h"
#include "ui_mainwindow.h"
#include <QHostInfo>
```

```
MainWindow::MainWindow(QWidget *parent) :
    QMainWindow(parent),
    ui(new Ui::MainWindow)
{
    ui->setupUi(this);
}

MainWindow::~MainWindow()
{
    delete ui;
}

void MainWindow::on_btn_hostinfo_clicked()
{
    ui->plainTextEdit->clear();
    QString hostName = QHostInfo::localHostName();
    ui->plainTextEdit->appendPlainText("本机主机名: " + hostName +
"\n");
    QHostInfo hostInfo =  QHostInfo::fromName(hostName);
    QList<QHostAddress> addrList = hostInfo.addresses();

    for(int i=0; i<addrList.size(); i++){
        if(addrList[i].protocol() ==
QAbstractSocket::IPv4Protocol)
            ui->plainTextEdit->appendPlainText("本机IP: " +
addrList[i].toString());
    }
    /*
    QList<QHostAddress>::iterator it;
    for(it=addrList.begin(); it!=addrList.end(); it++){
        if(addrList[i].protocol() ==
QAbstractSocket::IPv4Protocol)
            ui->plainTextEdit->appendPlainText("本机IP: " +
(*it).toString());
    }
    */

}


void MainWindow::on_btn_domain_info_clicked()
{
    QString domain = ui->le_domain->text();
```

```
    ui->plainTextEdit->appendPlainText("正在搜索域名: " + domain +
" 对应的IP...");

    QHostInfo::lookupHost(domain, this, SLOT(do_lookup_host(const
QHostInfo&)));
}

void MainWindow::do_lookup_host(const QHostInfo &hostInfo)
{
    QList<QHostAddress> addrList = hostInfo.addresses();

    foreach(QHostAddress host, addrList){
        if(host.protocol() == QAbstractSocket::IPv4Protocol)
            ui->plainTextEdit->appendPlainText(ui->le_domain-
>text()+ " IP: " + host.toString());
    }

}
```

QNetworkInterface类可以获得运行主机的所有IP地址和网络接口列表。静态函数allInterface()返回主机上所有的网络接口列表，一个网络接口可能包含多个IP地址，每个IP地址与掩码或广播地址关联。如果无须知道子网掩码和广播地址，使用静态函数allAddresses()可以获取主机上所有的IP地址列表。

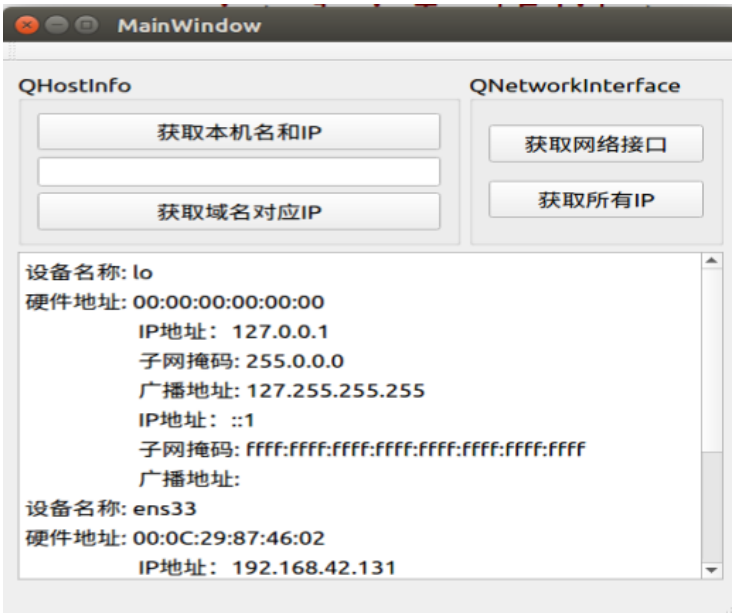
Public Functions

	QNetworkInterface()
	QNetworkInterface(const QNetworkInterface &*other*)
	~QNetworkInterface()
QList<QNetworkAddressEntry>	**addressEntries**() const
QNetworkInterface::InterfaceFlags	**flags**() const
QString	**hardwareAddress**() const
QString	**humanReadableName**() const
int	**index**() const
bool	**isValid**() const
int	**maximumTransmissionUnit**() const
QString	**name**() const
void	**swap**(QNetworkInterface &*other*)
QNetworkInterface::InterfaceType	**type**() const
QNetworkInterface &	**operator=**(QNetworkInterface &&*other*)
QNetworkInterface &	**operator=**(const QNetworkInterface &*other*)

Static Public Members

QList<QHostAddress>	**allAddresses**()
QList<QNetworkInterface>	**allInterfaces**()
QNetworkInterface	**interfaceFromIndex**(int *index*)
QNetworkInterface	**interfaceFromName**(const QString &*name*)
int	**interfaceIndexFromName**(const QString &*name*)
QString	**interfaceNameFromIndex**(int *index*)
const QMetaObject	**staticMetaObject**

实例程序:




```
void MainWindow::on_btn_nicInfo_clicked()
{
    ui->plainTextEdit->clear();
    //获取网络接口列表
    QList<QNetworkInterface> list =
QNetworkInterface::allInterfaces();

    foreach(QNetworkInterface interface , list){
```

```
        ui->plainTextEdit->appendPlainText("设备名称: " +
interface.humanReadableName());
        ui->plainTextEdit->appendPlainText("硬件地址: " +
interface.hardwareAddress());
        //地址列表
        QList<QNetworkAddressEntry> entryList =
interface.addressEntries();
        foreach(QNetworkAddressEntry entry, entryList){
            ui->plainTextEdit->appendPlainText("\tIP地址: " +
entry.ip().toString());
            ui->plainTextEdit->appendPlainText("\t子网掩码:
"+entry.netmask().toString());
            ui->plainTextEdit->appendPlainText("\t广播地址: " +
entry.broadcast().toString());
        }
    }
}

void MainWindow::on_btn_allIP_clicked()
{
    ui->plainTextEdit->clear();

    QList<QHostAddress> list = QNetworkInterface::allAddresses();

    QList<QHostAddress>::iterator it;
    for(it=list.begin(); it!=list.end(); it++){
        ui->plainTextEdit->appendPlainText((*it).toString());
    }
}
```

8.2 TCP通信

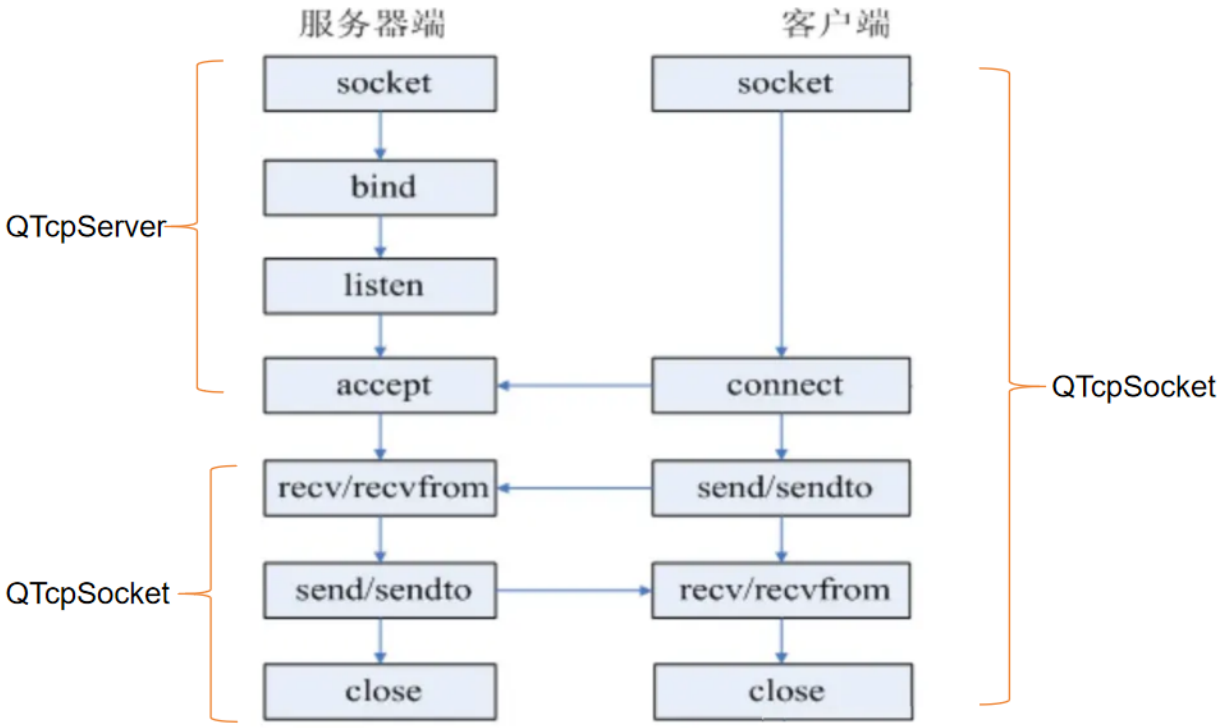
传输控制协议(transmission control protocol, TCP)是一种被大多数Internet网络协议用于数据传输的底层网络协议，它是可靠的、面向流和连接的传输协议，适用于连续数据的传输。TCP通信必须先建立TCP连接，通信端分为客户端和服务器端。

使用Qt提供的类进行基于TCP的套接字通信需要用到两个类：

- QTcpServer：服务器类，用于监听客户端连接以及和客户端建立连接。
- QTcpSocket：通信的套接字类，客户端、服务器端都需要使用。

这两个套接字通信类都属于网络模块`network`。

整个编程流程，如下图所示。



8.2.1 QTcpServer

重点关注的函数：

Public Functions

	QTcpServer(QObject *parent = nullptr)
virtual	**~QTcpServer**()
void	**close**()
QString	**errorString**() const 返回出错信息
virtual bool	**hasPendingConnections**() const
bool	**isListening**() const 监听指定IP 端口
bool	**listen**(const QHostAddress &*address* = QHostAddress::Any, quint16 *port* = 0)
int	**maxPendingConnections**() const
virtual QTcpSocket *	**nextPendingConnection**() 返回下一个等待接入的连接
void	**pauseAccepting**()
QNetworkProxy	**proxy**() const
void	**resumeAccepting**()
QHostAddress	**serverAddress**() const
QAbstractSocket::SocketError	**serverError**() const
quint16	**serverPort**() const
void	**setMaxPendingConnections**(int *numConnections*)
void	**setProxy**(const QNetworkProxy &*networkProxy*)
bool	**setSocketDescriptor**(qintptr *socketDescriptor*)
qintptr	**socketDescriptor**() const
bool	**waitForNewConnection**(int *msec* = 0, bool *timedOut* = nullptr)

重点关注的信号：

Signals

void **acceptError**(QAbstractSocket::SocketError *socketError*)

void **newConnection**() 有新的客户端连接时发射该信号

• 2 signals inherited from QObject

8.2.2 QTcpSocket

重点关注的函数：

```
//继承自QAbstractSocket
//连接服务器
virtual void connectToHost(const QHostAddress &address, quint16
port, QIODevice::OpenMode openMode = ReadWrite)
//断开连接
virtual void disconnectFromHost()

//继承自QIODevice
//判断是否有数据可读
virtual qint64 bytesAvailable() const
//读取接收到的数据
QByteArray read(qint64 maxSize)
QByteArray readAll()
//发送数据
qint64 write(const QByteArray &byteArray)
```

重点关注的信号：

```
//继承自QAbstractSocket
//connectToHost连接服务器成功后发射此信号
void connected()
//断开连接后发射此信号
void disconnected()
//有错误发生时 发射此信号
void error(QAbstractSocket::SocketError socketError)

//继承自QIODevice
//当缓冲区中有数据需要读时发射此信号
void readyRead()
```

8.2.3 网络聊天室编程实例



服务器端代码

```
#ifndef DIALOG_H
#define DIALOG_H

#include <QDialog>
#include <QTcpServer>
#include <QTcpSocket>
#include <QDebug>
#include <QTimer>

namespace Ui {
class Dialog;
}

class Dialog : public QDialog
{
    Q_OBJECT

public:
    explicit Dialog(QWidget *parent = nullptr);
    ~Dialog();

private slots:
    void on_btn_start_clicked();

    void onNewConnection();
    void onReadyRead();
    void sendMessage(const QByteArray&);
    void onTimeout(void);
    void on_btn_stop_clicked();
```

```
    void on_btn_clear_clicked();

    void on_btn_quit_clicked();

private:
    Ui::Dialog *ui;
    QTcpServer tcpServer;
    quint16 port;
    //保存和客户端通信的套接字
    QList<QTcpSocket*> tcpCLientList;
    QTimer freshTimer;

};

#endif // DIALOG_H
```

```
#include "dialog.h"
#include "ui_dialog.h"
#include <QAbstractSocket>

Dialog::Dialog(QWidget *parent) :
    QDialog(parent),
    ui(new Ui::Dialog)
{
    ui->setupUi(this);

    connect(&tcpServer, &QTcpServer::newConnection,
            this, &Dialog::onNewConnection);
    connect(&freshTimer, SIGNAL(timeout()),
            this, SLOT(onTimeout()));

    freshTimer.start(1000);
}

Dialog::~Dialog()
{
    delete ui;
}

void Dialog::on_btn_start_clicked()
{
```

```
    //获取端口
    port = static_cast<quint16>(ui->sb_port->value());
    if(tcpServer.listen(QHostAddress::Any, port)== true){
        qDebug()<<port;
        qDebug()<<"创建服务器成功!";
    }
    else{
        qDebug()<<"创建服务器失败!";
    }

}

void Dialog::onNewConnection()
{
    QTcpSocket *tcpClient = tcpServer.nextPendingConnection();
    tcpCLientList.append(tcpClient);

    connect(tcpClient, &QTcpSocket::readyRead,
            this, &Dialog::onReadyRead);
}

void Dialog::onReadyRead()
{
    for(int i=0; i<tcpCLientList.size(); i++){
        if(tcpCLientList.at(i)->bytesAvailable()){
            QByteArray buf = tcpCLientList.at(i)->readAll();
            ui->textEdit->append(buf);

            sendMessage(buf);
        }
    }
}

void Dialog::sendMessage(const QByteArray &buf)
{
    for(int i=0; i<tcpCLientList.size(); i++){
        tcpCLientList.at(i)->write(buf);
    }
}

void Dialog::onTimeout()
{
    for(int i=0; i<tcpCLientList.size(); i++){
```

```
        if(tcpCLientList.at(i)->state() ==
QAbstractSocket::UnconnectedState){
            tcpCLientList.removeAt(i);
            i--;
        }

    }
}

void Dialog::on_btn_stop_clicked()
{
    tcpServer.close();
    for(int i=0; i<tcpCLientList.size(); i++){
        tcpCLientList.at(i)->disconnectFromHost();
    }
}

void Dialog::on_btn_clear_clicked()
{
    ui->textEdit->clear();
}

void Dialog::on_btn_quit_clicked()
{
    close();
}
```

客户端代码

```
#ifndef DIALOG_H
#define DIALOG_H

#include <QDialog>
#include <QTcpSocket>
#include <QHostAddress>
#include <QDebug>

namespace Ui {
class Dialog;
}

class Dialog : public QDialog
{
```

```
    Q_OBJECT

public:
    explicit Dialog(QWidget *parent = nullptr);
    ~Dialog();

private slots:


    void on_btn_connect_clicked();

    void on_btn_send_clicked();

    void onConnected();
    void onDisConnected();
    void onReadyRead();
    void onError();


private:
    Ui::Dialog *ui;
    QTcpSocket tcpSocket;
    QHostAddress serverIP;
    quint16 serverPort;
    QString username;

    bool connectFlag;
};

#endif // DIALOG_H
```

```
#include "dialog.h"
#include "ui_dialog.h"
#include <QMessageBox>
Dialog::Dialog(QWidget *parent) :
    QDialog(parent),
    ui(new Ui::Dialog)
{
    ui->setupUi(this);

    connect(&tcpSocket, SIGNAL(connected()), this,
SLOT(onConnected()));
```

```
    connect(&tcpSocket, SIGNAL(disconnected()), this,
SLOT(onDisConnected()));
    connect(&tcpSocket, SIGNAL(readyRead()), this,
SLOT(onReadyRead()));
    connect(&tcpSocket,
SIGNAL(error(QAbstractSocket::SocketError)), this,
SLOT(onError()));

    connectFlag = false;
}

Dialog::~Dialog()
{
    delete ui;
}



void Dialog::on_btn_connect_clicked()
{
    if(connectFlag){
        QString msg = QString("%1 离开聊天室").arg(username);
        tcpSocket.write(msg.toUtf8());
        tcpSocket.disconnectFromHost();
    }
    else{
       serverIP.setAddress(ui->le_ip->text());
       serverPort =static_cast<quint16>(ui->le_port-
>text().toShort());
       username = ui->le_name->text();
       //连接后发射 成功或者失败的信号
       tcpSocket.connectToHost(serverIP, serverPort);
    }
}
void Dialog::on_btn_send_clicked()
{
    QString msg = ui->edit_send->toPlainText();
    msg = username +"说: "+ msg;

    tcpSocket.write(msg.toUtf8());
    ui->edit_send->clear();
}
```

```
void Dialog::onConnected(){
    connectFlag = true;

    QString msg = QString("欢迎 %1 进入聊天室").arg(username);
    tcpSocket.write(msg.toUtf8());

    ui->btn_connect->setText("断开连接");
}
void Dialog::onDisConnected(){
    connectFlag = false;
    qDebug()<<"aa";
    ui->btn_connect->setText("连接服务器");
}

void Dialog::onReadyRead(){
    QByteArray buf = tcpSocket.readAll();

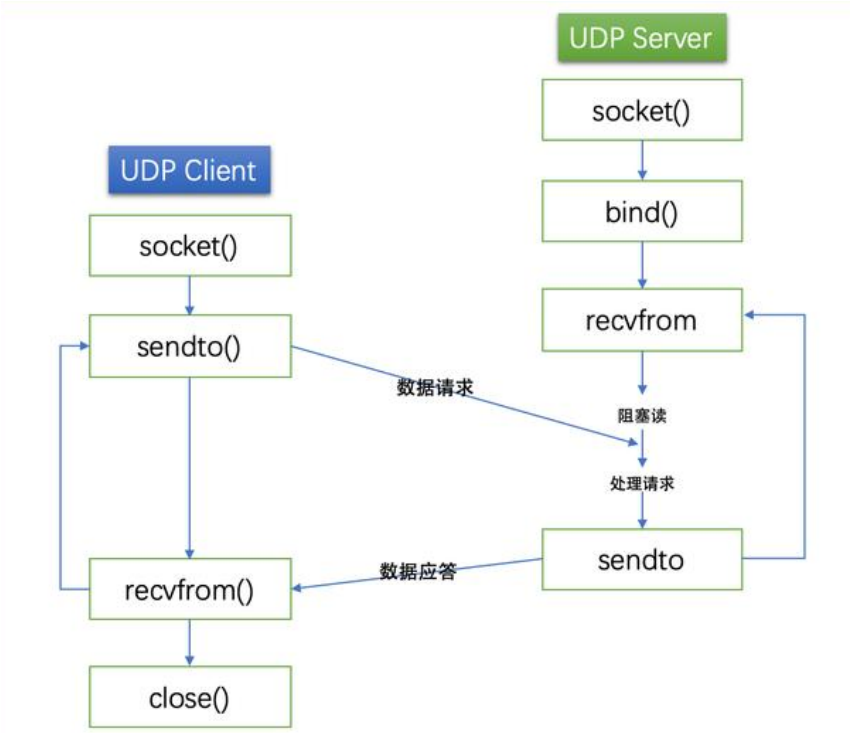

    ui->edit_recv->append(QString(buf));

}
void Dialog::onError(){
    QMessageBox::critical(this, "连接服务器",
tcpSocket.errorString());
}
```

8.3 UDP通信

用户数据报协议（User Datagram Protocol ,UDP)是一种无连接的传输层协议，提供面向事务的简单不可靠信息传送服务，与 TCP 不同， UDP 并不提供对 IP 协议的可靠机制、流控制以及错误恢复功能等。 适用于需要快速传输少量数据的场景，因为它不会建立连接，所以传输效率比较高， 通常用于实时应用程序，比如视频会议在线游戏、语音聊天等，因为这些应用程序通常要求实时性比较高，而且数据包丢失对应用程序的影响也不是很大。

Linux系统编程中，UDP通信的编程模型如下图所示：



Qt环境下UDP编程，如果通信的双方都绑定特定的端口号，就不再严格区分客户端与服务器端了。 TCP中需要用到两个类：QTcpSever和QTcpSocket，而UDP中只需要使用一个类QUdpSocket。 UDP通信时需要用到本地端口，目标端口，目标IP地址，其中：

本地端口：自己为该UDP主机设置的端口号，目的是对方发送数据时，会发到本机此端口上。

目标端口：对方的UDP主机设置的端口号，作用是可以在本机上向某个IP用户的端口上发送数据。

8.3.1 QUdpSocket

QUdpSocket类用于实现UDP通信，它与QTcpSocket具有相同的父类QAbstractSocket，因而这两个类共享大部分的接口函数。

重点关注的函数

```
//继承自QAbstractSocket
//绑定端口号
bool bind(quint16 port, QAbstractSocket::BindMode mode)
//关闭套接字
void QAbstractSocket::close()

//QUdpSocket自身定义
//数据报发送，形参分别为数据报，目的IP地址，目的端口号
qint64 writeDatagram(const char *data, qint64 size, const
QHostAddress &address, quint16 port)
```

```
qint64 writeDatagram(const QNetworkDatagram &datagram)
qint64 writeDatagram(const QByteArray &datagram, const
QHostAddress &host, quint16 port)
//socket中有接收到用户发来的数据报时，返回值为true
bool hasPendingDatagrams() const
//返回待读取数据报的大小
qint64 pendingDatagramSize() const
//读取数据报，返回读取数据报的字节数，maxSize为设置的读取字节数
qint64 readDatagram(char *data, qint64 maxSize, QHostAddress
*address = nullptr, quint16 *port = nullptr)
```

重点关注的信号

```
//继承自QIODevice
//当缓冲区中有数据需要读时发射此信号
void readyRead()
```

8.3.2 编程实例




```
#ifndef DIALOG_H
#define DIALOG_H
```

```h
#include <QDialog>
#include <QUdpSocket>

QT_BEGIN_NAMESPACE
namespace Ui { class Dialog; }
QT_END_NAMESPACE

class Dialog : public QDialog
{
    Q_OBJECT

public:
    Dialog(QWidget *parent = nullptr);
    ~Dialog();

private slots:

    void on_btn_bind_clicked();
    void on_readyRead();
    void on_btn_send_clicked();

private:
    Ui::Dialog *ui;
    QUdpSocket socket;
};
#endif // DIALOG_H
```

```cpp
#include "dialog.h"
#include "ui_dialog.h"
#include <QDebug>
#include <QMessageBox>
#include <QHostAddress>

Dialog::Dialog(QWidget *parent)
    : QDialog(parent)
    , ui(new Ui::Dialog)
{
    ui->setupUi(this);
}

Dialog::~Dialog()
{
    delete ui;
```

```
}


void Dialog::on_btn_bind_clicked()
{
    if(ui->btn_bind->text() == "打开"){
        quint16 localPort = ui->le_local_port->text().toUShort();

        if(socket.bind(localPort)){
            qDebug()<<"successed";
            ui->label_status-
>setPixmap(QPixmap(":/res/stop.png"));
            ui->btn_bind->setText("关闭");
            connect(&socket, &QUdpSocket::readyRead, this,
&Dialog::on_readyRead);
        }
        else{
            QMessageBox::critical(this, "Error",
                                  QString("%1 :
%2").arg(socket.errorString()).arg(localPort));
        }
    }
    else{
        ui->btn_bind->setText("打开");
        ui->label_status-
>setPixmap(QPixmap(":/res/running.png"));
        socket.close();
    }
}

void Dialog::on_readyRead()
{
    QByteArray recvBuf;

    while(socket.hasPendingDatagrams()){
        recvBuf.resize(socket.pendingDatagramSize());
        socket.readDatagram(recvBuf.data(), recvBuf.size());
        ui->ple_recv_data->appendPlainText(QString(recvBuf));
    }
}


void Dialog::on_btn_send_clicked()
```

```
{
    //QHostAddress destAddr;
    //destAddr.setAddress(ui->le_remote_ip->text());
    QHostAddress destAddr = QHostAddress(ui->le_remote_ip-
>text());
    quint16  destPort = ui->le_remote_port->text().toUShort();

    QString sendBuff = ui->le_send_data->text();

    socket.writeDatagram(sendBuff.toUtf8(), destAddr, destPort);

}
```

8.4 http下载器的实现

Qt网络模块提供了一些类来实现OSI七层网络模型中高层的网络协议，例如HTTP、FTP、SNMP等，这些类主要是QNetworkRequest、QNetworkAccessManager和QNetworkReply。

- QNetworkRequest， 通过URL发起网络协议请求，也保存网络请求的信息，目前支持HTTP、FTP和本地文件URL的上传和下载。

  ```
  //设置url
  void setUrl(const QUrl &url)
  //设置头部
  void setRawHeader(const QByteArray &headerName, const
  QByteArray &headerValue)
  ```

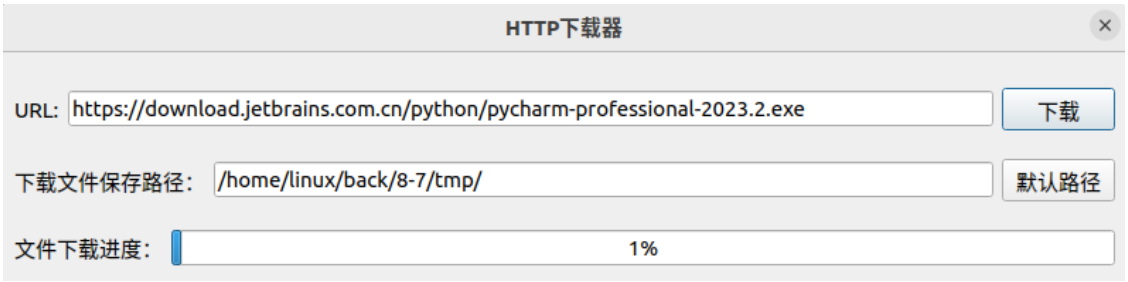
- QNetworkReply，表示网络请求的响应，由 QNetworkAccessManager 在发送网络请求后创建网络响应。QNetworkReply提供的信号finished()、readyRead()、downloadProgress（）可用于监测网络响应的执行情况，进而进行相应的操作。

  ```
  //下载进度信号
  void downloadProgress(qint64 bytesReceived, qint64 bytesTotal)
  //有错误发生发出信号
  void error(QNetworkReply::NetworkError code)
  //结束
  void finished()
  //有数据来时会发出信号
  void readyRead()
  ```

- QNetworkAccessManager，负责发送网络请求，以及创建网络响应。

```
//将数据内容上载到目标请求
QNetworkReply *QNetworkAccessManager::put(const QNetworkRequest
&request, QIODevice *data)
//发送请求，获取响应
QNetworkReply *get(const QNetworkRequest &request)
```

实例效果图：



代码：

```
#ifndef DIALOG_H
#define DIALOG_H

#include <QDialog>
#include <QNetworkAccessManager>
#include <QNetworkReply>
#include <QFile>

QT_BEGIN_NAMESPACE
namespace Ui { class Dialog; }
QT_END_NAMESPACE

class Dialog : public QDialog
{
    Q_OBJECT

public:
    Dialog(QWidget *parent = nullptr);
    ~Dialog();

private:
    Ui::Dialog *ui;
    QNetworkAccessManager networkManager; //网络管理
    QNetworkReply *reply; //网络响应
    QFile *downloadedFile; //下载保存的临时文件
private slots:
```

```
    void do_finished();
    void do_readyRead();
    void do_downloadProgress(qint64 bytesRead, qint64
totalBytes);
    void on_btn_defPath_clicked();
    void on_btn_download_clicked();
};
#endif // DIALOG_H
```

```
#include "dialog.h"
#include "ui_dialog.h"
#include <QDir>
#include <QMessageBox>
#include <QDebug>

Dialog::Dialog(QWidget *parent)
    : QDialog(parent)
    , ui(new Ui::Dialog)
{
    ui->setupUi(this);
    ui->progressBar->hide();
}

Dialog::~Dialog()
{
    delete ui;
}

void Dialog::do_finished()
{
    QFileInfo fileInfo;
    fileInfo.setFile(downloadedFile->fileName());

    downloadedFile->close();
    delete downloadedFile;
    downloadedFile = Q_NULLPTR;

    reply->deleteLater(); //
    reply = Q_NULLPTR;

}
```

```
void Dialog::do_readyRead()
{
    downloadedFile->write(reply->readAll());
}

void Dialog::do_downloadProgress(qint64 bytesRead, qint64
totalBytes)
{
    ui->progressBar->setMaximum(totalBytes);
    ui->progressBar->setValue(bytesRead);

}



void Dialog::on_btn_defPath_clicked()
{
    QString  curPath=QDir::currentPath();
    QDir    dir(curPath);

    QString  sub="tmp";
    dir.mkdir(sub);

    ui->le_savePath->setText(curPath+"/"+sub+"/");
}

void Dialog::on_btn_download_clicked()
{
    QString urlSpec = ui->le_url->text().trimmed();
    if (urlSpec.isEmpty())
    {
        QMessageBox::information(this, "错误",
          "请指定需要下载的URL");
        return;
    }

    QUrl newUrl = QUrl::fromUserInput(urlSpec);//URL地址
    if (!newUrl.isValid())
    {
        QMessageBox::information(this, "错误",
          QString("无效URL: %1 \n 错误信息: %2").arg(urlSpec,
newUrl.errorString()));
        return;
    }
```

```
    QString tempDir =ui->le_savePath->text().trimmed();//临时目录
    if (tempDir.isEmpty())
    {
        QMessageBox::information(this, tr("错误"), "请指定保存下载文
件的目录");
        return;
    }

    QString fullFileName =tempDir+newUrl.fileName(); //文件名

    if (QFile::exists(fullFileName))
        QFile::remove(fullFileName);

    downloadedFile =new QFile(fullFileName);//创建临时文件

    if (!downloadedFile->open(QIODevice::WriteOnly))
    {
        QMessageBox::information(this, tr("错误"),downloadedFile-
>errorString());
        return;
    }



    //发送网络请求，创建网络响应
    reply = networkManager.get(QNetworkRequest(newUrl));
    connect(reply, SIGNAL(finished()), this,
SLOT(do_finished()));
    connect(reply, SIGNAL(readyRead()), this,
SLOT(do_readyRead()));
    connect(reply, SIGNAL(downloadProgress(qint64,qint64)),
            this, SLOT(do_downloadProgress(qint64,qint64)));

    ui->progressBar->show();

}
```

9.多媒体编程

Qt Multimedia模块为多媒体编程提供支持。多媒体编程实现的功能注意包括播放音频和视频文件，通过麦克风录制音频，通过摄像头拍照和录像等。相比Qt5,Qt6的多媒体模块做了全新的改写，舍弃了一些类，也引入了一些新的类。所以Qt6中编译Qt5编写的多媒体程序基本是无法成功的。

Qt 6 多媒体模块是在 Qt 6.2 中才正式发布的。我们在前面安装Qt开发环境时选择的是Qt6.2.4版本。



在菜单“工具---》外部---》配置”，中也可以查看到我们使用的Qt版本。



Qt 6 多媒体模块在不同的平台上使用不同的后端，Linux上是 GStreamer，Windows 上是 WMF，macOS 和 iOS 上是 AVFoundation，Android 上是 Android 多媒体 API。

Qt 6 多媒体模块包含两个子模块：Qt Multimedia 模块提供了多媒体编程用到的大部分类；Qt Multimedia Widgets 模块提供了多媒体编程中用到的界面组件类，如播放视频时用于显示视频的界面组件类 QVideoWidget。要在项目中使用这两个模块，需要在项目配置文件（.pro 文件）中加入

```
QT += multimedia
QT += multimediawidgets
```

在程序文件中，如果需要用到这两个模块中的类，使用下面的包含（include）语句可以包含模块中的大部分类。

```
#include <QtMultimedia>
#include <QtMultimediaWidgets>
```

9.1 播放音频

QMediaPlayer 可以用于播放经过压缩的音频文件，如 MP3 文件和 WMA 文件。QSoundEffect 可以用于播放低延迟音效文件，例如无压缩的 WAV 文件。这两个类都可以用于播放本地文件和网络文件。

The QMediaPlayer class allows the playing of a media files. More...

Header:	#include <QMediaPlayer>
CMake:	find_package(Qt6 COMPONENTS Multimedia REQUIRED) target_link_libraries(mytarget PRIVATE Qt6::Multimedia)
qmake:	QT += multimedia
Instantiated By:	MediaPlayer
Inherits:	QObject

常用函数(包括SLOT)

```
//构造函数
QMediaPlayer::QMediaPlayer(QObject *parent = nullptr)
//设置一个音频输出设备
void setAudioOutput(QAudioOutput *output)
//设置播放媒介来源，本地文件或者网络文件
[slot] void QMediaPlayer::setSource(const QUrl &source)
//设置播放速度，1.0 表示正常速度
void setPlaybackRate(qreal rate)
void setLoops(int loops) //设置播放的循环次数
//返回当前播放器状态
QMediaPlayer::PlaybackState playbackState() const
QMediaMetaData metaData() //返回当前媒介的元数据
qint64 duration() //媒介的持续时间，单位为 ms
void setPosition(qint64 position) //设置当前的播放位置，单位为 ms
qint64 position() //返回当前的播放位置，单位为 ms
void play() //开始播放
void pause() //暂停播放
void stop() //停止播放
```

在创建一个 QMediaPlayer 对象后，必须先用函数 setAudioOutput()设置一个音频输出设备，再用函数 setSource()设置播放媒介来源（可以是本地文件或网络文件），然后就可以用函数 play() 开始播放了。使用 pause()和 stop()函数可以暂停和停止播放。

常用信号

```
void durationChanged(qint64 duration) //媒介的持续时间发生变化
void mediaStatusChanged(QMediaPlayer::MediaStatus status) //媒介状态发生变化
void metaDataChanged() //媒介的元数据发生变化
void playbackStateChanged(QMediaPlayer::PlaybackState newState) //播放器状态发生变化
void positionChanged(qint64 position) //播放位置发生变化
void sourceChanged(const QUrl &media) //媒介来源发生变化
```

QMediaPlayer 在开始、暂停或停止播放时，播放器状态发生变化，会发射 playbackStateChanged() 信号，函数 playbackState()会返回当前播放器状态。播放器状态用枚举类型 QMediaPlayer::PlaybackState 表示，有如下几种取值。

- QMediaPlayer::PlayingState，正在播放。
- QMediaPlayer::PausedState，暂停播放。
- QMediaPlayer::StoppedState，已停止播放。

媒介有元数据，函数 metaData()可以返回当前媒介的元数据，重新设置媒介时会发射 metaDataChanged()信号。媒介的元数据是 QMediaMetaData 类型数据，元数据用“key-value”形式的键值对表示，QMediaMetaData 主要有以下几个函数：

```
QList<QMediaMetaData::Key> keys() //返回键名称列表
QString stringValue(QMediaMetaData::Key key) //以字符串形式返回一个键的数据
QVariant value(QMediaMetaData::Key key) //以 QVariant 类型返回一个键的数据
```

常见枚举常量表示的元数据的类型和意义如下表所示：

枚举常量	类型	意义
Title	QString	媒介的标题
Author	QStringList	媒介的作者
Description	QString	对媒介的描述
FileFormat	QMediaFormat::FileFormat	媒介的文件格式
Duration	qint64	媒介的持续时间，单位为 ms
AudioBitRate	int	音频流的比特率
AudioCodec	QMediaFormat::AudioCodec 枚举类型	音频流的编码格式

枚举常量	类型	意义
VideoFrameRate	qreal	视频的帧率
VideoBitRate	int	视频的比特率
VideoCodec	QMediaFormat::VideoCodec 枚举类型	视频流的编码格式
ThumbnailImage	QImage	嵌入的专辑缩略图
AlbumTitle	QString	专辑标题

利用媒介的元数据，我们可以提取媒介的一些信息，例如对于 MP3 和 WMA 格式的音乐文件， 可以提取歌手（Author）和专辑图片（ThumbnailImage）等信息。

9.1.1 命令行的mp3播放器

```
#include <QApplication>
#include <QMediaPlayer>
#include <QAudioOutput>
#include <QDebug>

void playFinish(QMediaPlayer::PlaybackState state){
    if (state == QMediaPlayer::StoppedState) {
        qDebug() << "Media playback finished!";
    }
}
int main(int argc, char *argv[])
{
    QApplication a(argc, argv);

    QMediaPlayer *player = new QMediaPlayer;
    QAudioOutput *audioOutput= new QAudioOutput;
    player->setAudioOutput(audioOutput);
    player->setSource(QUrl::fromLocalFile("/tmp/maque.mp3"));
    player->audioOutput()->setVolume(50/100.0);

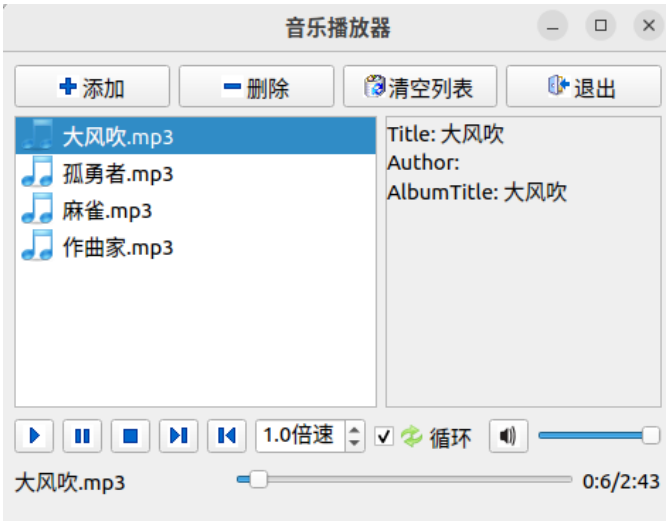
    QObject::connect(player, &QMediaPlayer::playbackStateChanged,
playFinish);

    player->play();

    return a.exec();
}
```

```
#工程文件中加入    QT += multimedia
#准备好 /tmp/maque.mp3
qmake
make
./player
```

9.1.2 带UI的mp3播放器




```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QListWidget>
#include <QMainWindow>
#include <QtMultimedia>

QT_BEGIN_NAMESPACE
namespace Ui { class MainWindow; }
QT_END_NAMESPACE

class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    MainWindow(QWidget *parent = nullptr);
    ~MainWindow();

private:
    Ui::MainWindow *ui;
    QMediaPlayer  *player;      //播放器
```

```
    bool    loopPlay=true;      //是否循环播放
    QString  durationTime;      //文件总长度，mm:ss字符串
    QString  positionTime;      //当前播放到位置，mm:ss字符串

    QUrl getUrlFromItem(QListWidgetItem *item);          //获取item
的用户数据
private slots:
    //自定义槽函数
    void do_stateChanged(QMediaPlayer::PlaybackState state);
 //播放器状态发生变化

    void do_sourceChanged(const QUrl &media);   //文件发生变化

    void do_durationChanged(qint64 duration);   //文件长度发生变化

    void do_positionChanged(qint64 position);   //播放位置发生变化

    void do_metaDataChanged();          //元数据发生变化
    void on_btnAdd_clicked();

    void on_btnStop_clicked();
    void on_btnNext_clicked();
    void on_btnPrev_clicked();
    void on_doubleSpinBox_valueChanged(double arg1);
    void on_btnMute_clicked();
    void on_listWidget_doubleClicked(const QModelIndex &index);
    void on_btnPlay_clicked();
    void on_btnPause_clicked();
    void on_cbCycle_clicked(bool checked);
    void on_btnDel_clicked();
    void on_btnClear_clicked();
    void on_btnExit_clicked();
    void on_sliderPos_sliderMoved(int position);
    void on_sdVolume_sliderMoved(int position);
};
#endif // MAINWINDOW_H
```

```
#include "mainwindow.h"
#include "ui_mainwindow.h"
#include <QFileDialog>
#include <QDebug>
```

```
QUrl MainWindow::getUrlFromItem(QListWidgetItem *item){
    QVariant itemData= item->data(Qt::UserRole);    //获取用户数据
    QUrl source =itemData.value<QUrl>();    //QVariant转换为QUrl类
型
    return source;
}
MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
    , ui(new Ui::MainWindow)
{
    ui->setupUi(this);

    player = new QMediaPlayer(this);
    QAudioOutput *audioOutput = new QAudioOutput(this);   //音频输
出，指向默认的音频输出设备
    player->setAudioOutput(audioOutput);    //设置音频输出
    connect(player,&QMediaPlayer::positionChanged,      //播放位置
发生变化
            this, &MainWindow::do_positionChanged);

    connect(player,&QMediaPlayer::durationChanged,      //播放源长
度发生变化
            this, &MainWindow::do_durationChanged);

    connect(player, &QMediaPlayer::sourceChanged,       //播放源发
生变化
            this, &MainWindow::do_sourceChanged);

    connect(player, &QMediaPlayer::playbackStateChanged,    //播放
器状态发生变化
            this,  &MainWindow::do_stateChanged);

    connect(player, &QMediaPlayer::metaDataChanged,     //元数据发
生变化
            this,  &MainWindow::do_metaDataChanged);
}

MainWindow::~MainWindow()
{
    delete ui;
}

void MainWindow::on_btnAdd_clicked()
```

```
{
    QString curPath=QDir::currentPath();  //获取系统当前目录
    QString dlgTitle="选择音频文件";
    QString filter="音频文件(*.mp3 *.wav *.wma);;所有文件(*.*)";
//文件过滤器
    QStringList
fileList=QFileDialog::getOpenFileNames(this,dlgTitle,curPath,filt
er);

    for (int i=0; i<fileList.size();i++)
    {
        QString  aFile=fileList.at(i);
        QFileInfo  fileInfo(aFile);
        QListWidgetItem *aItem =new
QListWidgetItem(fileInfo.fileName());
        aItem->setIcon(QIcon(":/images/music.png"));
        aItem->setData(Qt::UserRole, QUrl::fromLocalFile(aFile));
 //设置用户数据，QUrl对象
        ui->listWidget->addItem(aItem);
    }

    if (player->playbackState() != QMediaPlayer::PlayingState)
    {  //当前没有在播放，就播放第1个文件
        ui->listWidget->setCurrentRow(0);
        QUrl source= getUrlFromItem(ui->listWidget-
>currentItem());
        player->setSource(source);
    }
    player->play();
}
void MainWindow::do_stateChanged(QMediaPlayer::PlaybackState
state){

    //播放器状态变化时执行
    qDebug()<<"player status:"<< state;

    //播放完一曲后停止了，如果loopPlay为true，自动播放下一曲
    if (loopPlay && (state ==QMediaPlayer::StoppedState))
    {
        int count=ui->listWidget->count();
        int curRow=ui->listWidget->currentRow();
        curRow++;
        curRow= curRow>=count? 0:curRow;
```

```
        ui->listWidget->setCurrentRow(curRow);
        player->setSource(getUrlFromItem(ui->listWidget-
>currentItem()));
        player->play();
    }

}

void MainWindow::do_sourceChanged(const QUrl &media){
    //播放的文件发生变化时的响应
    ui->lbMusicName->setText(media.fileName());
}

void MainWindow::do_durationChanged(qint64 duration){
    //播放源时长变化时执行，更新进度显示
    ui->sliderPos->setMaximum(duration);

    int   secs=duration/1000;  //秒
    int   mins=secs/60;        //分钟
    secs=secs % 60;            //余数秒
    durationTime=QString::asprintf("%d:%d",mins,secs);
    ui->lbMusicPos->setText(positionTime+"/"+durationTime);
}

void MainWindow::do_positionChanged(qint64 position){

    //播放位置变化时执行，更新进度显示
    if (ui->sliderPos->isSliderDown())     //滑条正被鼠标拖动
        return;

    ui->sliderPos->setSliderPosition(position);
    int   secs=position/1000;   //秒
    int   mins=secs/60;         //分钟
    secs=secs % 60;             //余数秒
    positionTime=QString::asprintf("%d:%d",mins,secs);
    ui->lbMusicPos->setText(positionTime+"/"+durationTime);
}

void MainWindow::do_metaDataChanged(){

    qDebug()<<"metaDataChanged";
#if 1
    //元数据变化时执行，显示歌曲图片
```

```
    QMediaMetaData metaData=player->metaData();     //元数据对象


    QVariant  metaTitle = metaData.value(QMediaMetaData::Title);
    QVariant  metaAuthor =
metaData.value(QMediaMetaData::Author);
    QVariant  metaAlbumTitle =
metaData.value(QMediaMetaData::AlbumTitle);

    QString content = QString("Title: %1\nAuthor: %2\nAlbumTitle:
%3").arg(metaTitle.value<QString>())
                            .arg(metaAuthor.value<QString>
()).arg(metaAlbumTitle.value<QString>());

    ui->lbContent->setText(content);


#endif
}

void MainWindow::on_btnStop_clicked()
{
    player->stop();
}


void MainWindow::on_btnNext_clicked()
{
    //下一曲
    int count=ui->listWidget->count();
    int curRow=ui->listWidget->currentRow();
    curRow++;
    curRow= curRow>=count? count-1:curRow;
    ui->listWidget->setCurrentRow(curRow);

    loopPlay=false;     //暂时设置为false，防止do_stateChanged()里切
换曲目
    player->setSource(getUrlFromItem(ui->listWidget-
>currentItem()));
    player->play();
    loopPlay=ui->cbCycle->isChecked();
}
```

```
void MainWindow::on_btnPrev_clicked()
{
    //前一曲
    int curRow=ui->listWidget->currentRow();
    curRow--;
    curRow= curRow<0? 0:curRow;
    ui->listWidget->setCurrentRow(curRow);  //设置当前行

    loopPlay=false;     //暂时设置为false，防止do_stateChanged()里切换曲目
    player->setSource(getUrlFromItem(ui->listWidget->currentItem()));
    player->play();
    loopPlay=ui->cbCycle->isChecked();
}


void MainWindow::on_doubleSpinBox_valueChanged(double arg1)
{
    //"倍速" DoubleSpinbox
    player->setPlaybackRate(arg1);
}

void MainWindow::on_btnMute_clicked()
{
    bool mute=player->audioOutput()->isMuted();
    player->audioOutput()->setMuted(!mute);
    if (mute)
        ui->btnMute->setIcon(QIcon(":/images/volumn.bmp"));
    else
        ui->btnMute->setIcon(QIcon(":/images/mute.bmp"));
}

void MainWindow::on_listWidget_doubleClicked(const QModelIndex &index)
{
    Q_UNUSED(index);
    loopPlay=false;     //暂时设置为false，防止do_stateChanged()里切换曲目
    player->setSource(getUrlFromItem(ui->listWidget->currentItem()));
    player->play();
```

```
    loopPlay=ui->cbCycle->isChecked();

}


void MainWindow::on_btnPlay_clicked()
{
    //开始播放
    if (ui->listWidget->currentRow()<0)   //没有选择文件，就播放第1个
        ui->listWidget->setCurrentRow(0);
    player->setSource(getUrlFromItem(ui->listWidget-
>currentItem()));
    player->play();
    loopPlay=ui->cbCycle->isChecked();  //是否循环播放
}


void MainWindow::on_btnPause_clicked()
{
    player->pause();
}


void MainWindow::on_cbCycle_clicked(bool checked)
{
    loopPlay = checked;
}


void MainWindow::on_btnDel_clicked()
{
    //"移除"按钮，移除列表中的当前项
    int index =ui->listWidget->currentRow();
    if (index>=0)
    {
        QListWidgetItem *item= ui->listWidget->takeItem(index);
        delete item;
    }
}


void MainWindow::on_btnClear_clicked()
{
```

```
    loopPlay=false;     //防止do_stateChanged()里切换曲目
    ui->listWidget->clear();
    player->stop();
}


void MainWindow::on_btnExit_clicked()
{
    close();
}


void MainWindow::on_sliderPos_sliderMoved(int position)
{
    //qDebug()<<"pos moved";


    player->setPosition(position);
}


void MainWindow::on_sdVolume_sliderMoved(int position)
{
     player->audioOutput()->setVolume(position/100.0);
}
```

9.2 播放视频

QMediaPlayer 类不仅能播放音频文件，也可以播放各种常见的视频文件，如 MP4 文件和 WMV文件。QMediaPlayer 能对视频文件进行解码，并在某个界面组件上显示视频帧。

我们在 9.1 节介绍过 QMediaPlayer 的一些接口函数，有些函数对于播放视频也同样适用， 例如函数 setSource()也可用于设置视频文件。QMediaPlayer 类与播放视频相关的几个接口函数的定义如下：

```
void setVideoOutput(QObject *) //设置用于显示视频的界面组件
QObject *videoOutput() //返回显示视频的界面组件
```

要使用 QMediaPlayer 播放视频，必须用函数 setVideoOutput()设置用于显示视频的界面组件，有 QVideoWidget 和QGraphicsVideoItem 两种显示视频的组件：QVideoWidget 是 QWidget 的子类， 是普通的界面组件；QGraphicsVideoItem 是适用于图形/视图架构的图形项。还需要用函数

setAudioOutput()设置音频输出通道，否则播放视频时会没有声音。

注意：开始以下实验时ubuntu系统需要安装播放mp4的软件：

```
sudo apt install libdvdnav4 libdvd-pkg gstreamer1.0-plugins-bad
gstreamer1.0-plugins-ugly libdvd-pkg
```




```
#工程文件中加入
QT       += core gui multimedia   multimediawidgets
```

```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>
#include <QtMultimedia>

QT_BEGIN_NAMESPACE
namespace Ui { class MainWindow; }
QT_END_NAMESPACE

class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
```

```
    MainWindow(QWidget *parent = nullptr);
    ~MainWindow();

private:
    Ui::MainWindow *ui;
    QMediaPlayer  *player;  //视频播放器
    QString  durationTime;
    QString  positionTime;
private slots:
    //自定义槽函数
    void do_stateChanged(QMediaPlayer::PlaybackState state);
    void do_durationChanged(qint64 duration);
    void do_positionChanged(qint64 position);

    void on_btnOpen_clicked();
    void on_btnPlay_clicked();
    void on_btnPause_clicked();
    void on_btnStop_clicked();
    void on_btnMute_clicked();

    void on_btnFullScreen_clicked();

    void on_sliderVolumn_valueChanged(int value);


    void on_sliderProgress_sliderMoved(int position);
};
#endif // MAINWINDOW_H
```

```
#include "mainwindow.h"
#include "ui_mainwindow.h"

#include    <QFileDialog>


MainWindow::MainWindow(QWidget *parent) :
    QMainWindow(parent),
    ui(new Ui::MainWindow)
{
    ui->setupUi(this);

    player = new QMediaPlayer(this);        //创建视频播放器
```

```
    QAudioOutput *audioOutput= new QAudioOutput(this);
    player->setAudioOutput(audioOutput);        //设置音频输出通道
    player->setVideoOutput(ui->videoWidget);    //设置视频显示组件

    ui->videoWidget->setMediaPlayer(player);//设置显示组件的关联播放
器

    connect(player,&QMediaPlayer::playbackStateChanged,this,
&MainWindow::do_stateChanged);

    connect(player,&QMediaPlayer::positionChanged,this,
&MainWindow::do_positionChanged);

    connect(player,&QMediaPlayer::durationChanged,this,
&MainWindow::do_durationChanged);
}

MainWindow::~MainWindow()
{
    delete ui;
}

void MainWindow::do_stateChanged(QMediaPlayer::PlaybackState
state)
{//播放器状态变化
    bool isPlaying = (state==QMediaPlayer::PlayingState);
    ui->btnPlay->setEnabled(!isPlaying);
    ui->btnPause->setEnabled(isPlaying);
    ui->btnStop->setEnabled(isPlaying);
}

void MainWindow::do_durationChanged(qint64 duration)
{

    //文件时长变化
    ui->sliderProgress->setMaximum(duration);

    int   secs=duration/1000;   //秒
    int   mins=secs/60;         //分钟
    secs=secs % 60;             //余数秒
    durationTime=QString::asprintf("%d:%d",mins,secs);
    ui->lbProgress->setText(positionTime+"/"+durationTime);
```

```
}

void MainWindow::do_positionChanged(qint64 position)
{

    //文件播放位置变化
    if (ui->sliderProgress->isSliderDown())
        return;     //如果正在拖动滑条，退出
    ui->sliderProgress->setValue(position);
    int   secs=position/1000;   //秒
    int   mins=secs/60;         //分钟
    secs=secs % 60;             //余数秒
    positionTime=QString::asprintf("%d:%d",mins,secs);
    ui->lbProgress->setText(positionTime+"/"+durationTime);

}



void MainWindow::on_btnPlay_clicked()
{
    //播放
    player->play();
}

void MainWindow::on_btnPause_clicked()
{
    //暂停
    player->pause();
}

void MainWindow::on_btnStop_clicked()
{
    //停止
    player->stop();
}

void MainWindow::on_sliderVolumn_valueChanged(int value)
{

    //调节音量
    player->audioOutput()->setVolume(value/100.0);
}
```

```
void MainWindow::on_btnOpen_clicked()
{
    //打开文件
    QString curPath=QDir::currentPath();
    QString dlgTitle="选择视频文件";
    QString filter="视频文件(*.wmv, *.mp4);;所有文件(*.*)";
    QString
aFile=QFileDialog::getOpenFileName(this,dlgTitle,curPath,filter);

    if (aFile.isEmpty())
        return;

    QFileInfo   fileInfo(aFile);
    ui->lbMediaName->setText(fileInfo.fileName());

    player->setSource(QUrl::fromLocalFile(aFile));  //设置播放文件
    player->play();
}


void MainWindow::on_btnMute_clicked()
{
    //静音按钮
    bool mute=player->audioOutput()->isMuted();
    player->audioOutput()->setMuted(!mute);
    if (mute)
        ui->btnMute->setIcon(QIcon(":/images/volumn.bmp"));
    else
        ui->btnMute->setIcon(QIcon(":/images/mute.bmp"));

}

void MainWindow::on_btnFullScreen_clicked()
{

    //全屏按钮
    ui->videoWidget->setFullScreen(true);

}
```

```
void MainWindow::on_sliderProgress_sliderMoved(int position)
{
    player->setPosition(position);
}
```

为了实现退出全屏功能， 按任意键实现暂停效果，从 QVideoWidget 继承并 自定义了一个类 TMyVideoWidget。

```
#ifndef TMYVIDEOWIDGET_H
#define TMYVIDEOWIDGET_H

#include <QVideoWidget>
#include <QtMultimedia>

class TMyVideoWidget : public QVideoWidget
{
private:
    QMediaPlayer    *m_player;

protected:
    void keyPressEvent(QKeyEvent *event);

    void mousePressEvent(QMouseEvent *event);

public:
    TMyVideoWidget(QWidget *parent =nullptr);

    void    setMediaPlayer(QMediaPlayer *player);
};

#endif // TMYVIDEOWIDGET_H
```

```
#include    "tmyvideowidget.h"
#include    <QKeyEvent>
#include    <QMouseEvent>

void TMyVideoWidget::keyPressEvent(QKeyEvent *event)
{//按键事件处理函数，ESC退出全屏状态
    if ((event->key() == Qt::Key_Escape)&&(isFullScreen()))
    {
        setFullScreen(false);       //退出全屏状态
        event->accept();
        QVideoWidget::keyPressEvent(event);
```

```
    }
}

void TMyVideoWidget::mousePressEvent(QMouseEvent *event)
{//鼠标事件处理函数，单击时暂停或继续播放

    if (event->button()==Qt::LeftButton)
    {
        if (m_player->playbackState()==QMediaPlayer::PlayingState)
            m_player->pause();
        else
            m_player->play();
    }
    QVideoWidget::mousePressEvent(event);
}

TMyVideoWidget::TMyVideoWidget(QWidget *parent):QVideoWidget(parent)
{

}

void TMyVideoWidget::setMediaPlayer(QMediaPlayer *player)
{//设置播放器
    m_player=player;
}
```

10.项目的发布

10.1 window10系统中项目的发布

一个 Qt 项目被以 Release 模式构建后会生成可执行文件，如果我们在 Windows 的资源管理器里直接双击这个可执行文件，系统会提示找不到一些 DLL 文件，程序无法运行。这是因为用 Qt开发的应用程序在运行时需要用到 Qt 运行库（run-time libraries），而这些库的路径并没有被添加到系统的搜索路径里。解决方式之一是将 Qt 运行库的路径添加到系统的 PATH 环境变量里，例如

Qt 6.2.4 MinGW 64-bit 的运行库的路径是：

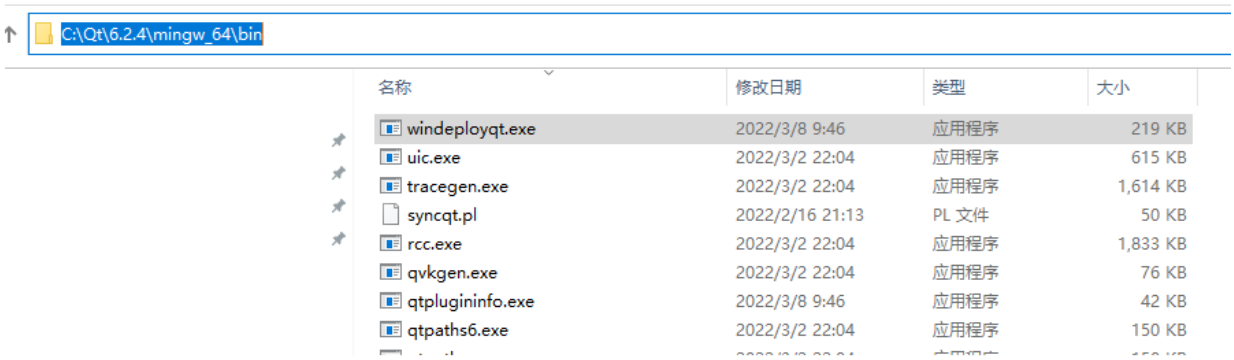
```
C:\Qt\6.2.4\mingw_64\bin
```

如果要将使用 Qt 开发的应用程序发布给用户，在其他计算机上运行，就需要将 Qt 的运行库随应用程序一起发布给用户，因为用户计算机上不一定安装了 Qt。所以发布应用程序的第一步就是提取应用程序需要的 Qt 运行库文件，Qt 提供了相关的工具软件来完成这个工作。

提取了 Qt 运行库文件后，将应用程序和运行库直接复制到用户计算机上就可以运行。如果要稍微正式一点，可以制作一个安装文件，有很多制作安装文件的工具软件。如果在安装 Qt 时选择了安装 Qt Installer Framework，我们也可以使用 Qt 自带的这个工具软件制作应用程序的安装文件。

10.1.1 使用windeployqt发布Qt 应用程序

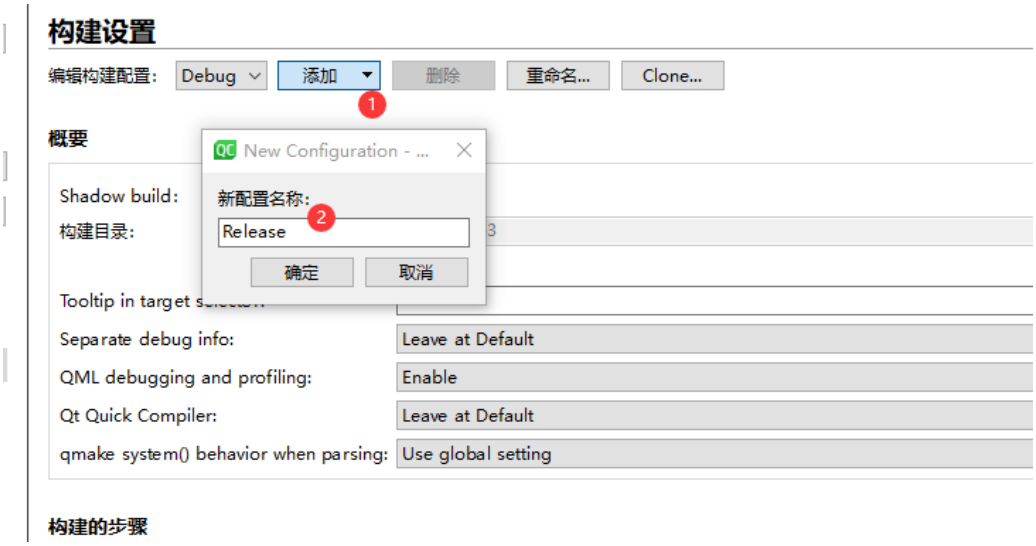
Qt 应用程序的发布就是提取应用程序相应开发套件的 Qt 运行库文件，使应用程序能直接在用户计算机上运行。windeployqt.exe 是 Qt 自带的 Windows 平台发布工具，它可以为 Qt 应用程序复制其运行所需要的各种库文件、插件和翻译文件，生成可发布的文件和目录。windeployqt.exe 在 Qt 开发套件的bin 目录下。



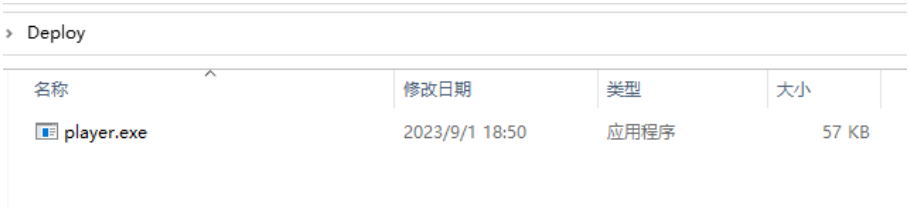
其语法格式如下所示：

```
windeployqt [options] [files]
    files, #是需要生成发布文件的应用程序文件名
    optins,
            --release 发布 Release 版本的二进制文件
            --no-translations 忽略翻译相关的文件
            --no-virtualkeyboard 忽略虚拟键盘相关的文件
```

- 编译生成Release版本的QT程序



- 将新构建处理的可执行程序放入新建目录

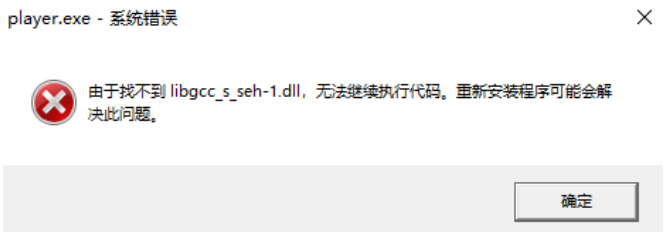


- 启动命令行终端执行

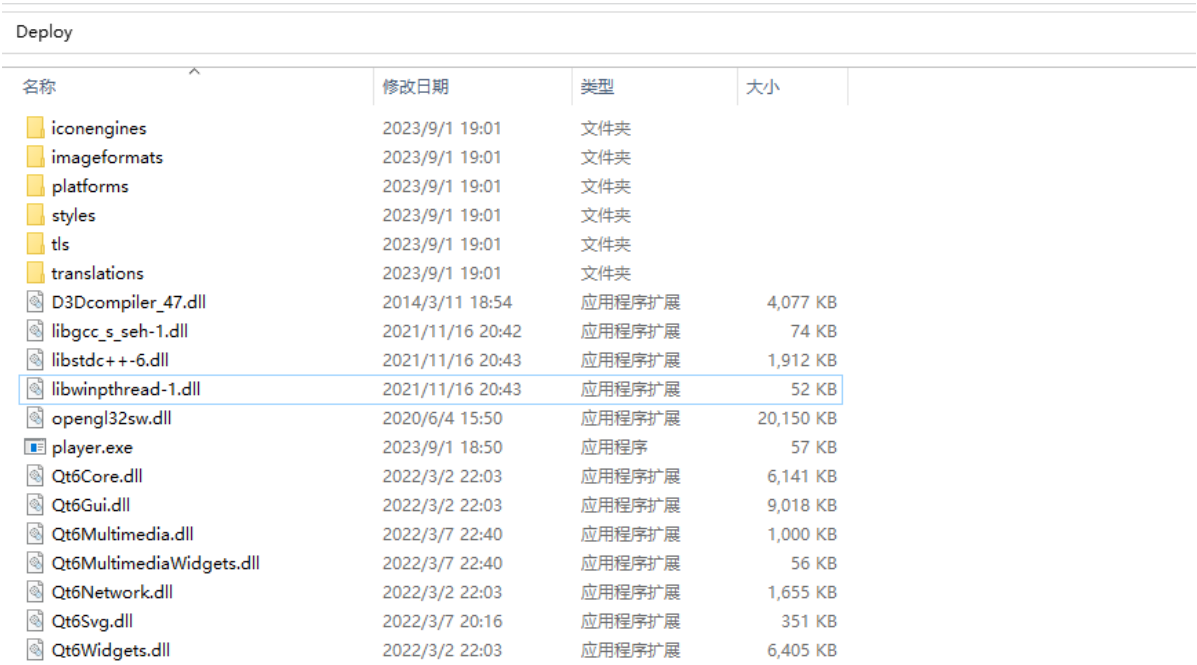
```
C:\Qt\6.2.4\mingw_64\bin\windeployqt  player.exe
```

- 拷贝缺少的DLL

  此时双击player.exe，运行报错



  将提示缺少的库文件从C:\Qt\6.2.4\mingw_64\bin拷贝到当前目录下来

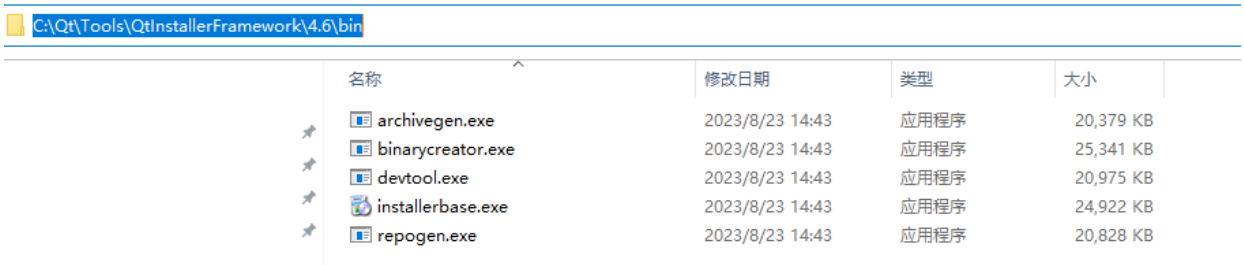


再次运行player.exe, 正常启动。

将整个文件夹复制到其他没有安装 Qt的计算机上也可以运行，如果不要求制作安装程序，这样就算完成了应用程序的发布。

10.1.2 制作安装文件

Qt Installer Framework（以下简称 QIF）是 Qt 提供的一个制作安装文件的工具，在安装 Qt 时可以选择安装 QIF。QIF 提供了一些工具软件，这些工具软件存储在如下的目录里。



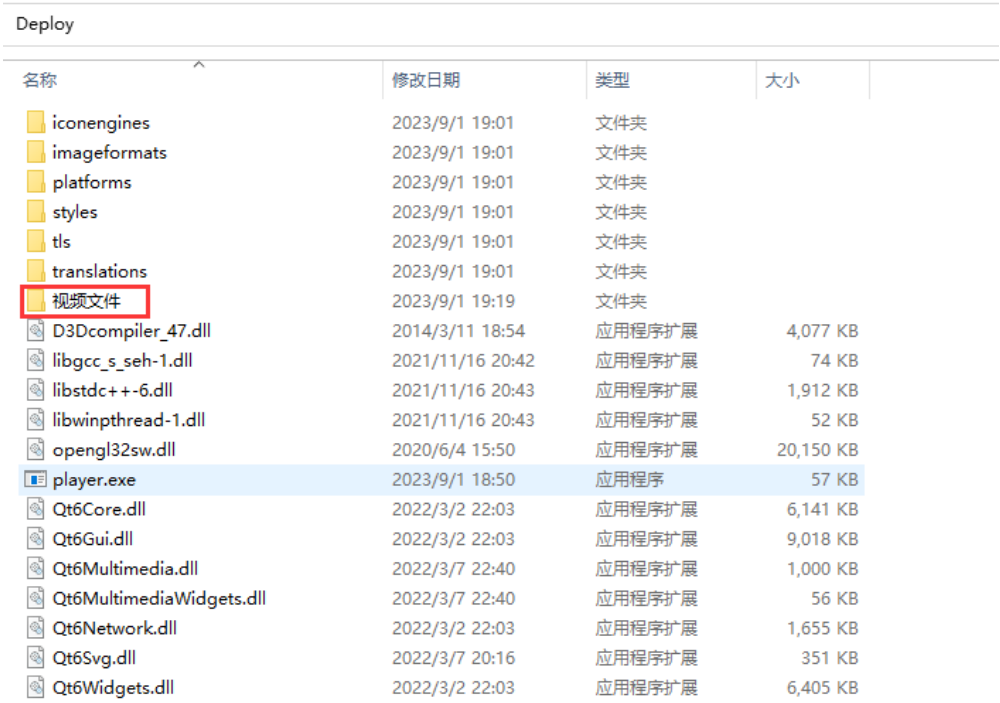
QIF 可以为任何需要发布的内容制作安装文件，而不仅是为 Qt 开发的应用软件制作安装文件。QIF 具有如下功能和特点。

- 使用 QIF 可以制作离线安装文件，也可以制作在线安装文件。
- 安装的内容可以划分为多个模块和层级，并且可以设置依赖性，安装过程中可以选择安装模块。
- 安装向导的定制性很强，可以使用自定义 UI 文件，可以通过脚本程序添加交互操作功能。
- 可以用多语言开发工具 lupdate 和 lrelease 生成翻译文件，使安装向导具有本地化语言界面。

- 用 QIF 生成的安装文件在安装时，会自动安装一个工具软件 maintenancetool.exe，运行它可以添加、移除、更新或完全卸载软件。

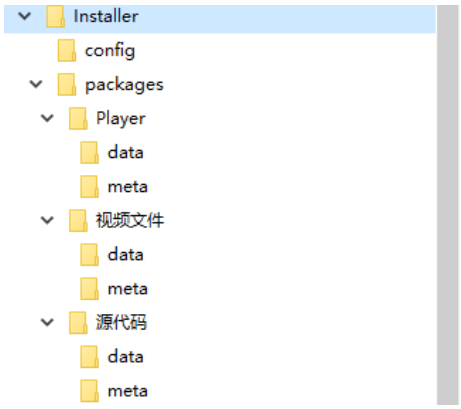
接下来我们演示如何使用QIF制作安装文件。

- 在上一节使用的发布里添加一个文件夹，存放任意多个mp4文件



- 为了使用 QIF 制作安装项目，我们再新建一个目录Installer
- 在Installer目录下创建一个名称为PlayerInstaller.pro的文件
- 在此目录下创建文件夹config和packages, 这两个目录名字固定不能修改

  config 文件夹里一般只有一个文件 config.xml，包含整个安装程序的一些配置内容。packages文件夹里是需要安装的组件，组件就是可选的安装内容，每个组件是一个文件夹。例如，本示例中有 3 个组件，对应“Player”、“视频文件”和“源代码”3 个文件夹，这 3 个文件夹下面各有两个固定名称的文件夹：data 和 meta。



- 修改 PlayerInstaller.pro 文件，写入如下内容

```
TEMPLATE = aux     #项目模板类型

INSTALLER = VideoPlayer_installer     #生成的安装文件名，
VideoPlayer_installer.exe

INPUT = $$PWD/config/config.xml $$PWD/packages
demo.input = INPUT
demo.output = $$INSTALLER
demo.commands =
C:/Qt/Tools/QtInstallerFramework/4.6/bin/binarycreator -c
$$PWD/config/config.xml -p $$PWD/packages ${QMAKE_FILE_OUT}
demo.CONFIG += target_predeps no_link combine

QMAKE_EXTRA_COMPILERS += demo
```

- 修改安装配置文件 config.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<Installer>
    <Name>Video Player</Name>
    <Version>1.0.0</Version>
    <Title>Video Player Created by Qt 6</Title>
    <Publisher>WWB</Publisher>
    <StartMenuDir>Qt6 Samples</StartMenuDir>
    <TargetDir>@HomeDir@/Qt6Samples</TargetDir>
    <CreateLocalRepository>true</CreateLocalRepository>
    <WizardStyle>Aero</WizardStyle>
    <WizardShowPageList>true</WizardShowPageList>
        <WizardDefaultWidth> 650   </WizardDefaultWidth>
        <WizardDefaultHeight> 430  </WizardDefaultHeight>
</Installer>
```

这是一个 XML 文件，定义了各种元素的值。<Name>定义了应用程序的名称，<Title>定义了 安装向导的标题，<StartMenuDir>定义了在开始菜单中创建的文件夹名称。<TargetDir>定义了初始的安装文件夹，其中“@HomeDir@”表示使用

QIF 中的预定义变量 HomeDir，也就是用户主目录。
<CreateLocalRepository>设置是否要建立本地仓库，若设置为 true，则完成安装后，用户可以运行维护工具来添加或删除组件。 <WizardStyle>设置安装向导的样式，有 Modern、Mac、Aero 和 Classic 几种样式，默认是 Aero 样式 。 <WizardShowPageList> 设置是否显示窗口左侧的向导页面列表，默认是显示的。

<WizardDefaultWidth>设置向导窗口默认宽度，<WizardDefaultHeight>设置向导窗口默认高度。

文件 config.xml 中可设置的元素的全部定义可参见 QIF 的帮助文档。

- 配置组件

目录\packages 下是需要安装的组件，每个组件是一个文件夹，本示例中有 3 个组件，每个组件下有 data 和 meta 两个固定名称的文件夹，data 文件夹里是需要安装到用户计算机上的文件，meta 文件夹里是对组件进行配置的文件。

- Player 组件，将Deploy目录下除了“视频文件”目录外，所有的子目录、文件压缩为 一个文件 player.7z，然后将其复制到\packages\Player\data 目录下。注意，必须压缩为 7z 格 式，不能使用 zip.rar 等其他格式。安装时，文件 player.7z 的内容会被自动解压到安装目录里。（笔者在window系统中安装了7-Zip软件）。

  一个组件的 meta 目录下必须有一个文件 package.xml，这是组件的配置文件。另外两个文件是根据需要创建的，文件 license.txt 里是安装过程中显示的许可协议内容，文件 installscript.qs 是一个 Qt 脚本文件，用于为安装后的文件 MusicPlayer.exe 创建开始菜单快捷方式和桌面快捷方式。

  文件license.txt的内容如下：

  ```
  本软件采用Qt 6.2开源版本开发，遵循Qt开源版本使用的GPL v2/v3和LPGL v3等许可协议。本软件可以自由复制和分发，但需要遵守相应的开源许可协议。
  ```

  文件 package.xml 的内容如下：

  ```
  <?xml version="1.0"?>
  <Package>
      <DisplayName>Video Player</DisplayName>
      <Description>Video Player应用程序</Description>
      <Version>1.0.0</Version>
      <ReleaseDate>2023-09-10</ReleaseDate>
      <Licenses>
          <License name="GNU Public License Agreement"
  file="license.txt" />
      </Licenses>
      <ForcedInstallation>true</ForcedInstallation>
      <Script>installscript.qs</Script>
      <SortingPriority>100</SortingPriority>
  </Package>
  ```

<Licenses>定义了许可协议，文件 license.txt 中的内容就是许可协议。<ForcedInstallation>定义是否 强制安装该组件，强制安装的组件前面不会出现复选框。<Script>定义组件要使 用脚本文件 installscript.qs，通过脚本文件可以实现更多的功能。

文件installscript.qs内容如下：

```
function Component()
{
    // default constructor
}

Component.prototype.createOperations = function()
{
    // call default implementation to actually install
VideoPlayer.exe!
    component.createOperations();

    if (systemInfo.productType === "windows") {
        component.addOperation("CreateShortcut",
"@TargetDir@\\VideoPlayer.exe", "@StartMenuDir@\\Video
Player.lnk",
            "workingDirectory=@TargetDir@");

        component.addOperation("CreateShortcut",
"@TargetDir@\\VideoPlayer.exe", "@DesktopDir@\\Video
Player.lnk",
            "workingDirectory=@TargetDir@");
    }
}
```

这个脚本程序为安装后的文件 Player.exe 创建了开始菜单快捷方式和桌面快捷方式。在创建桌面快捷方式的代码中，第一个参数表示操作名称，“CreateShortcut”就表示创建快捷方式； 第二个参数表示源文件，也就是表示安装之后的文件 MusicPlayer.exe；第三个参数表示快捷方式 的名称，QIF 的预定义变量 DesktopDir 表示用户桌面目录；第四个参数表示程序工作路径。

- “视频文件”组件。将Deploy目录下除了“视频文件”目录压缩为video.7z,然后将其复制到\packages\视频文件\data 目录下，在\packages\视频文件\meta 目录下创建一个文件 package.xml，其内容如下：

```
<?xml version="1.0"?>
<Package>
    <DisplayName>视频文件</DisplayName>
    <Description>一个MP4文件用于测试</Description>
    <Version>1.0.0</Version>
    <ReleaseDate>2023-09-10</ReleaseDate>
    <Default>true</Default>
    <SortingPriority>90</SortingPriority>
</Package>
```

其中，<Default>定义这个组件是默认安装的，也就是图 18-13 中组件前面的复选框是默认勾选的。

- “源代码”组件。将对应的源代码压缩为文件 SourceCode.7z，然后将其复制到\packages\源代码\data 目录下，在\packages\源代码\meta 目录下创建一个文件 package.xml，其内容如下：

```
<?xml version="1.0"?>
<Package>
    <DisplayName>源代码</DisplayName>
    <Description>VideoPlayer的Qt项目源代码</Description>
    <Version>1.0.0</Version>
    <ReleaseDate>2023-09-10</ReleaseDate>
    <Default>true</Default>
    <SortingPriority>80</SortingPriority>
</Package>
```

- 编译生成安装文件

  准备好这些文件后，在Qt Creator中打开文件PlayerInstaller.pro，然后构建项目。项目构建成功后会生成文Player_installer.exe，这就是制作好的安装文件。运行这个文件就可以开始安装

  安装完成后会在开始菜单中创建文件夹 Qt6 Samples，为文件 Player.exe 创建开始菜单快 捷方式和桌面快捷方式。根目录下有一个文 件 maintenancetool.exe，这是 QIF 自动安装的一个文件，运行这个文件就可以打开安装向导，可以添加或删除组件，或完全卸载软件。

10.2 ubuntu系统中项目的发布

Qt程序打包发行一直是一个不大不小的问题，在windows下Qt官方提供了windeployqt工具，可以通过简单的cmd命令实现Qt程序的打包发布，绝大部分情况下能正常工作（使用msvc编译器的话有可能需要在目标机上安装vs运行库），而在linux中不知为何官方没有提供类似的工具（可能是因为linux发行版种类很多，

碎片化严重，dqt官方没有足够的人手针对每个平台进行维护？），这就需要我们自己动手了。

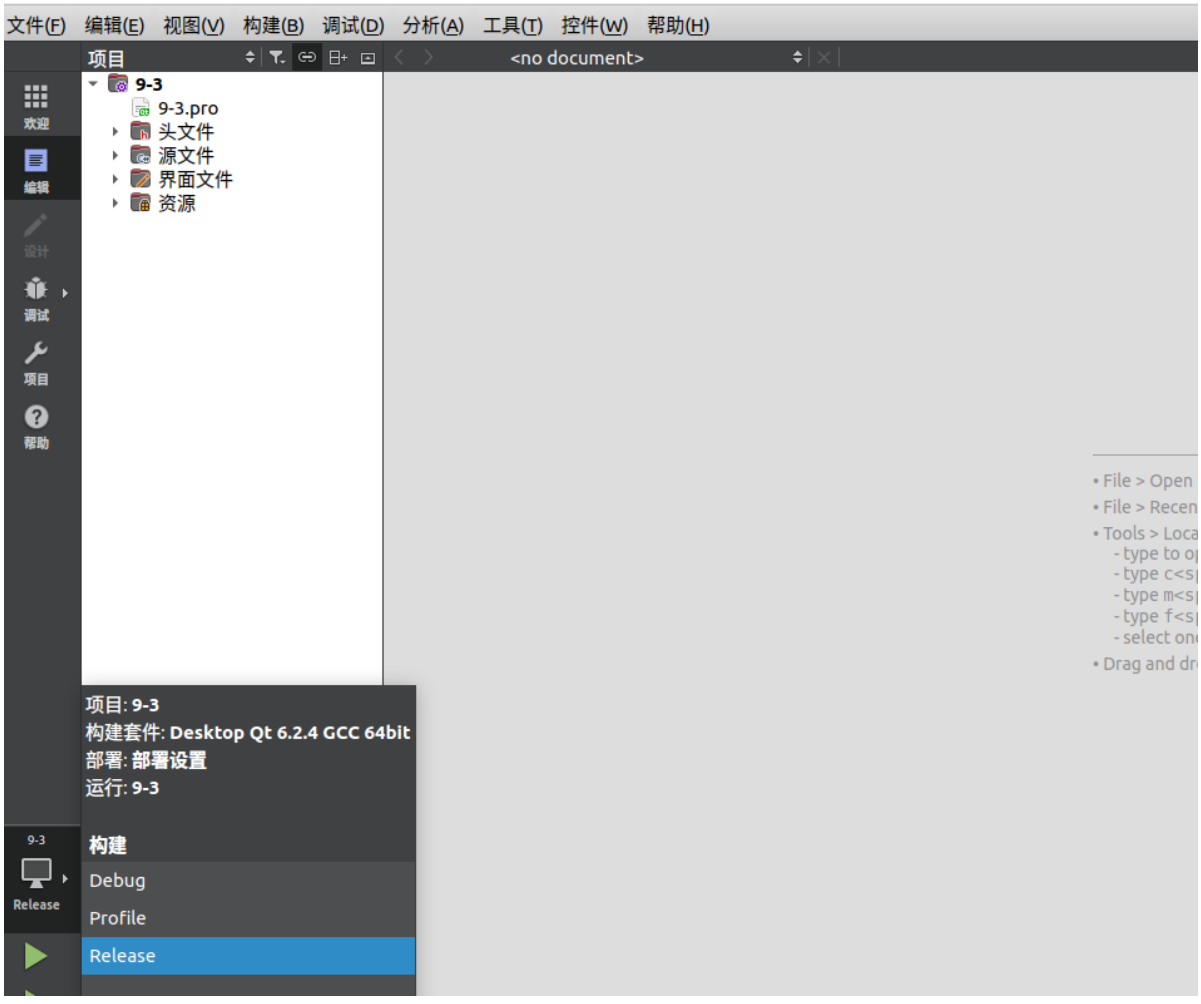
目前qt程序在Ubuntu上打包共有三种方法——**脚本法、静态编译法和linuxdeployqt工具法**.

不管使用哪种方式，其遵循的原则都是：

- 编程release版本可执行程序
- 找出可执行程序所有依赖的链接库
- 压缩或者打成deb包

下面我们通过自行编写脚本，来完成Qt项目的发布。（我们编写的脚本具有通用性，发布其它QT项目可以直接使用）

- 编译生成release版本的QT 程序



- 建立单独文件夹 将release版本可执行程序拷贝到该目录下

```
cd /tmp
mkdir release
cp ~/QtStudy/9-3/9-3 release/player
cd release
```

- 在系统中安装qt6版本的库文件

```
sudo apt install qt6-base-dev
sudo vim /usr/lib/x86_64-linux-gnu/qt-
default/qtchooser/default.conf
    /opt/Qt/6.2.4/gcc_64/bin/
    /usr/lib/x86_64-linux-gnu
```

- 在release目录下准备脚本 实现共享库的整理

```
vim prepare.sh
    #!/bin/bash
    LibDir=$PWD"/lib"
    Target=$1

    lib_array=($(ldd $Target | grep -o "/.*" | grep -o
"/.*/[^[:space:]]*"))

    $(mkdir $LibDir)

    for variable in ${lib_array[@]}
    do
        cp "$variable" $LibDir
    done
chmod  +x prepare.sh
./prepare.sh player
```

这个脚本**本质上是使用ldd命令查找程序依赖的动态库，再把动态库复制到程序目录**。

- 建立和可执行程序player同名的脚本问

```
vim player.sh
    #!/bin/sh
    appname=`basename $0 | sed s,\.sh$,,`
    dirname=`dirname $0`
    tmp="${dirname#?}"
    if [ "${dirname%$tmp}" != "/" ]; then
    dirname=$PWD/$dirname
    fi

    LD_LIBRARY_PATH=$dirname/lib
    export LD_LIBRARY_PATH
    $dirname/$appname "$@"
chmod +x player.sh
```

- 拷贝插件库到当前目录下

```
cp /opt/Qt/6.2.4/gcc_64/plugins/platforms/ ./ -a
```

- 验证测试
  - 本机测试

    ```
    ./player.sh
    ```

  - 其它未安装qt环境的ubuntu系统测试

    ```
    cd ..
    tar cjvf player.tar.bz2 release

    #拷贝到其它ubuntu系统后 执行解压运行
    tar xf player.tar.bz2
    release/player.sh
    #注意该机器上一定安装了相关的视频解码器才可以正常播放，若未安装，可以通过以下命令完成安装
    sudo apt install libdvdnav4 libdvd-pkg gstreamer1.0-plugins-bad gstreamer1.0-plugins-ugly libdvd-pkg
    ```